

尚硅谷大模型技术之 MySQL

（作者：尚硅谷研究院）

版本：V1.0

第 1 章 MySQL 概述

1.1 基本概念

1.1.1 数据库是什么？

数据库（DB: Database）：存储数据的地方。

1.1.2 为什么要用数据库？

应用程序产生的数据是在内存中的，如果程序退出或者是断电了，则数据就会消失。使用数据库是为了能够永久保存数据。当然这里指的是非内存数据库。

1.1.3 用普通文件存储行不行？

把数据写入到硬盘上的文件中，当然可以实现持久化的目标，但是不利于后期的检索和管理等。

1.1.4 MySQL、Oracle、SqlServer 是什么？

MySQL、Oracle、SqlServer 都是数据库管理系统（DBMS, Database Management System）是一种操纵和管理数据库的大型软件，例如建立、使用和维护数据库。

1.1.5 SQL 是什么？

SQL 是结构化查询语言（Structure Query Language），专门用来操作/访问数据库的通用语言。

1.2 MySQL 数据库管理系统

在互联网行业，MySQL 数据库毫无疑问已经是最常用的数据库。MySQL 数据库由瑞典 MySQL AB 公司开发。公司名中的“AB”是瑞典语“aktiebolag”股份公司的首字母缩写。该公司于 2008 年 1 月 16 号被 Sun（Stanford University Network）公司收购。然而 2009 年，SUN 公司又被 Oracle 收购。因此，MySQL 数据库现在隶属于 Oracle（甲骨文）公司。MySQL

中的“My”是其发明者（Michael Widenius，通常称为 Monty）根据其女儿的名字来命名的。对这位发明者来说，MySQL 数据库就仿佛是他可爱的女儿。

1.2.1 关系型数据库和非关系数据库

MySQL、Oracle、SqlServer 等是关系型数据库管理系统。

MongoDB、Redis、Elasticsearch 等是非关系型数据库管理系统。

关系型数据库，采用关系模型来组织数据，简单来说，关系模型指的就是二维表格模型。类似于 Excel 工作表。

非关系型数据库，可看成传统关系型数据库的功能阉割版本，基于键值对存储数据，通过减少不常用的功能来提高性能。

1.2.2 MySQL 的优点

- 可移植性：MySQL 数据库几乎支持所有的操作系统，如 Linux、Solaris、FreeBSD、Mac 和 Windows。

- 免费：MySQL 的社区版完全免费，一般中小型网站的开发都选择 MySQL 作为网站数据库。

- 开源：2000 年，MySQL 公布了自己的源代码，并采用 GPL（GNU General Public License）许可协议，正式进入开源的世界。开源意味着可以让更多人审阅和贡献源代码，可以吸纳更多优秀人才的代码成果。

- 关系型数据库：MySQL 可以利用标准 SQL 语法进行查询和操作。

- 速度快、体积小、容易使用：与其他大型数据库的设置和管理相比，其复杂程度较低，易于学习。MySQL 的早期版本（主要使用的是 MyISAM 引擎）在高并发下显得有些力不从心，随着版本的升级优化（主要使用的是 InnoDB 引擎），在实践中也证明了高压下的可用性。从 2009 年开始，阿里的“去 IOE”备受关注，淘宝 DBA 团队再次从 Oracle 转向 MySQL，其他使用 MySQL 数据库的公司还有 Facebook、Twitter、YouTube、百度、腾讯、去哪儿、魅族等等，自此，MySQL 在市场上占据了很大的份额。

- 安全性和连接性：十分灵活和安全的权限和密码系统，允许基于主机的验证。连接到服务器时，所有的密码传输均采用加密形式，从而保证了密码安全。由于 MySQL 是网络化的，因此可以在因特网上的任何地方访问，提高数据共享的效率。

- 丰富的接口：提供了用于 C、C++、Java、PHP、Python、Ruby 和 Eiffel、Perl 等语言的 API。

➤ **灵活:** MySQL 并不完美,但是却足够灵活,能够适应高要求的环境。同时,MySQL 既可以嵌入到应用程序中,也可以支持数据仓库、内容索引和部署软件、高可用的冗余系统、在线事务处理系统等各种应用类型。

➤ MySQL 最重要、最与众不同的特性是它的存储引擎架构,这种架构的设计将查询处理 (Query Processing) 及其他系统任务 (Server Task) 和数据的存储/提取相分离。这种处理和存储分离的设计可以在使用时根据性能、特性,以及其他需求来选择数据存储的方式。MySQL 中同一个数据库,不同的表格可以选择不同的存储引擎。其中使用最多的是 InnoDB 和 MyISAM,MySQL5.5 之后 InnoDB 是默认的存储引擎。

1.2.3 MySQL 不同版本

针对不同用户,MySQL 提供三个不同的版本:

➤ **MySQL Enterprise Server (企业版):** 能够以更高的性价比为企业提供数据仓库应用,该版本需要付费使用,官方提供电话技术支持。

➤ **MySQL Cluster (集群版):** MySQL 集群是 MySQL 适合于分布式计算环境的高可用、高冗余版本。它采用了 NDB Cluster 存储引擎,允许在 1 个集群中运行多个 MySQL 服务器。它不能单独使用,需要在社区版或企业版基础上使用。

➤ **MySQL Community Server (社区版):** 在开源 GPL 许可证之下可以自由的使用。该版本完全免费,但是官方不提供技术支持。本书是基于社区版讲解和演示的。在 MySQL 社区版开发过程中,同时存在多个发布系列,每个发布处在不同的成熟阶段。

1.3 表的关系

在关系数据库管理系统中,很多表之间是有关系的,表之间的关系分为一对一关系、一对多关系和多对多关系。

1.3.1 一对一

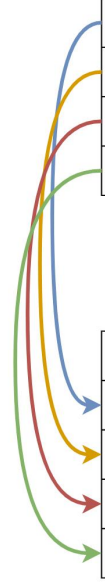
该关系中第一个表中的一个行只可以与第二个表中的一个行相关,且第二个表中的一个行也只可以与第一个表中的一个行相关。

例如,“员工基本信息表”和“员工紧急情况联系信息表”。“员工基本信息表”中存储的是频繁使用的信息,“员工紧急情况联系信息表”中存储的是不常用的信息,这两个表中的一条记录都代表一个员工的信息。“员工基本信息表”中的一条记录在“员工紧急情况

联系信息表”中只能找到唯一的一条对应记录，反过来也一样，即它们是一一对应关系。这两个表存在相同意义的“员工编号”字段，使它们建立了一对一关系。

员工编号	姓名	性别	手机号	职位编号	部门编号
S1001	张三	男	13789098765	J1001	D1001
S1002	李四	男	18678973456	J1002	D1001
S1003	如意	女	18945678986	J1003	D1002
S1004	吉祥	女	13576543210	J1004	D1002

员工编号	第一紧急联系人	关系	第一紧急联系人电话	员工住址
S1001	张才	父亲	18245678952	北京市回龙观
S1002	李双	母亲	13756824566	北京市平西府
S1003	如来	哥哥	13547582956	北京市宏福苑
S1004	吉吉	弟弟	17245675554	北京市白庙村



1.3.2 一对多

第一个表中的一个行可以与第二个表中的一个或多个行相关，但第二个表中的一个行只可以与第一个表中的一个行相关。

例如，“部门表”和“员工基本信息表”。“部门表”中的一条记录，在“员工基本信息表”中可以找到一条或多条记录对应，但反过来“员工基本信息表”中的一条记录在“部门表”中只能找到一条记录对应，即一个部门可以有多个员工，但是一个员工只能属于一个部门。这两个表存在相同意义的“部门编号”字段，使它们建立了一对多关系。



1.3.3 多对多

该关系中第一个表中的一个行可以与第二个表中的一个或多个行相关。第二个表中的一个行也可以与第一个表中的一个或多个行相关。通常两个表的多对多关系会借助第三张表，转换为两个一对多的关系。

例如，项目系统的“员工基本信息表”和“项目信息表”是多对多关系。一个员工可以参与多个项目，一个项目也可以有多个员工参与，即“员工基本信息表”中一条记录可以与“项目信息表”多条记录对应，反过来“项目信息表”的一条记录也可以与“员工基本信息表”中多条记录对应。它们之间借助第三张“员工项目表”实现关联关系，而“员工基本信息表”与“员工项目表”是一对多关系，“项目信息表”与“员工项目表”也是一对多关系。“员工项目表”中“员工编号”字段与“员工基本信息表”中“员工编号”字段意义相同。“员工项目表”中“项目编号”字段与“项目信息表”中“项目编号”字段意义相同。



第 2 章 MySQL 安装

2.1 MySQL 卸载

2.1.1 停止 MySQL 服务

Win + R，输入 services.msc 打开服务面板。找到 MySQL 的服务，右键停止。

2.1.2 卸载软件

1) 方式一：控制面板卸载

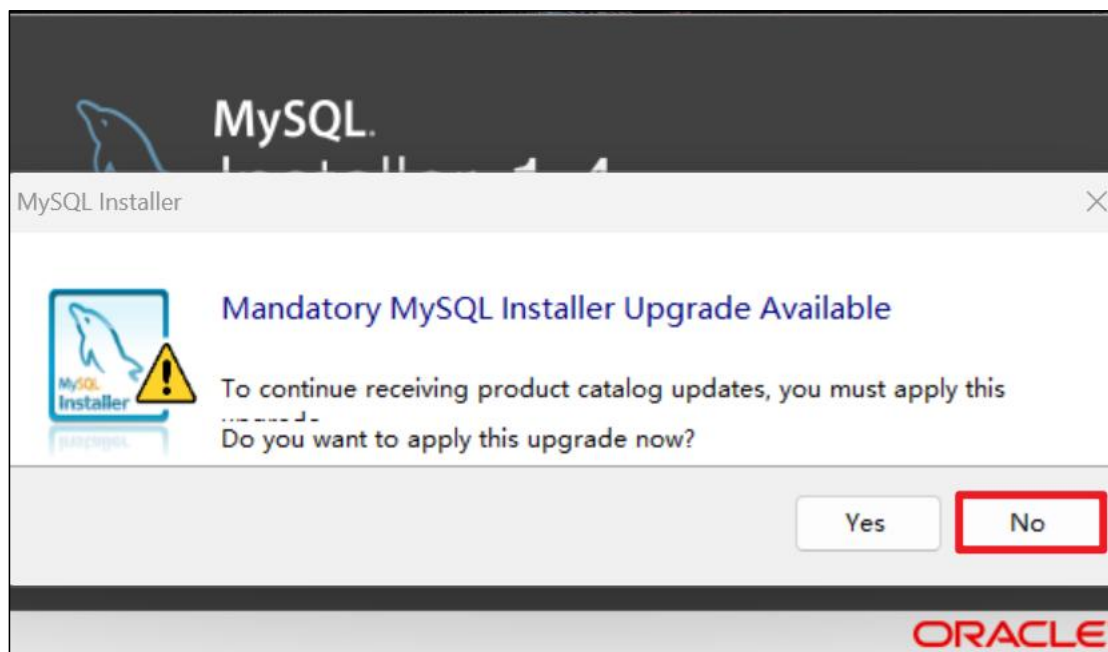


2) 方式二：MySQL8 安装向导卸载

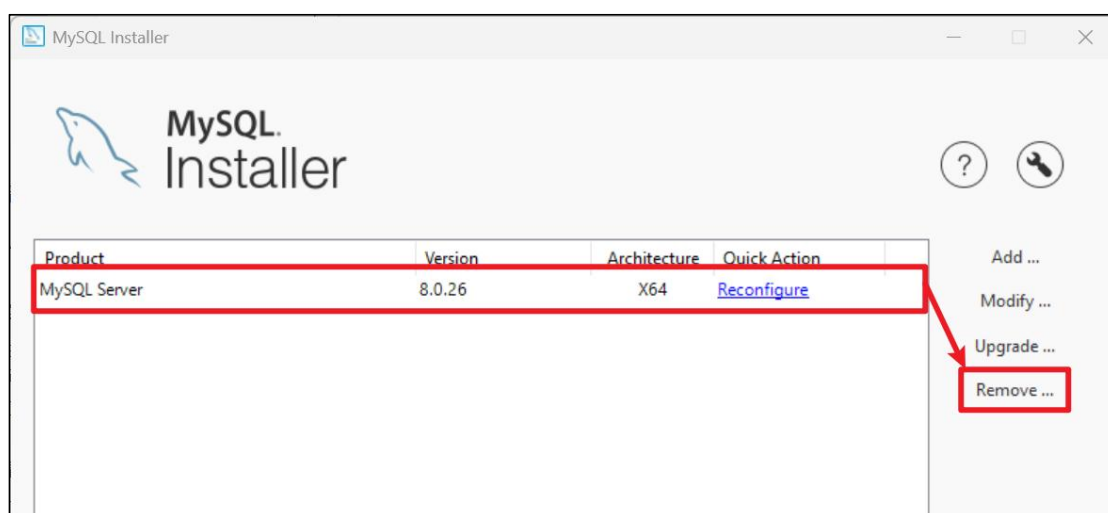
(1) 双击安装向导



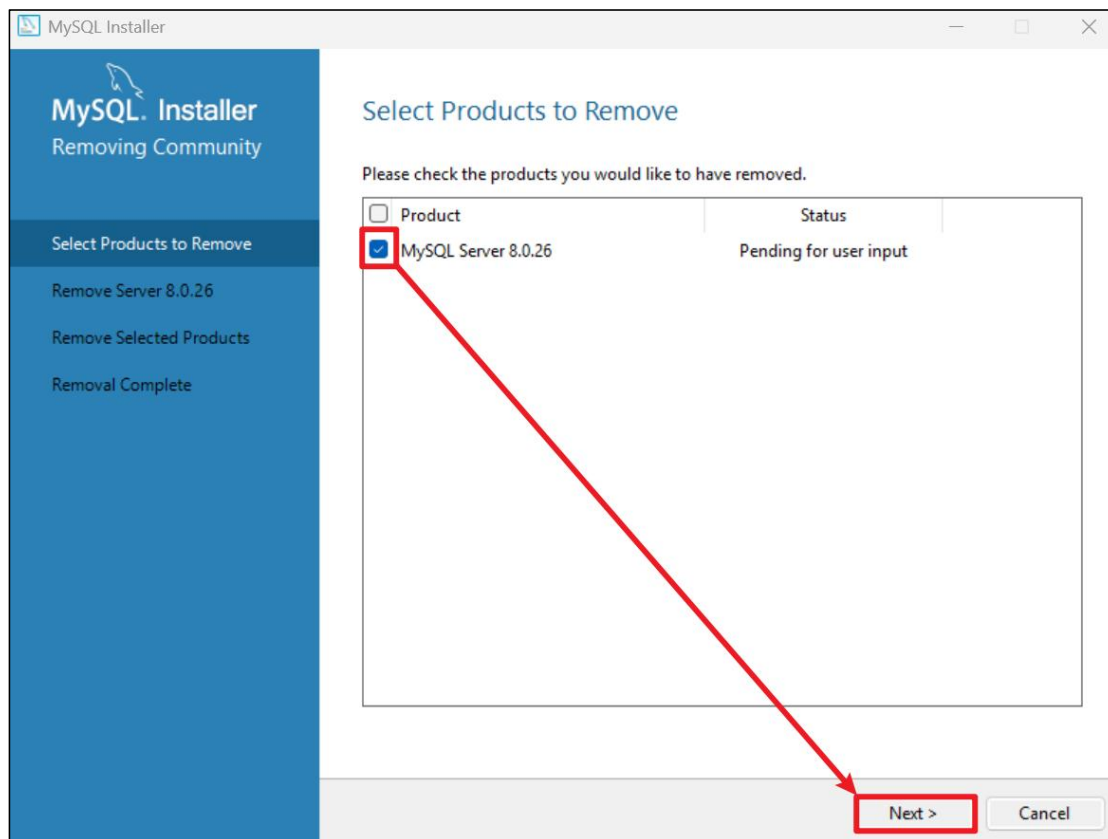
(2) 点击 No



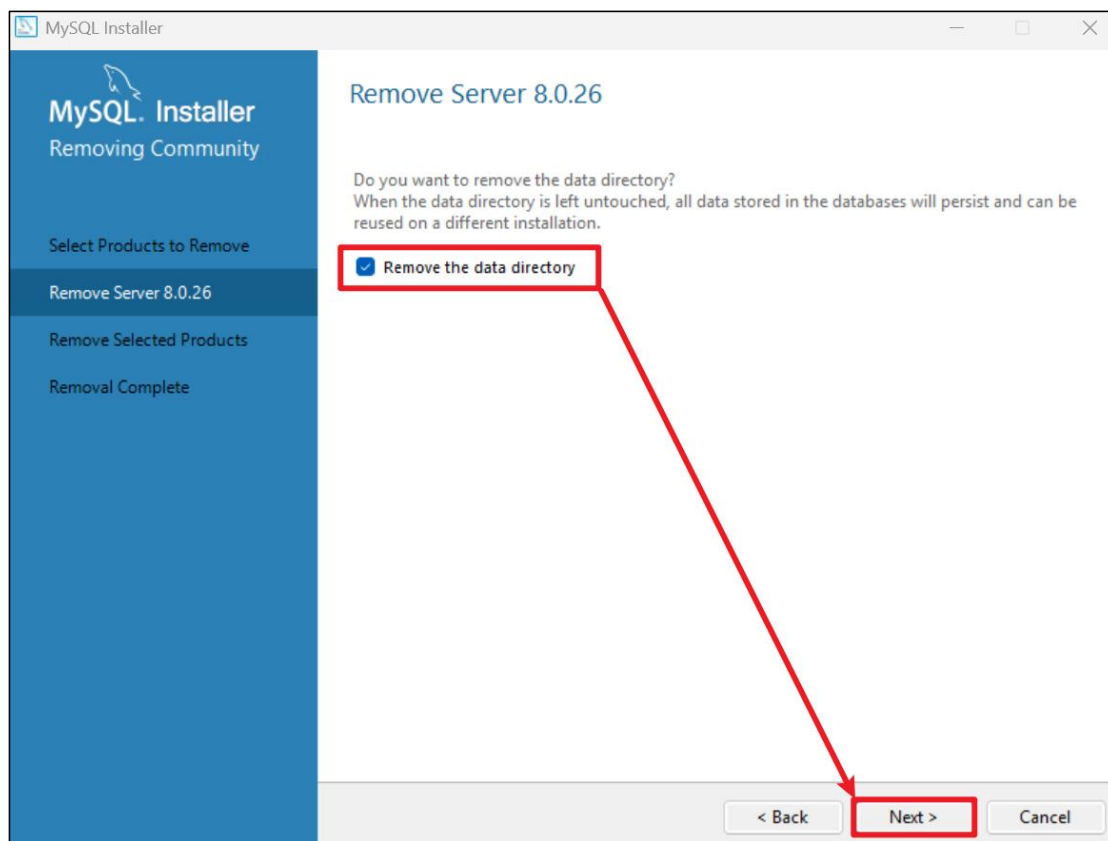
(3) 选择要卸载的 MySQL Server，点击 Remove



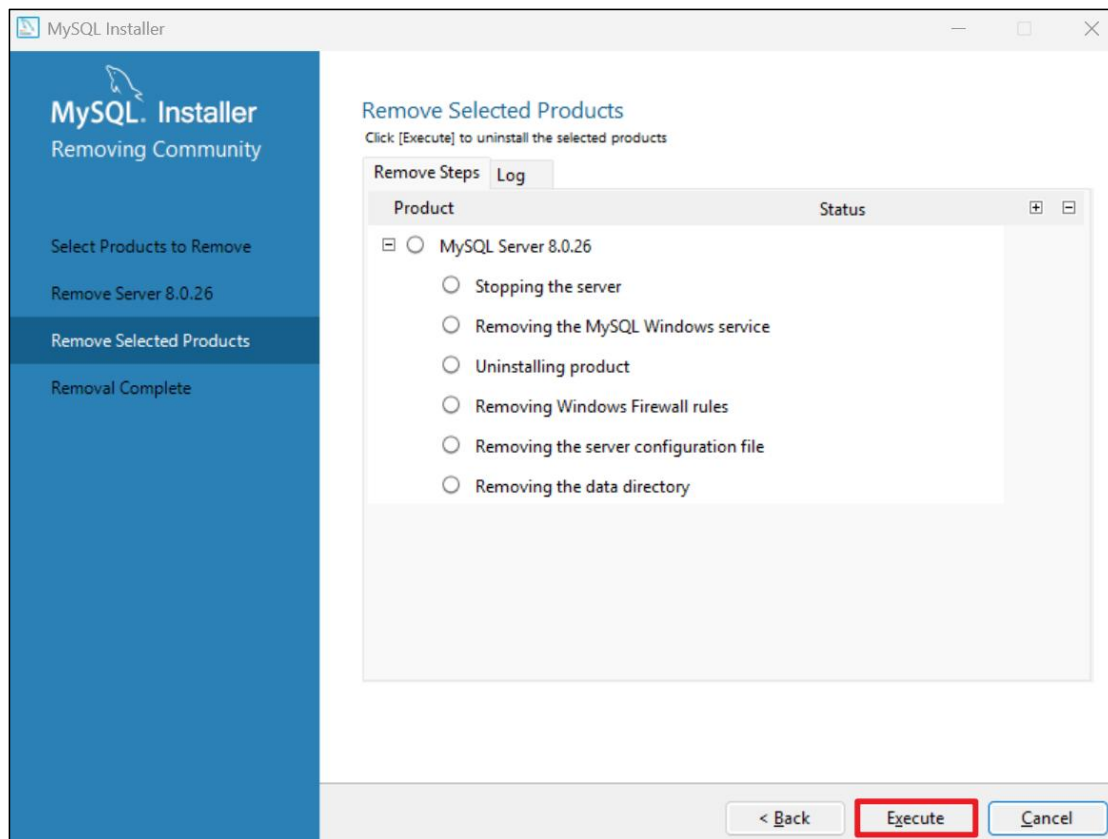
(4) 选择版本，点击 Next



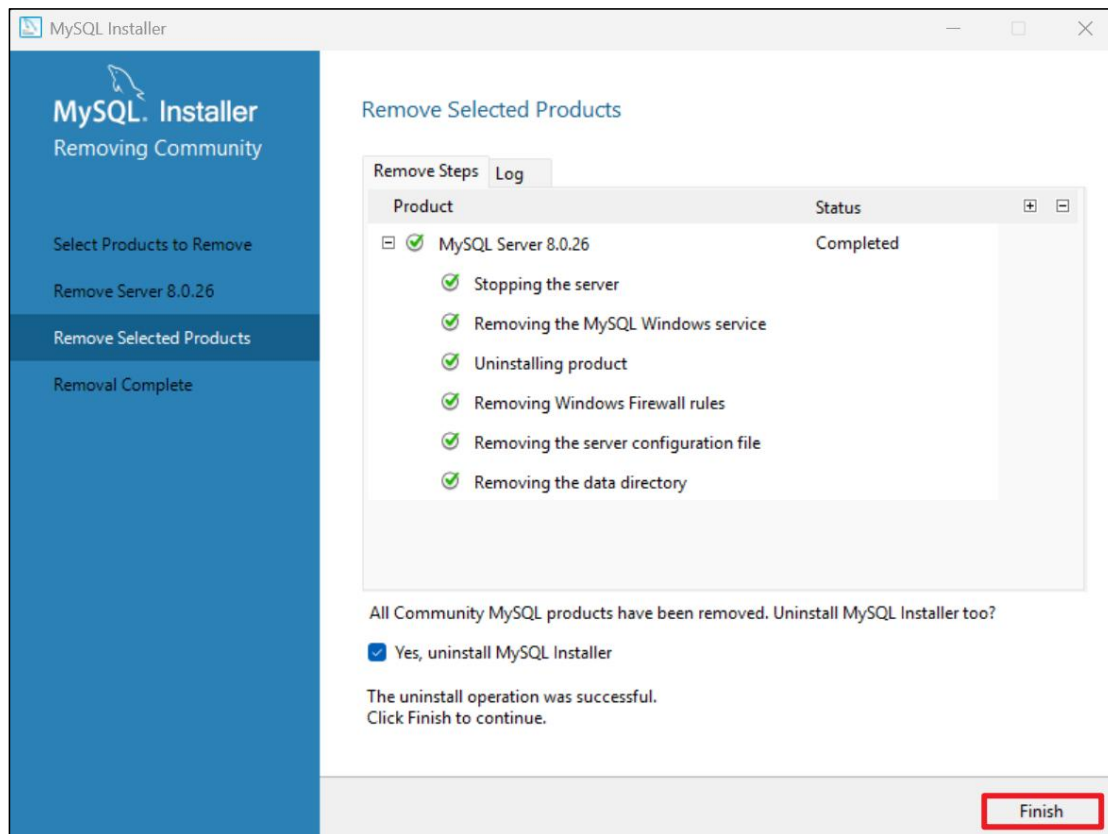
(5) 选择是否删除数据，之后点击 Next



(6) 点击 Execute，执行删除



(7) 删除完成，点击 Finish



2.1.3 清理残余文件

部分同学需要。如果卸载后还有残余文件，先对残余文件进行清理后再安装。

- 服务目录：MySQL 服务的安装目录
- 数据目录：默认在 C:\ProgramData\MySQL。如果自己单独指定过，就找到自己的数据目录

2.1.4 清理服务列表中的服务名

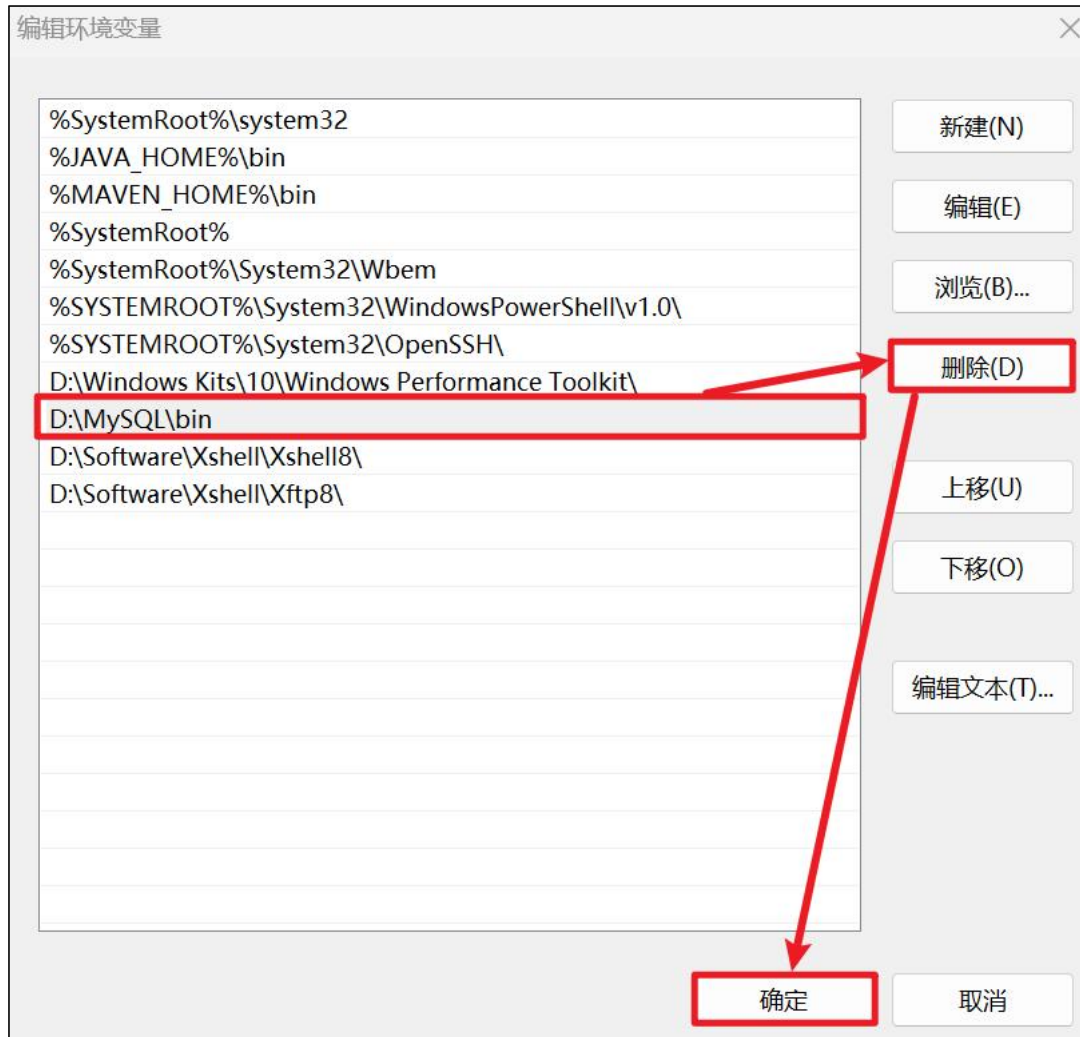
Windows 操作系统卸载后 MySQL 后，如果服务没有卸载干净，可以通过系统管理员在 cmd 命令行删除服务。

```
sc delete 服务名
例如：
sc delete MySQL80
```

2.1.5 清理环境变量

删除用户变量和系统变量的 Path 中关于 MySQL 的变量。





2.1.6 清理注册表

选做，反复安装不成功的，可以尝试。

Win + R，输入 regedit 打开注册表。

- HKEY_LOCAL_MACHINE\SYSTEM\ControlSet001\Services\Eventlog\Application\

MySQL 服务 目录删除

- HKEY_LOCAL_MACHINE\SYSTEM\ControlSet001\Services\MySQL 服务 目录删

除

- HKEY_LOCAL_MACHINE\SYSTEM\ControlSet002\Services\Eventlog\Application\

MySQL 服务 目录删除

- HKEY_LOCAL_MACHINE\SYSTEM\ControlSet002\Services\MySQL 服务 目录删

除

- HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\Eventlog\Application\MySQL 服务 目录删除
- HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\MySQL 服务 删除注册表中的 ControlSet001、ControlSet002 不一定是 001 和 002，可能是 ControlSet005、006 之类。

2.2 MySQL 安装

必须用系统管理员身份运行 **mysql** 安装程序

安装目录切记不要用中文

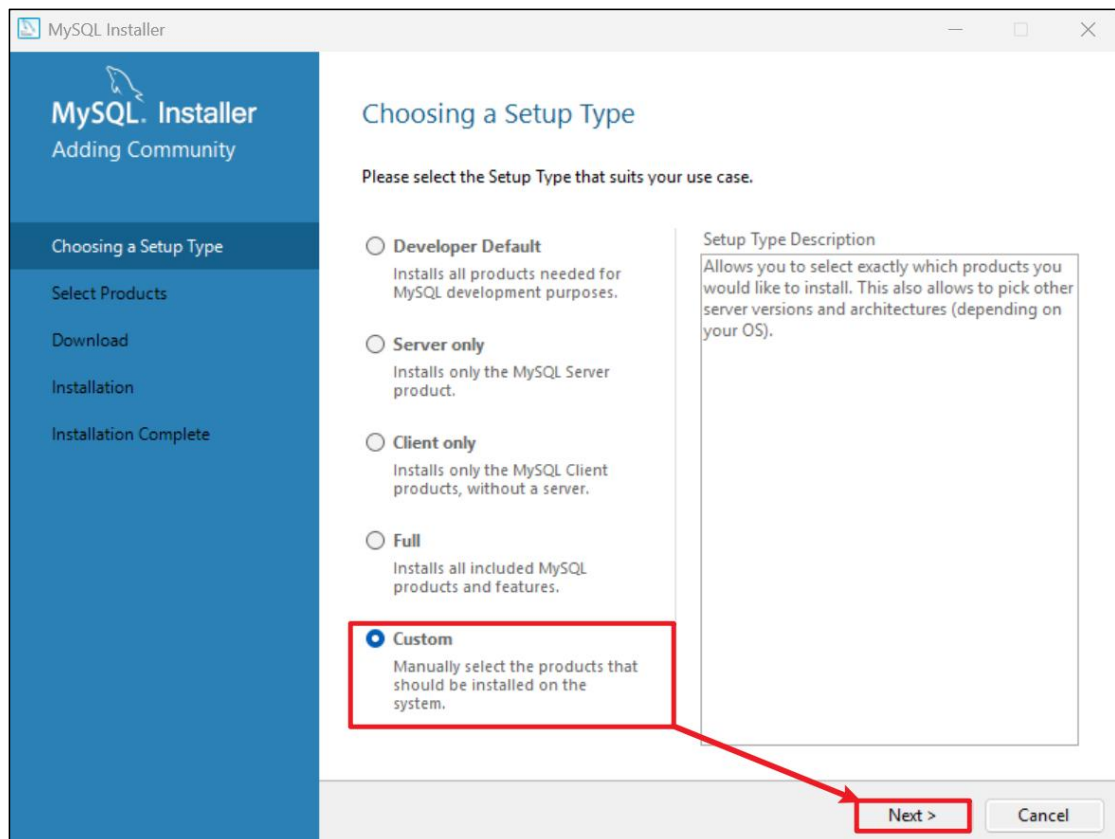
2.2.1 双击安装向导



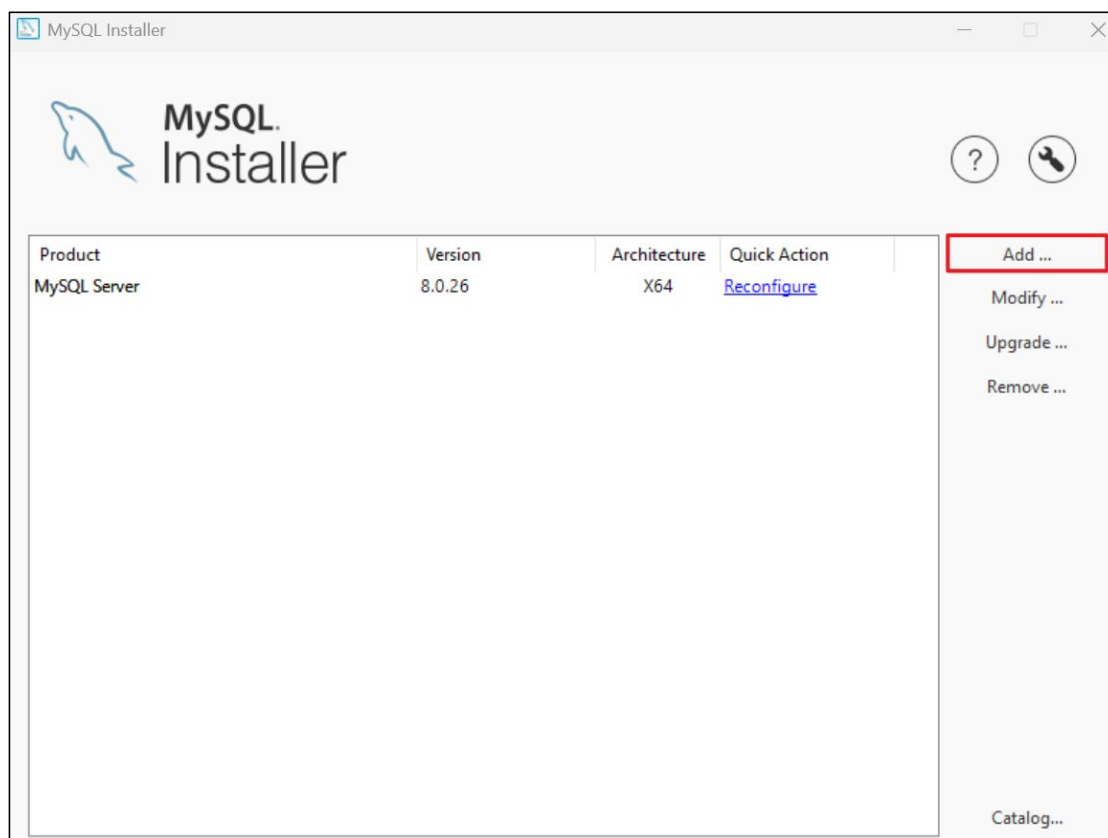
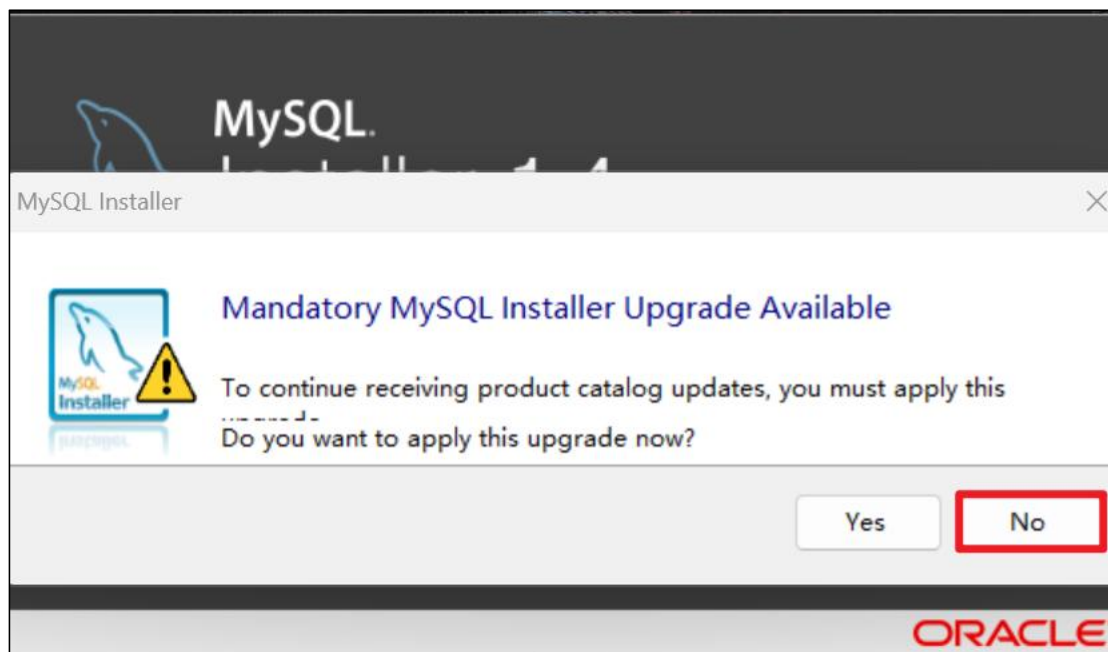
2.2.2 首次安装或再安装

1) 如果是首次安装

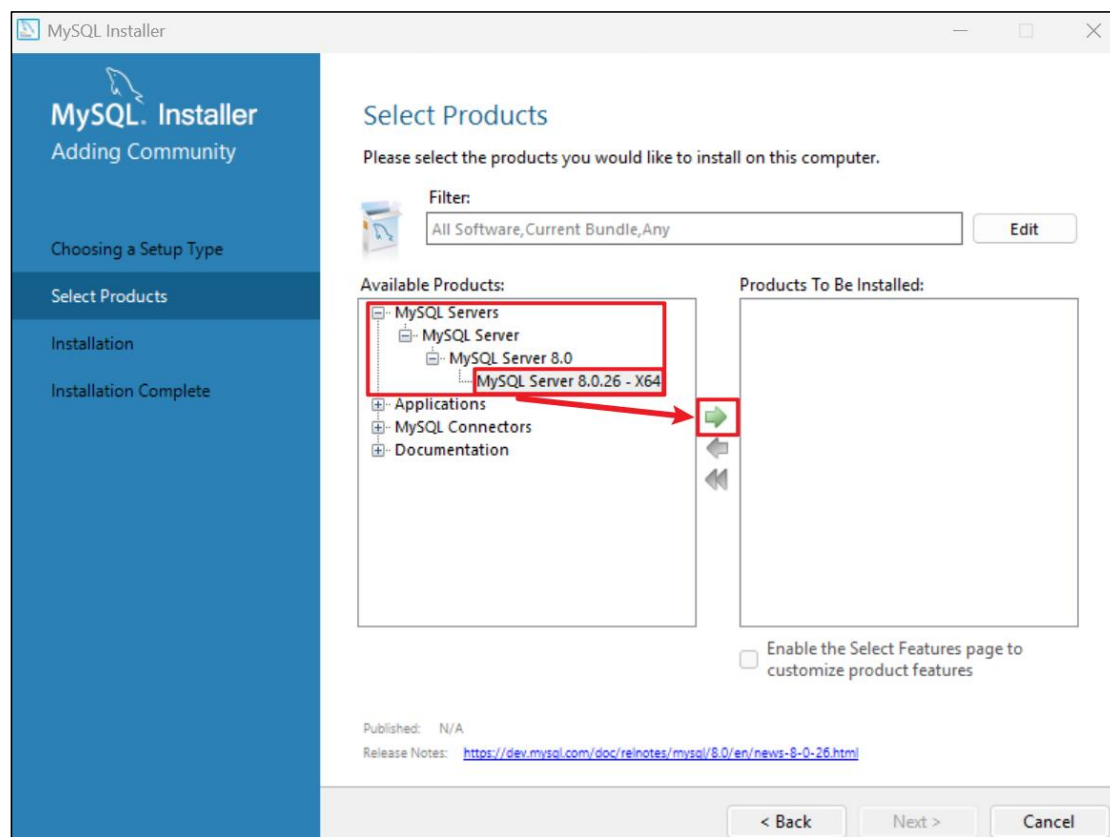
同意安装协议，之后选择安装模式为 Custom。



2) 如果不是首次安装

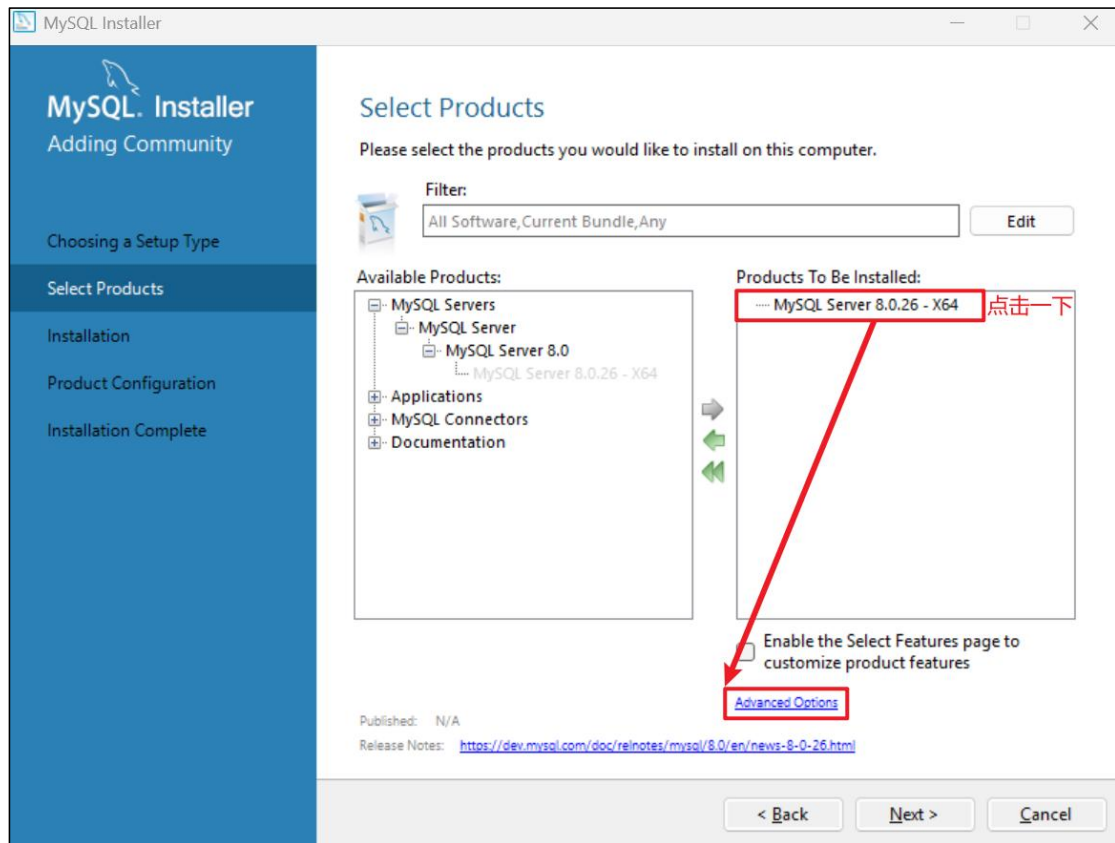


2.2.3 选择要安装的产品

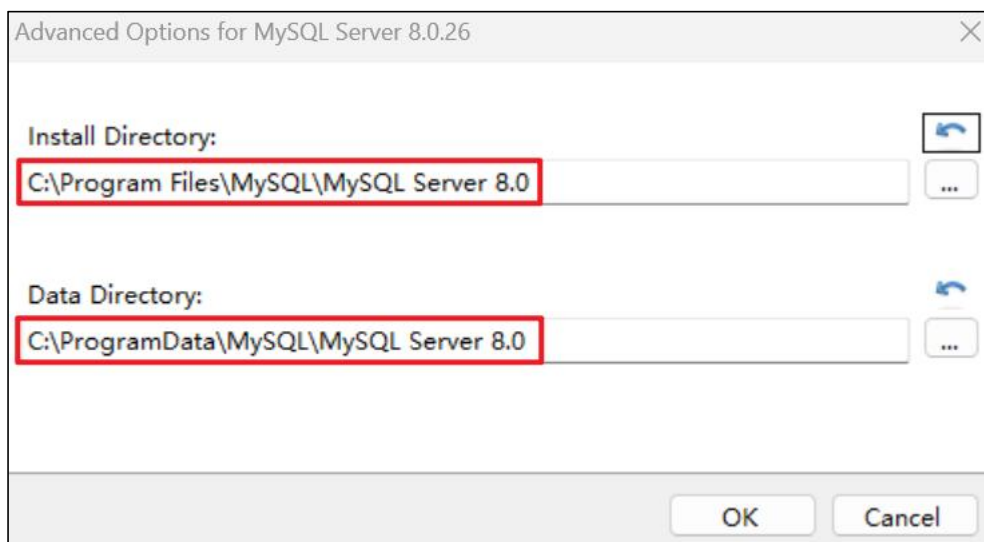


2.2.4 指定安装目录

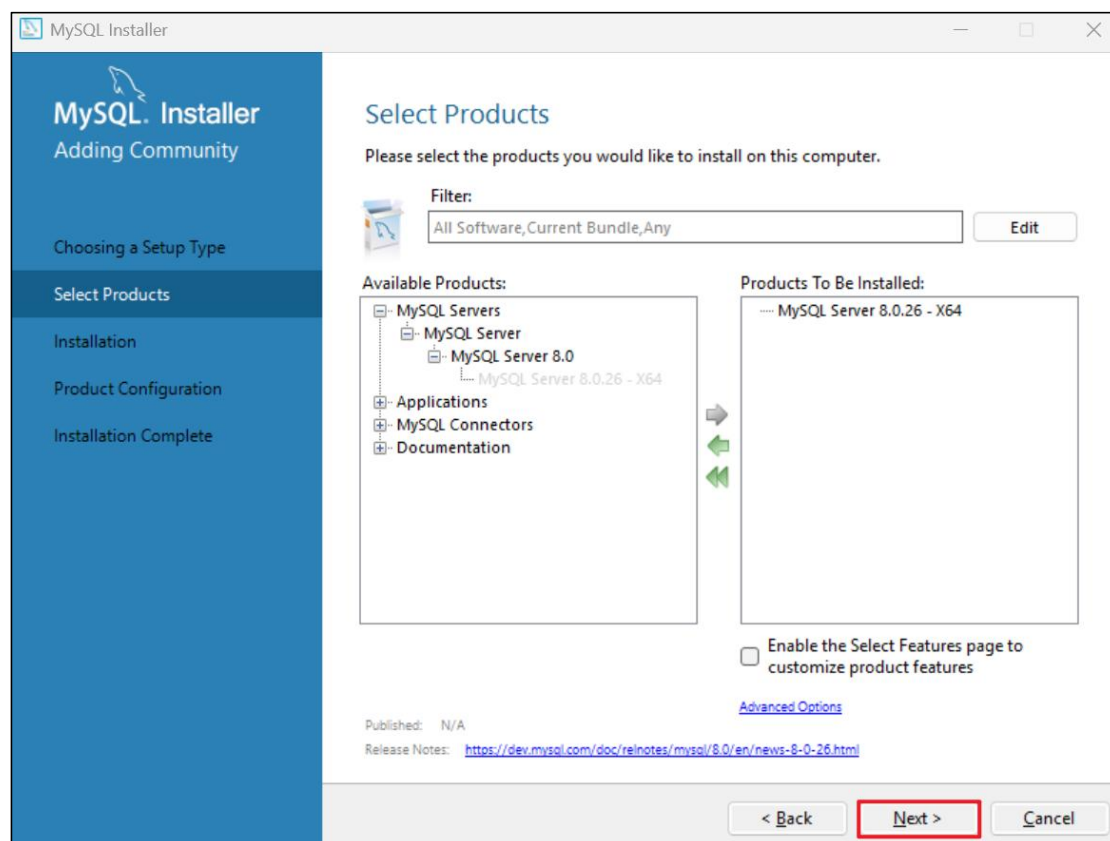
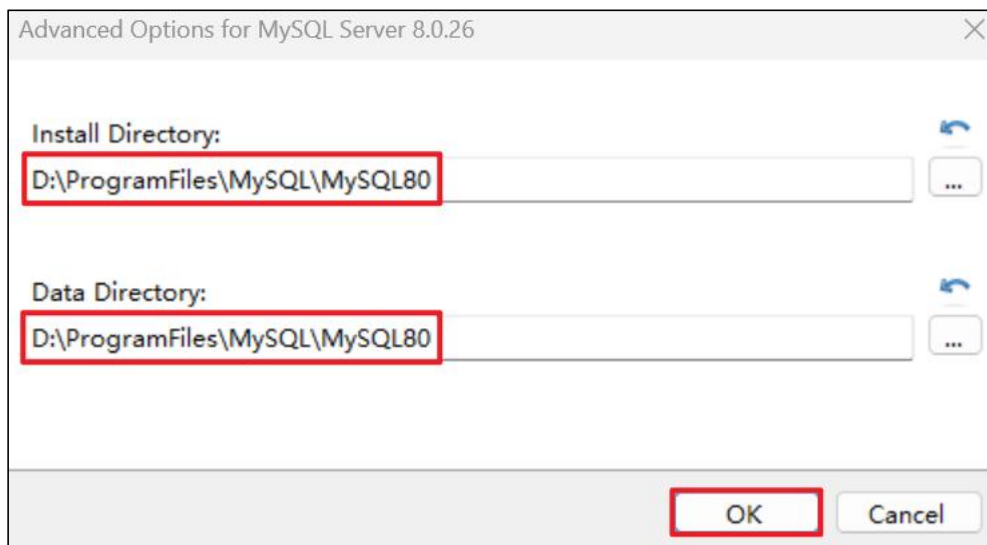
切记服务安装目录不要有中文，否则会有问题



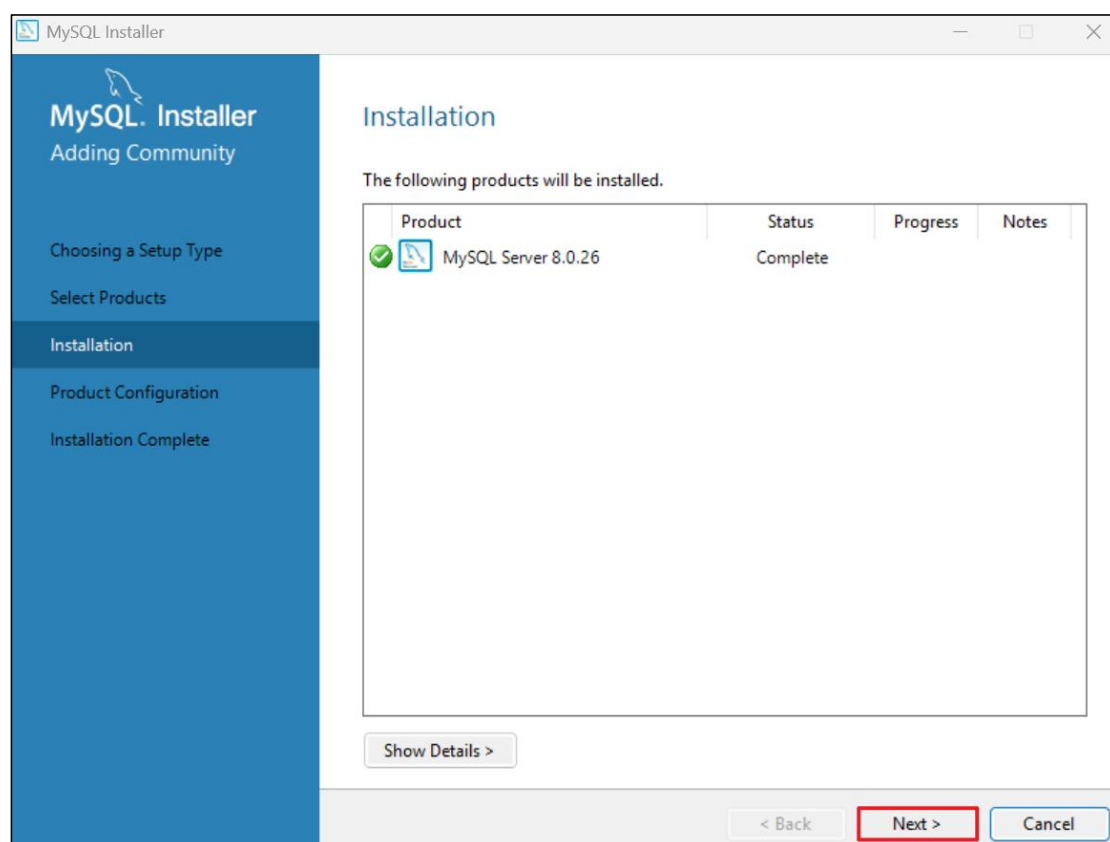
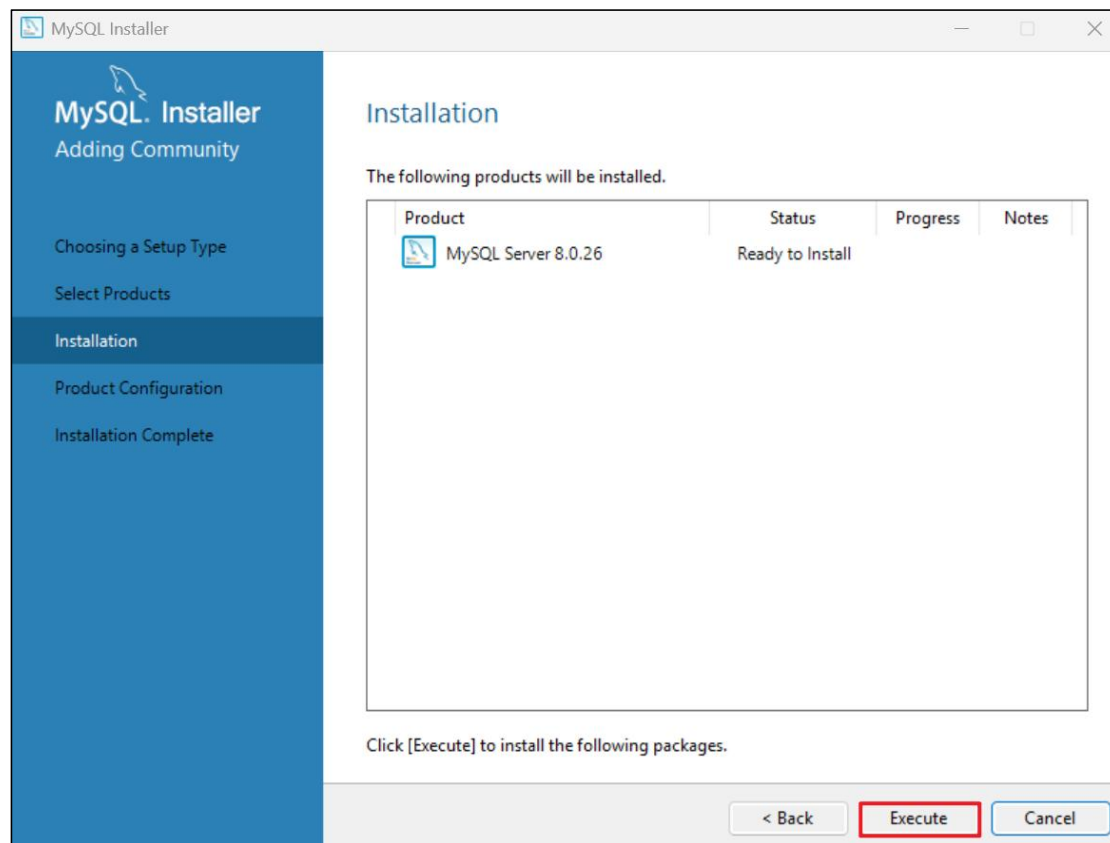
此处两个路径为默认路径，可以修改。

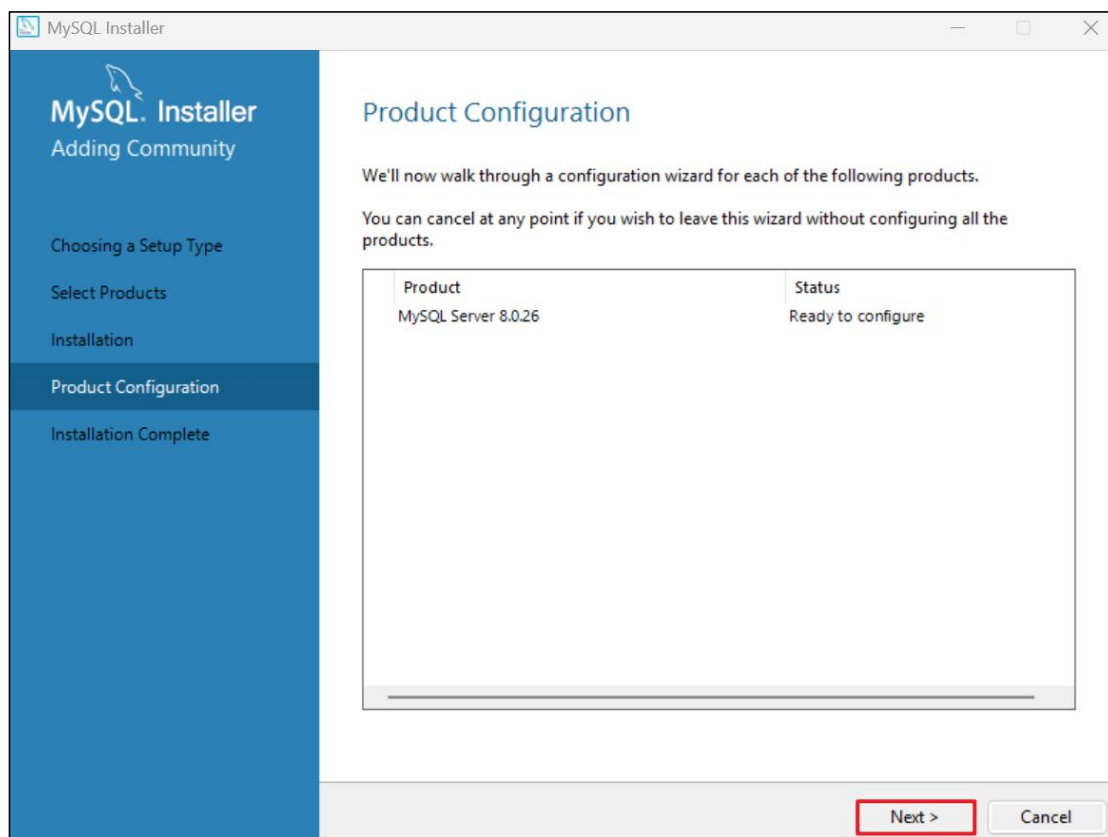


这两个路径不能提前创建，否则会提示文件夹已存在。



2.2.5 执行安装

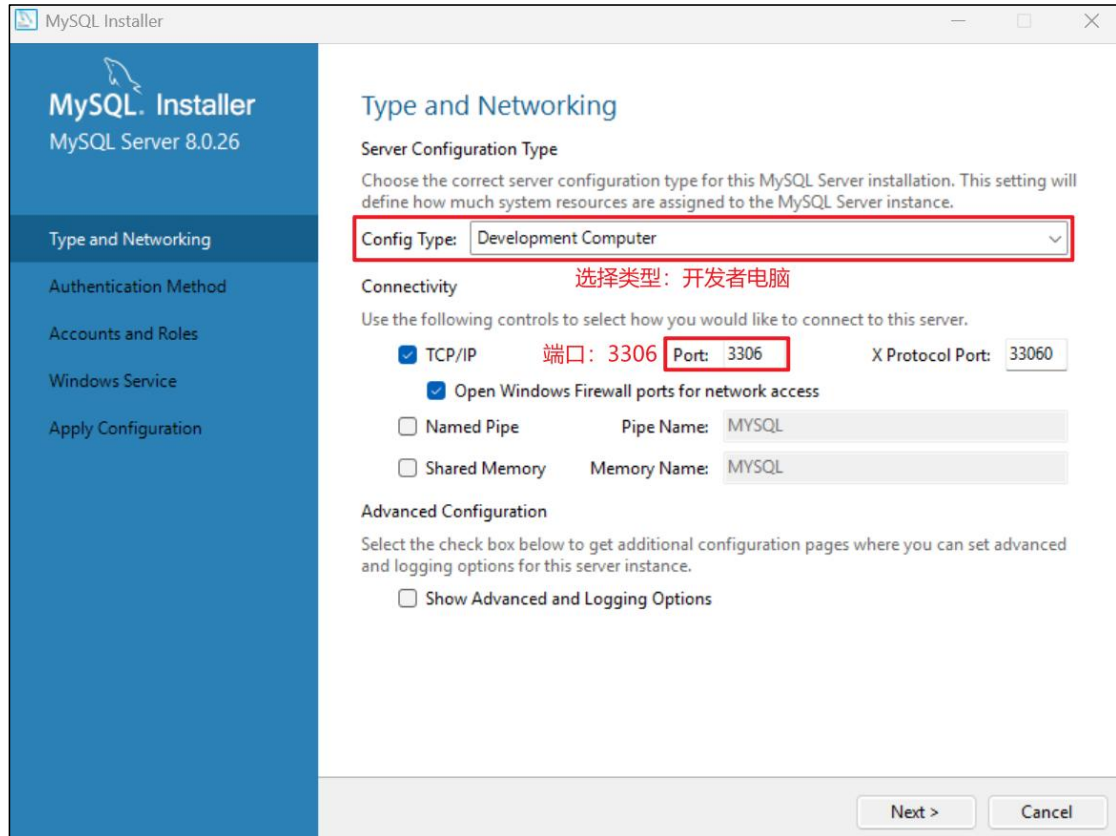




安装完成，准备设置。

2.3 MySQL 实例初始化和设置

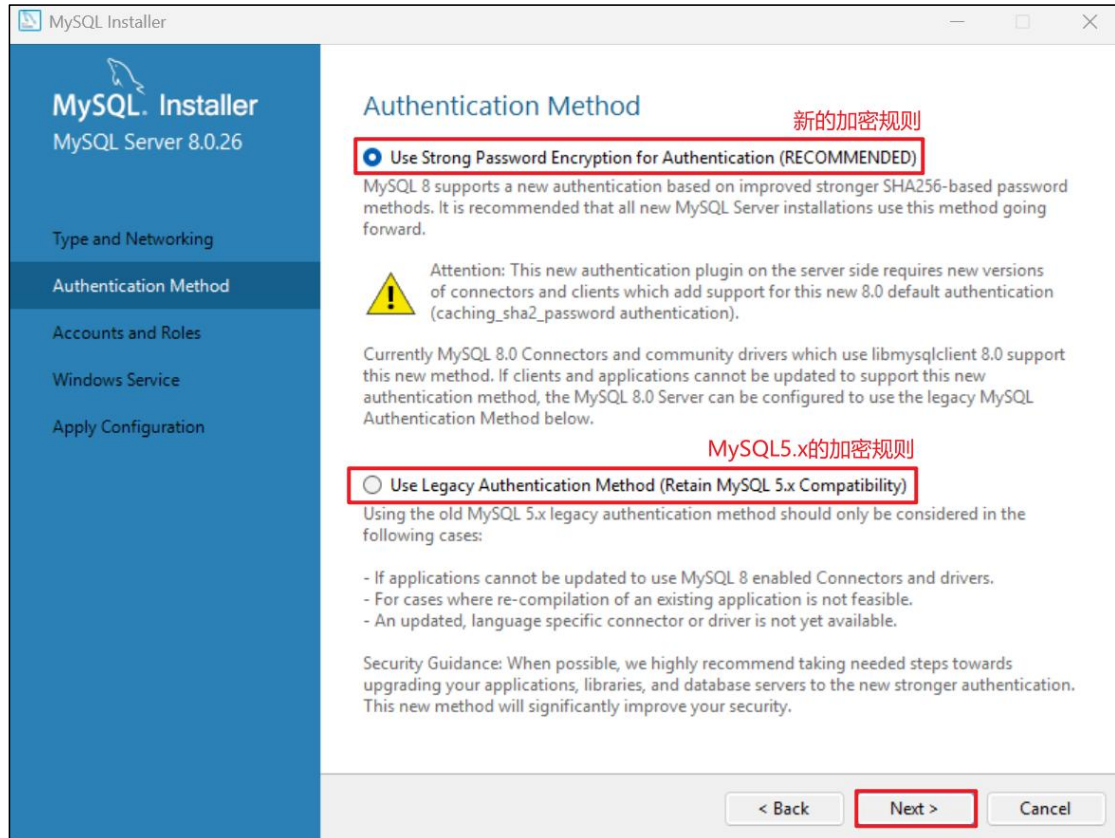
2.3.1 指定电脑类型与端口号



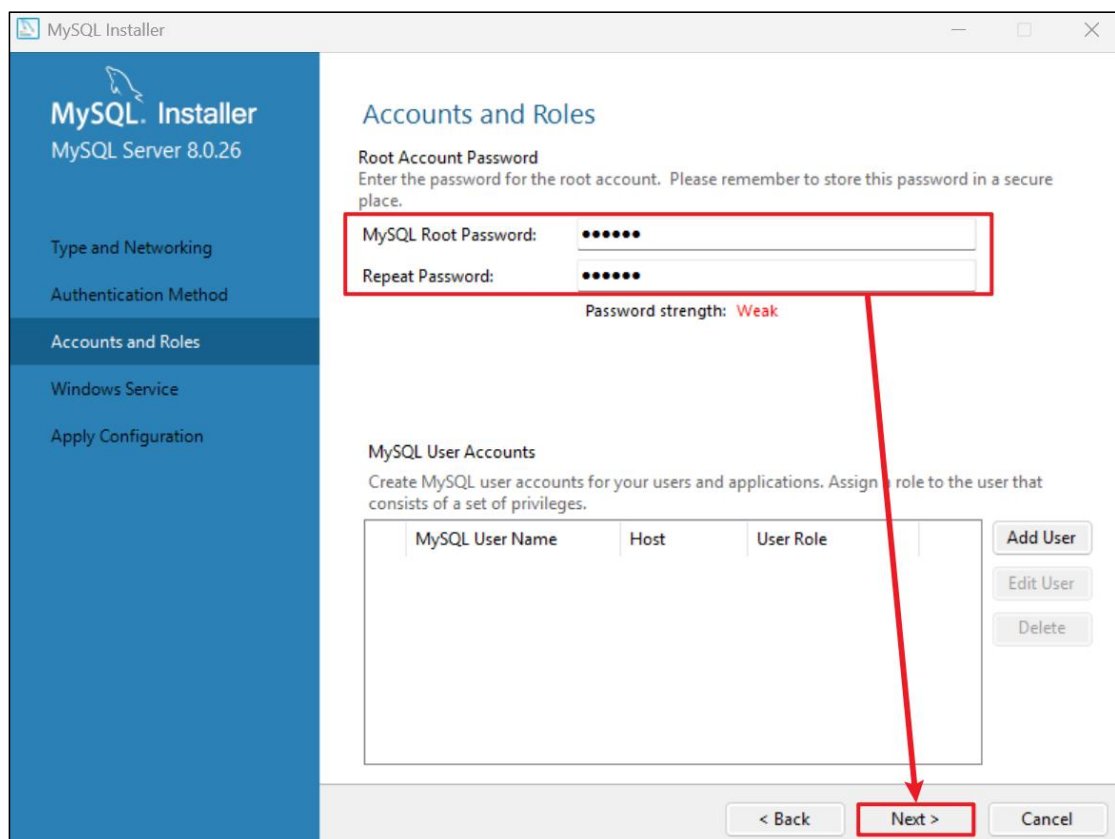
2.3.2 指定账号密码加密规则

在 MySQL5.x 中默认的身份认证插件为 “mysql_native_password”。

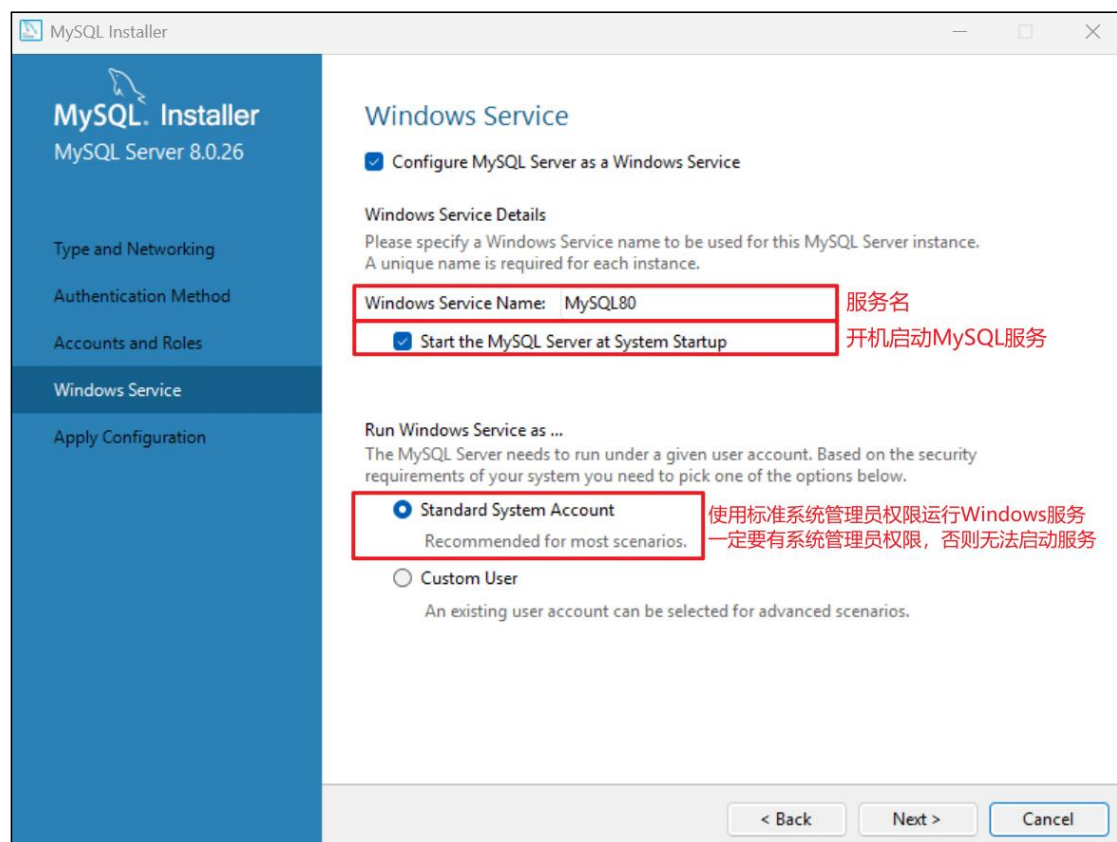
在 MySQL8.x 中，默认的身份认证插件是 “caching_sha2_password”，替代了之前的 “mysql_native_password”。



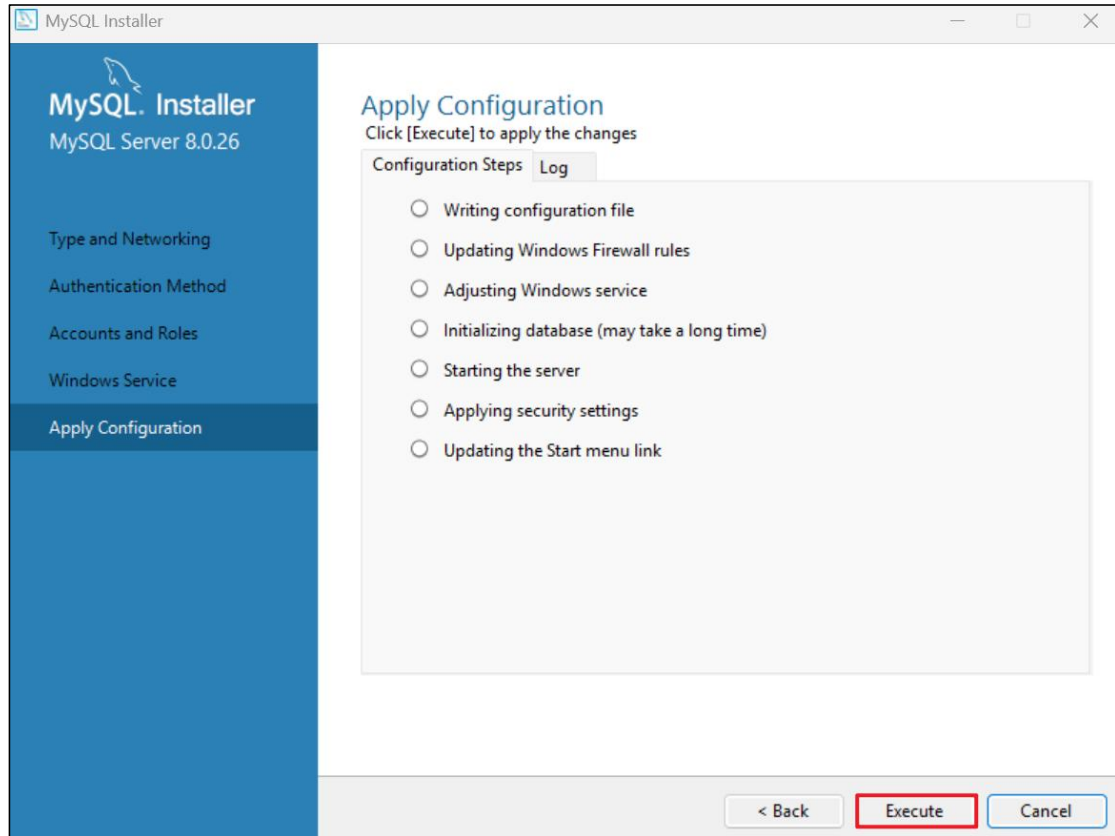
2.3.3 设置 root 账户密码



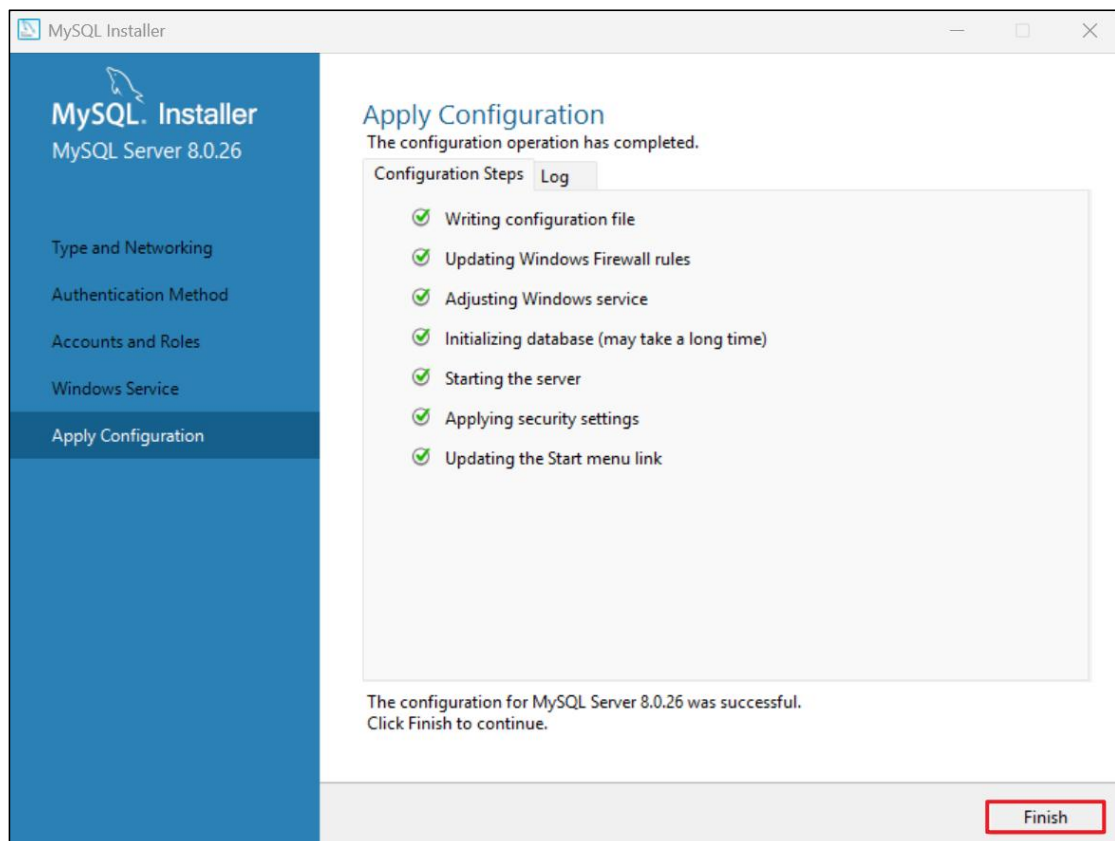
2.3.4 设置 MySQL 服务名与服务启动策略

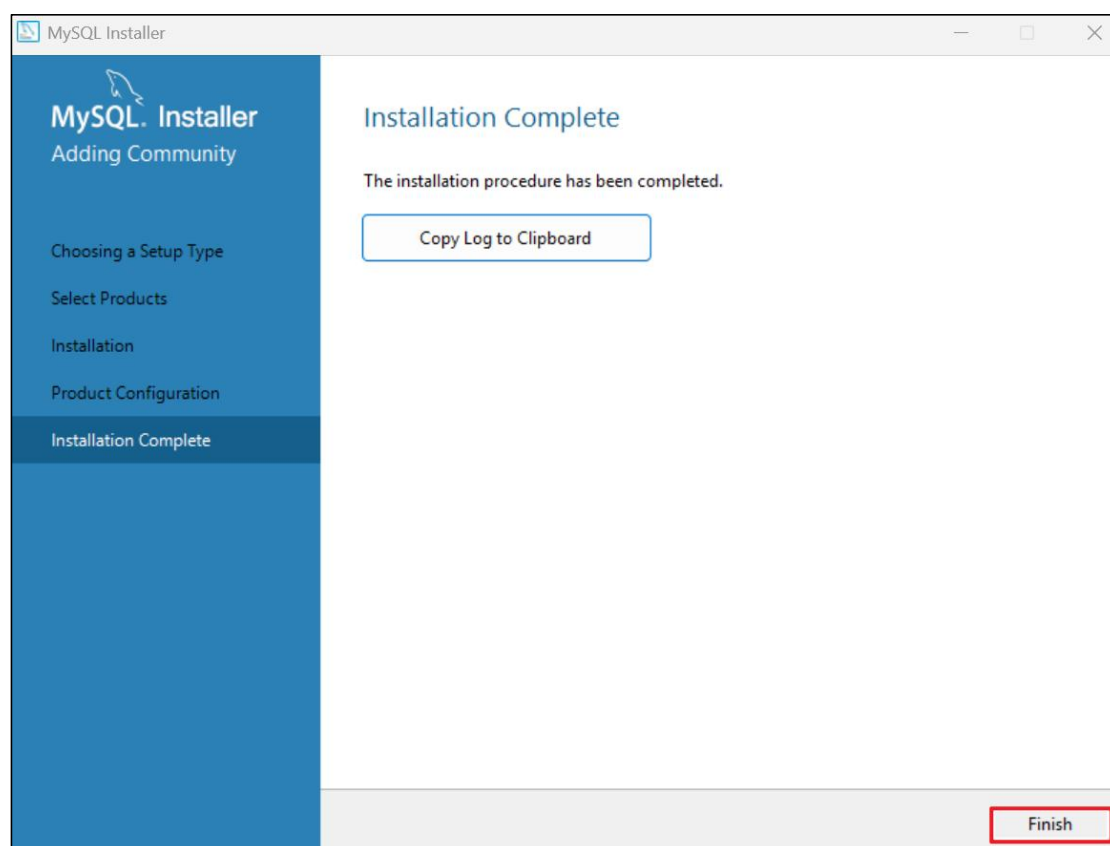
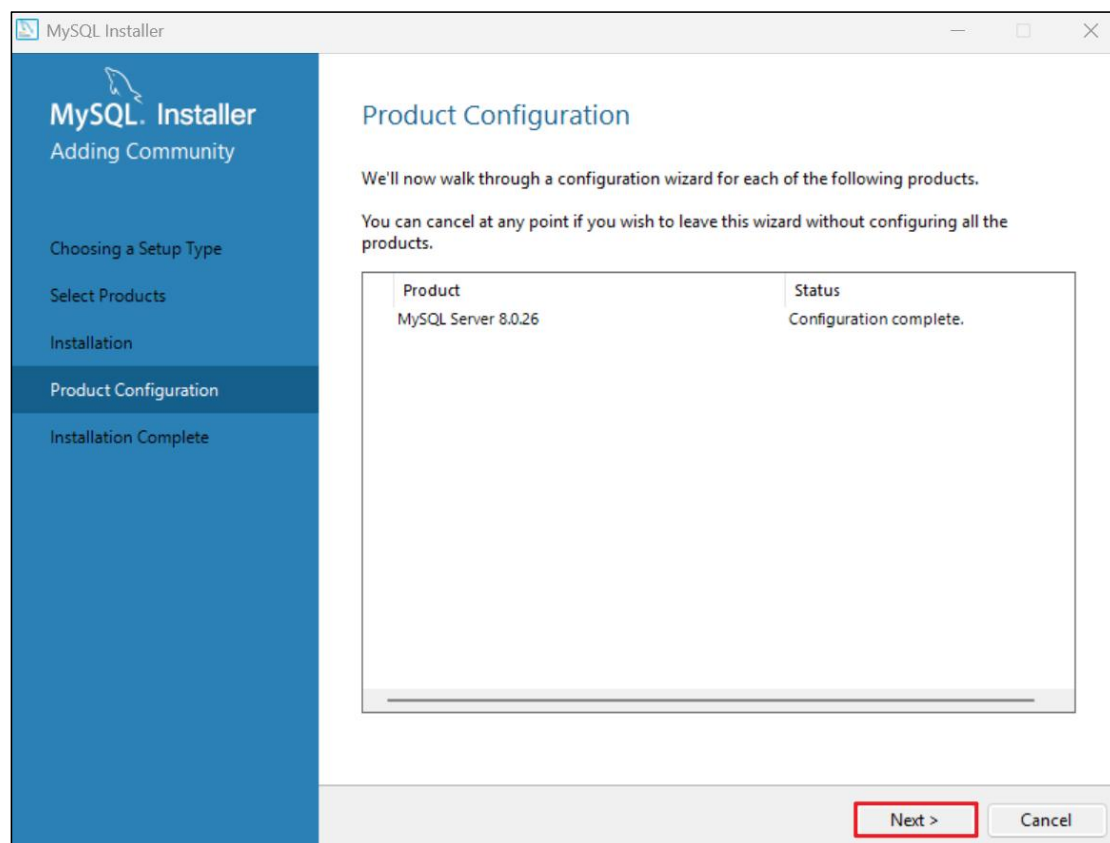


2.3.5 执行设置（初始化 MySQL 实例）



2.3.6 完成设置





2.4 MySQL 数据库服务的启动和停止

MySQL 软件的服务器端必须先启动，客户端才可以连接和使用数据库。如果常用可以设置自动启动。

1) 方式一：服务面板启动

Win + R，输入 services.msc 打开服务面板。找到 MySQL 的服务，右键启动或停止。

2) 方式二：命令行启动

cmd 中输入：net start/stop 服务名

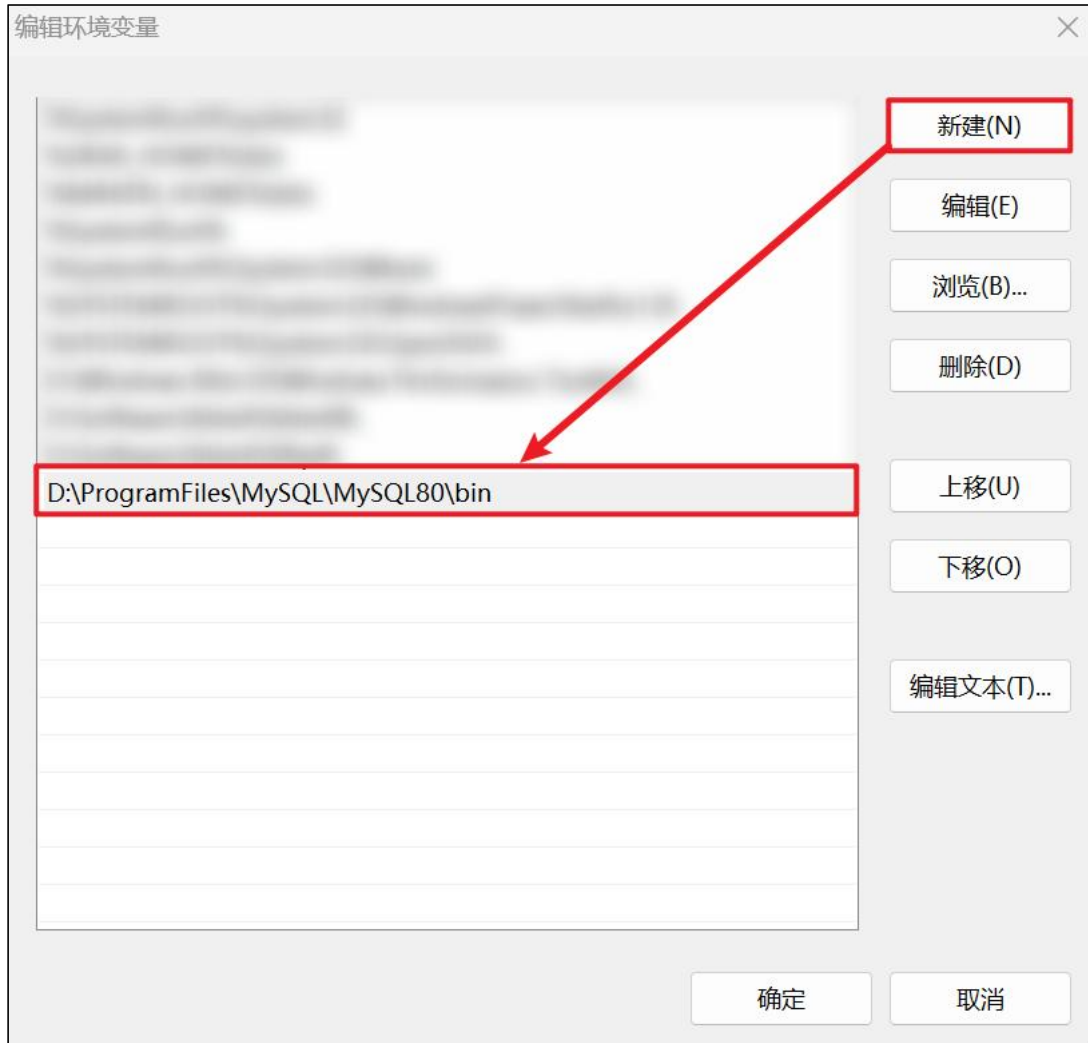
```
net start MySQL80
net stop MySQL80
```

2.5 MySQL 数据库环境变量的配置

```
> mysql -uroot -p
mysql : 无法将“mysql”项识别为 cmdlet、函数、脚本文件或可运行程序的名称。请检查名称的拼写，如果包括路径，请确保路径正确，然后再试一次。
所在位置 行:1 字符: 1
+ mysql -uroot -p
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (mysql:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException
```

如果连接 MySQL 时报类似上述的错误，需要配置环境变量。

打开环境变量，在 Path 中添加“MySQL 安装路径\bin”。



2.6 MySQL 数据库客户端的登录

2.6.1 方式一：MySQL 自带客户端

开始菜单→所有程序→MySQL→MySQL 8.0 Command Line Client



2.6.2 方式二：命令行客户端

```
mysql -h 主机名 -P 端口号 -u 用户名 -p 密码
```

例如：

```
mysql -hlocalhost -P3306 -uroot -p*****
```

建议在下一行输入密码：

```
mysql -hlocalhost -P3306 -uroot -p
Enter password:*****
```

如果是连本机，-hlocalhost 可以省略。如果端口号没有修改，-P3306 可以省略：

```
mysql -uroot -p
Enter password:*****
```

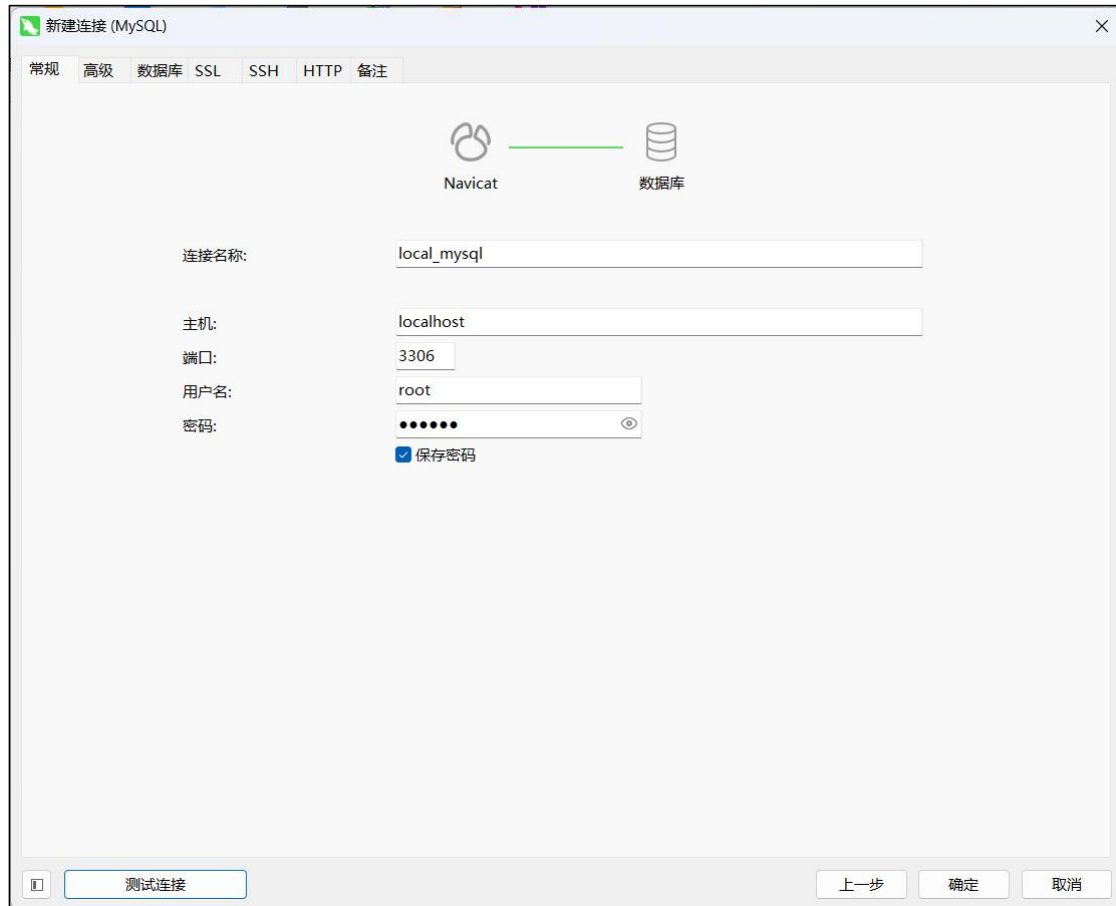
2.6.3 方式三：可视化工具 Navicat

Navicat 是一套可创建多个连接的数据库管理工具，用以方便管理 MySQL、Redis、Oracle、PostgreSQL、SQLite、SQL Server、MariaDB 和 MongoDB 等不同类型的数据库，它与 GaussDB、OceanBase、PolarDB 以及 阿里云、腾讯云、华为云、Amazon RDS、Amazon Aurora、Amazon Redshift、Microsoft Azure、Oracle Cloud 和 MongoDB Atlas 等云数据库兼容。它用于创建、管理和维护数据库。Navicat 的功能足以满足专业开发人员的所有需求，但是对数据库服务器初学者来说又简单易操作。Navicat 的用户界面 (GUI) 设计良好，以安全且简单的方法创建、组织、访问和共享信息。

使用 Navicat 图形化界面工具连接 MySQL 数据库的操作步骤如下：

(1) 步骤 1

文件→点击“新建连接”选项→MySQL。需要输入 MySQL 服务器 IP 地址、端口号、用户名、密码以及指定连接名。设置完成后，可以单击右侧的“测试连接”按钮，测试是否成功，如果没有问题，单击“连接”按钮连接数据库。



(2) 步骤 2

连接成功后，就可以对数据库进行管理和操作了。



(3) 步骤 3 新建查询，编写 SQL 语句



2.6.4 方式四：可视化工具 SQLyog

SQLyog 是一款简介高效且功能强大的图形化数据库管理工具。这款工具是使用 C++ 语言开发的。用户可以使用这款软件来有效地管理 MySQL 数据库。该工具可以方便地创建数据库、表、视图和索引等，还可以方便地进行插入、更新和删除等操作，同时可以方便地进行数据库、数据表的备份和还原。该工具不仅可以通过 SQL 文件进行大量文件的导入和导出，还可以导入和导出 XML、HTML 和 CSV 等多种格式的数据。使用 SQLyog 中文社区版进行演示，下载地址为 <https://github.com/webeyog/sqlyog-community/wiki/Downloads>。

使用 SQLyog 图形化界面工具连接 MySQL 数据库的操作步骤如下：

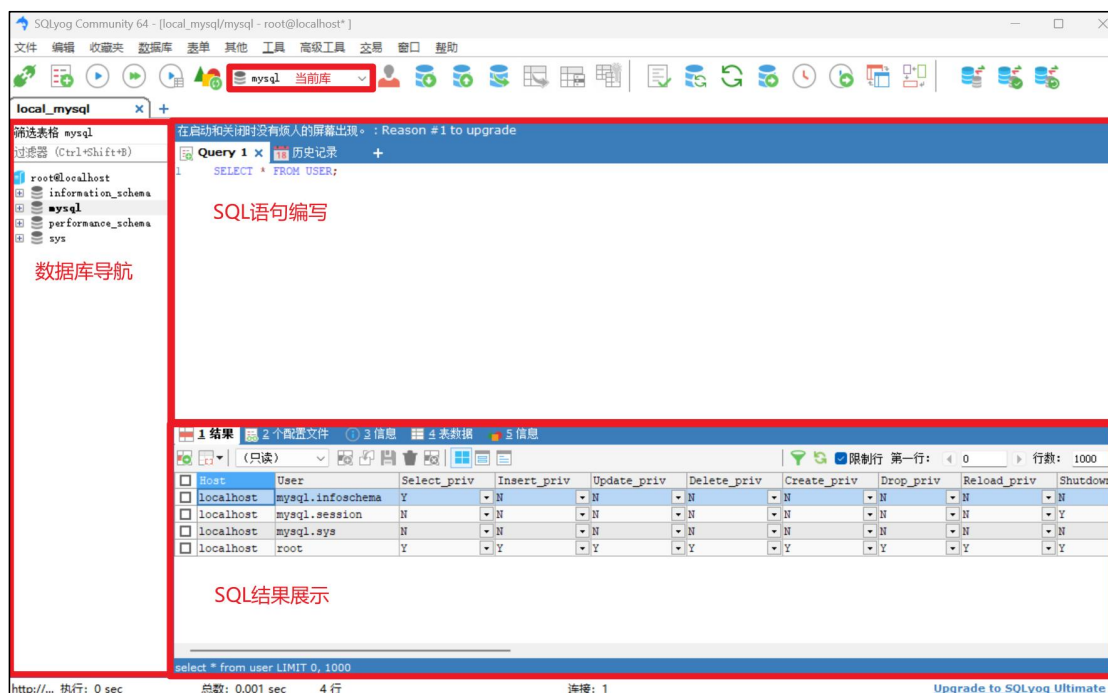
(1) 步骤 1

数据库菜单→点击“创建新连接”选项→打开连接管理窗口。在连接管理窗口可以选择“新建”按钮创建新的连接，也可以直接连接已保存的连接，然后进行参数设置，需要输入 MySQL 服务器 IP 地址、端口号、用户名、密码以及要连接的数据库名称等，其中数据库名称如果不写表示显示该用户有权限查看和操作的全部数据库。设置完成后，可以单击右侧的“测试连接”按钮，测试是否成功，如果没有问题，单击“连接”按钮连接数据库。



(2) 步骤 2

连接成功后，就可以对数据库进行管理和操作了。



2.6.5 方式五：可视化工具 MySQL WorkBench

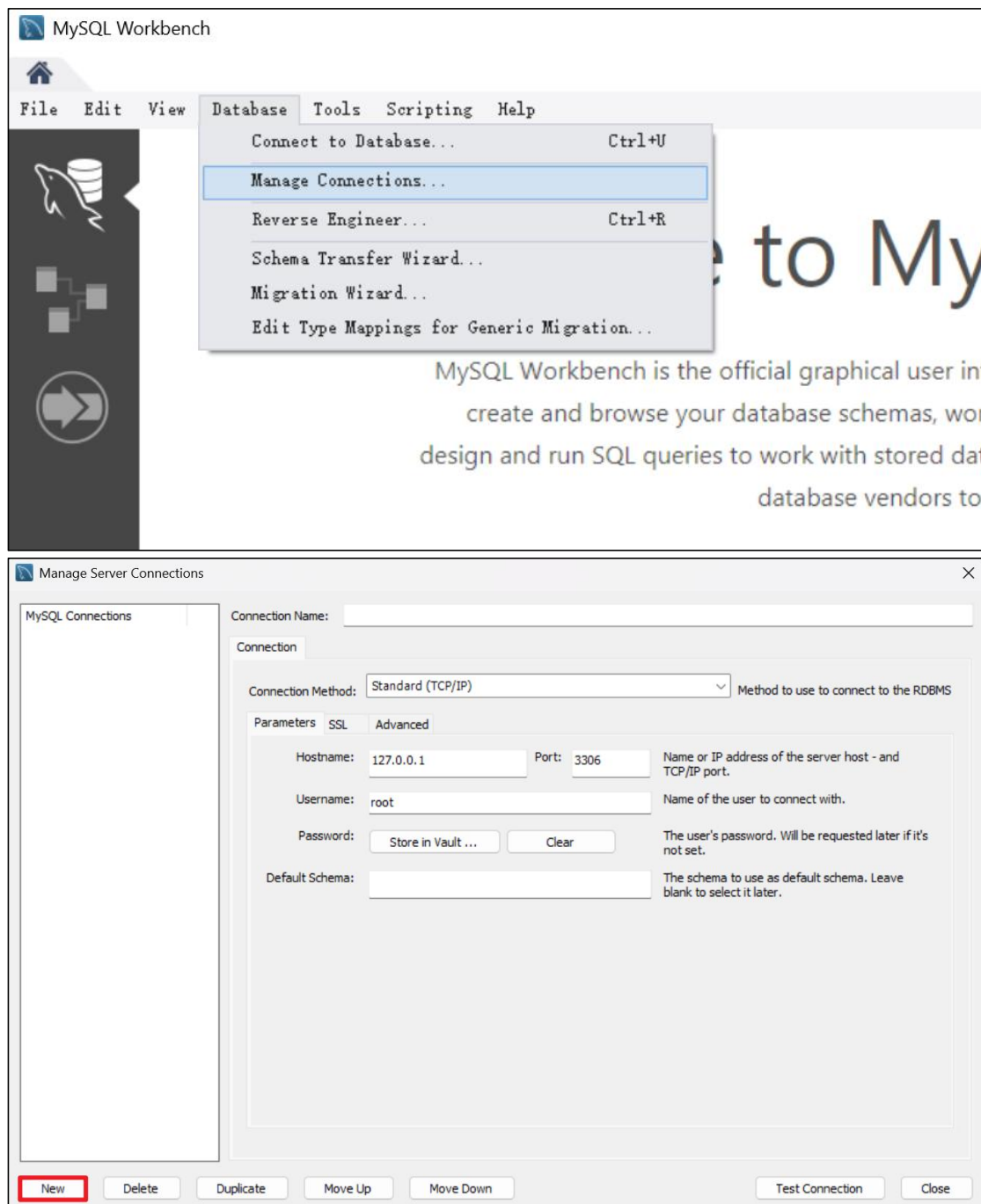
MySQL Workbench 是 MySQL 官方提供的图形化界面管理工具，完全支持 MySQL5.0 以上的版本。它是著名的数据库设计工具 DBDesigner4 的继任者。MySQL Workbench 为数据库管理员、程序开发者和系统规划师提供可视化设计、模型建立、以及数据库管理功能。它包含了用于创建复杂的数据建模 ER 模型，正向和逆向数据库工程，也可以用于执行通常需要花费大量时间的、难以变更和管理的文档任务。MySQL 工作台可在 Windows、Linux 和 Mac 上使用。随 MySQL8 一起发布的 MySQL Workbench 8，可以直接连接 MySQL8，不需要修改加密方式。当你创建、修改数据库及其表等数据库对象时，或针对表中的数据添加、修改、删除操作时，可以提供生成 SQL 功能，对已经存在的表、函数等也可以提供生成 SQL 功能，这对于开发人员，或者初学者 SQL 的读者来说是个福音。下载地址：<https://dev.mysql.com/downloads/workbench/>。

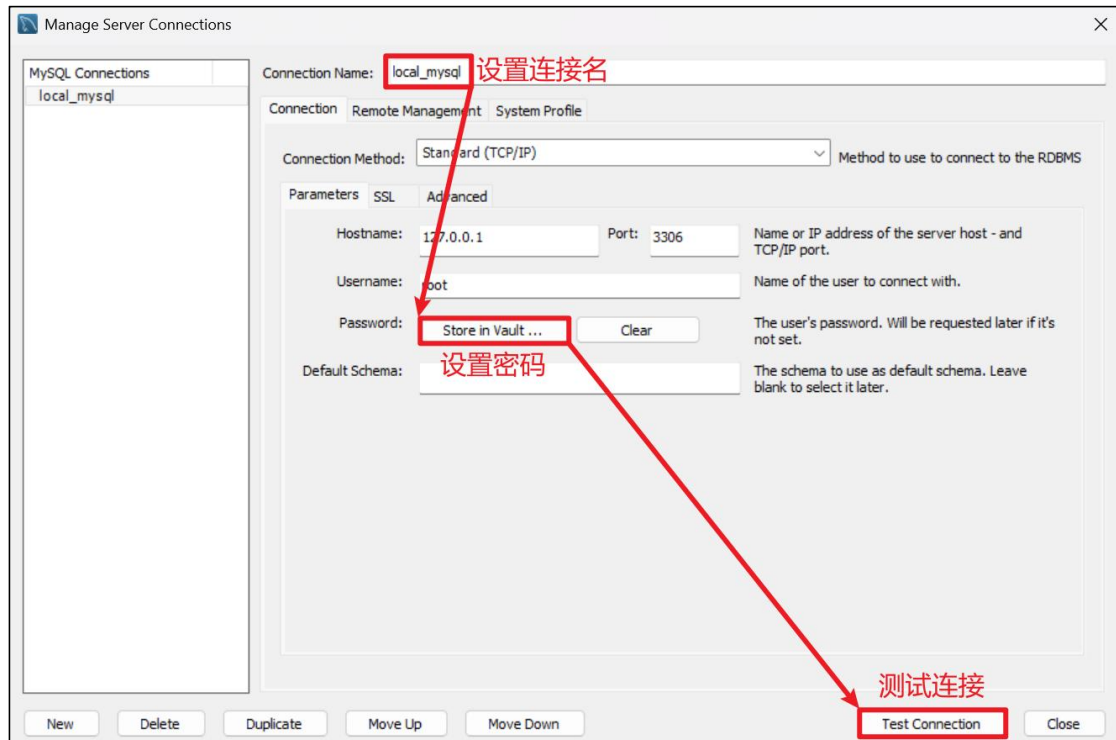
使用 MySQL Workbench 图形化界面工具连接 MySQL 数据库的操作步骤如下：

(1) 步骤 1

Database 菜单→单击“Manage Server Connections”选项→打开连接管理窗口，如图 2-37 所示。在连接管理窗口中可以选择“New”按钮创建新的连接，也可以在左边“已有连接列表”中选择某个连接进行参数设置。需要指定要连接的 MySQL 服务器的 IP 地址，端口号，用户名和密码等。参数设置完成之后，可以单击“Test Connection”按钮测试某个连接是否

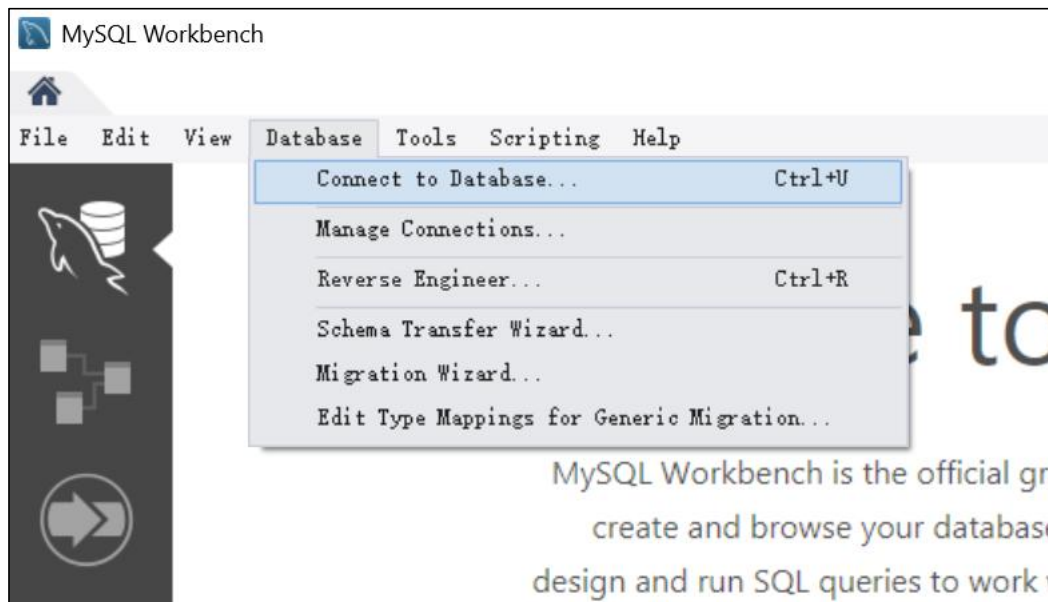
可以连接成功。如果测试成功，可以看到“Successfully made the MySQL connection”的提示对话框。

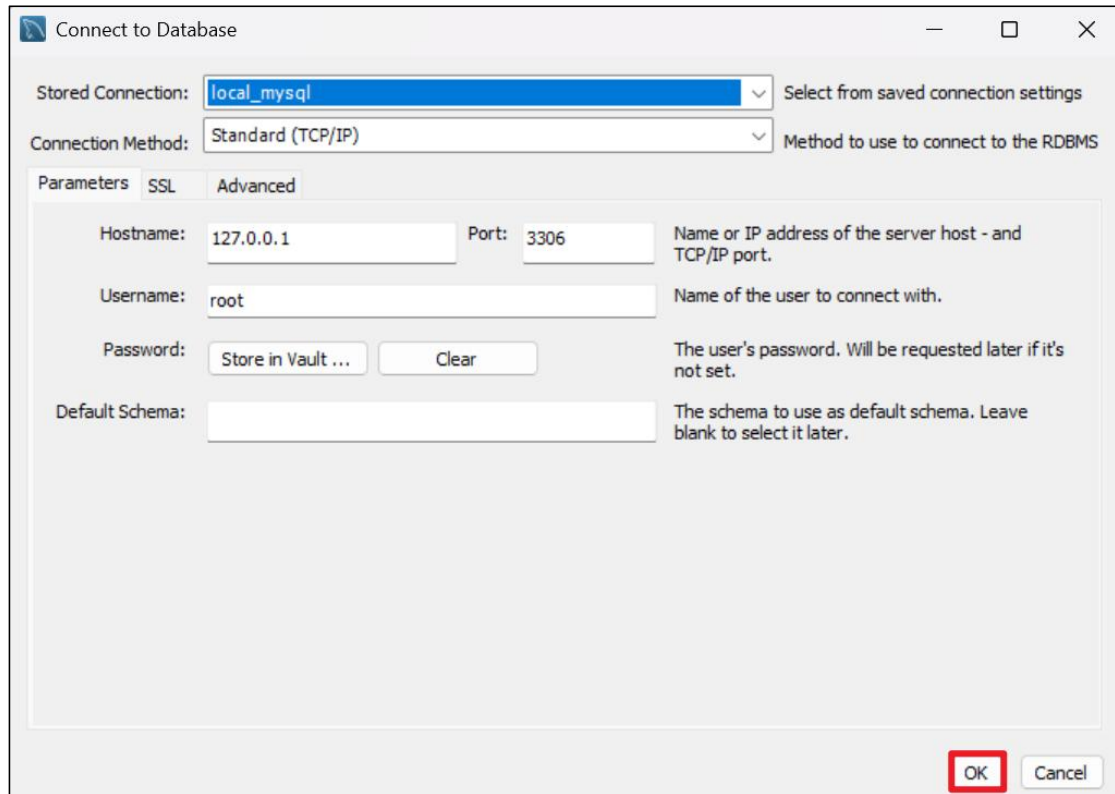




(2) 步骤 2

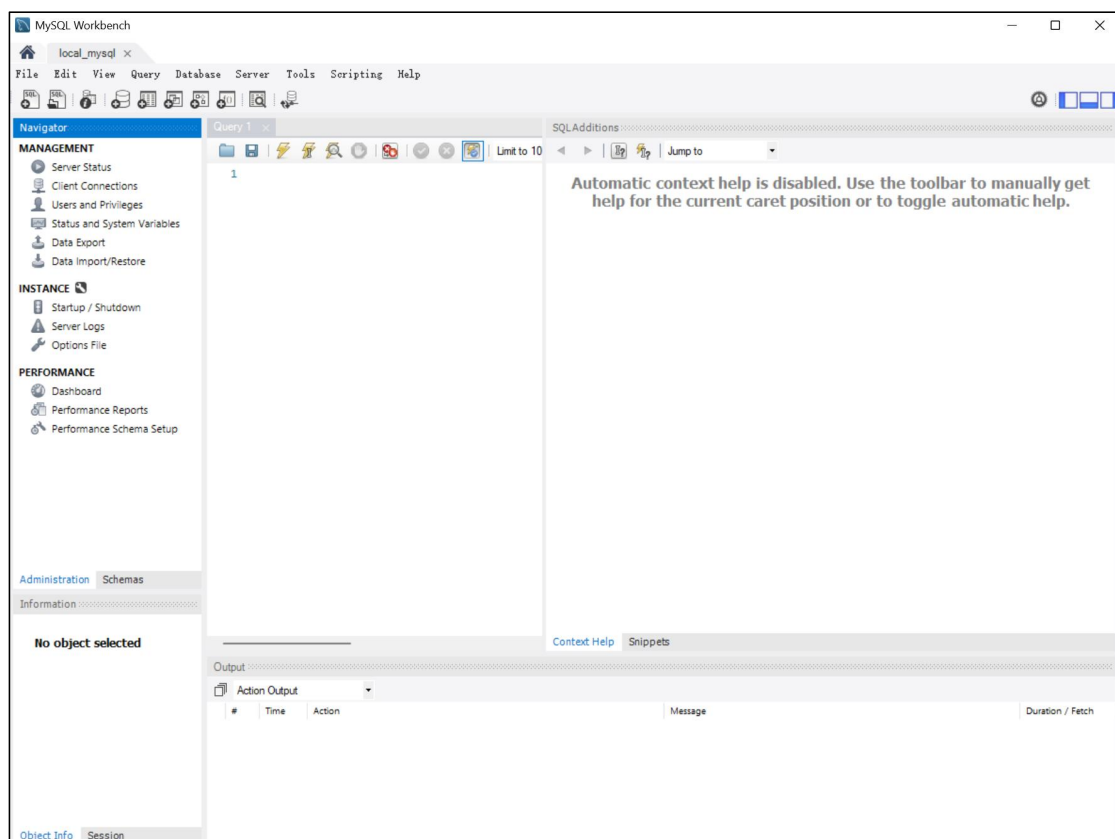
Database 菜单→单击“Connect to Database”选项→打开数据库连接窗口。选择之前创建并设置的某个连接后，单击“OK”按钮进行连接登录 MySQL 数据库。





(3) 步骤 3

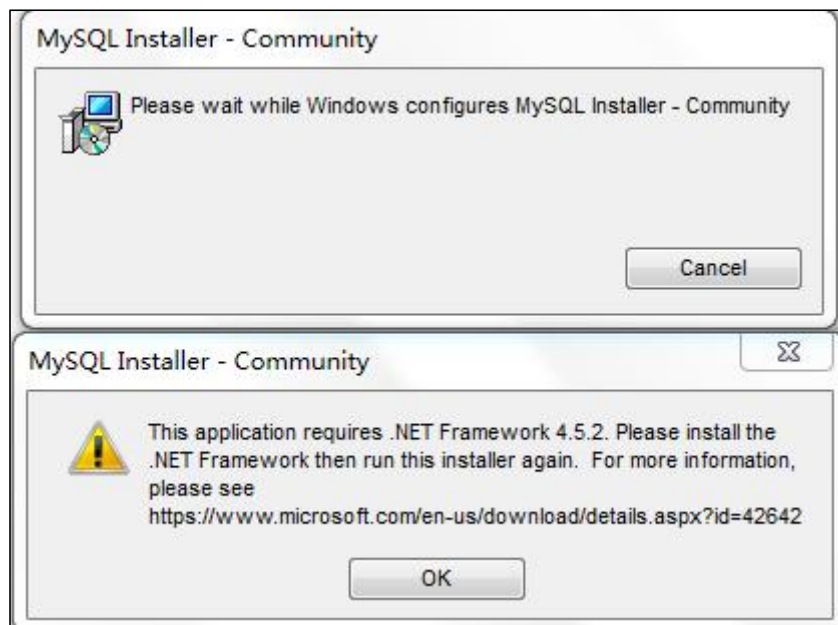
连接成功后，就可以对 MySQL 数据库进行管理了。



2.7 安装失败问题

2.7.1 无法打开 MySQL8.0 软件安装包

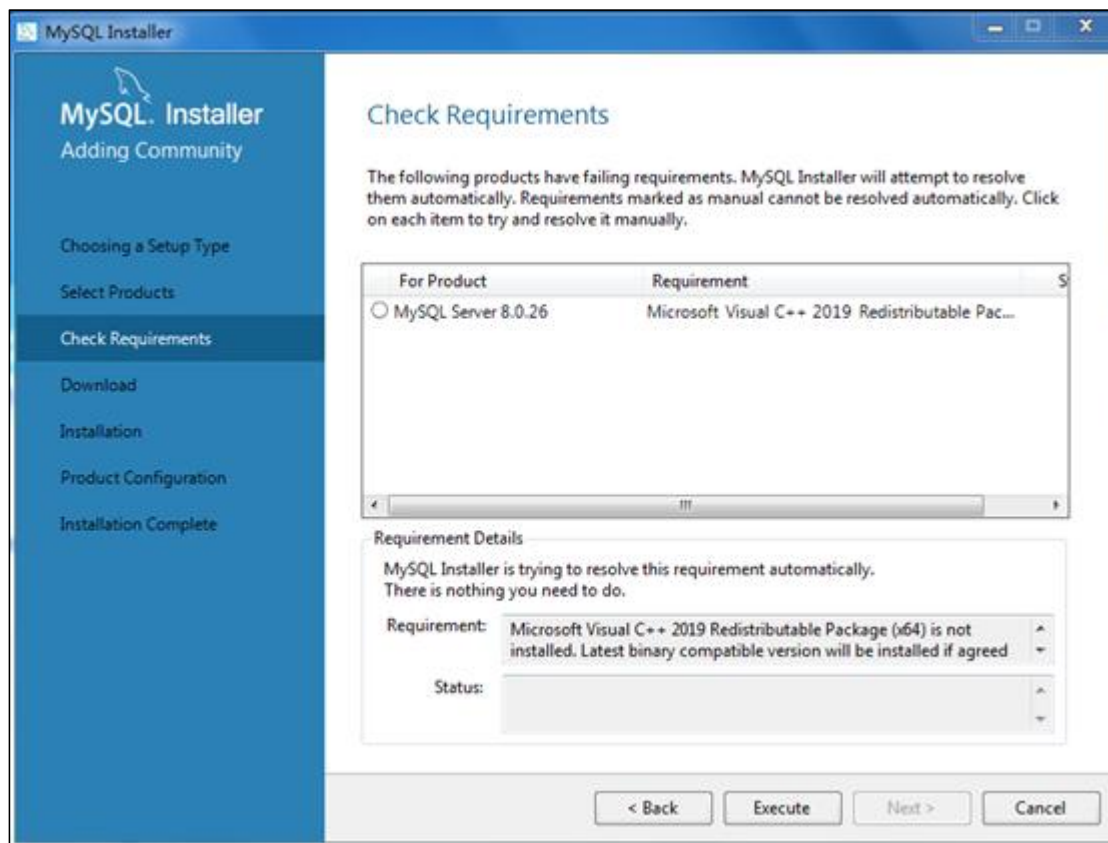
在运行 MySQL8.0 软件安装包之前，用户需要确保系统中已经安装了 .Net Framework 相关软件，如果缺少此软件，将不能正常地安装 MySQL8.0 软件。



解决方案：<https://www.microsoft.com/en-us/download/details.aspx?id=42642> 下载 Microsoft .NET Framework 4.5 并安装后，再安装 MySQL。

2.7.2 需要 C++库

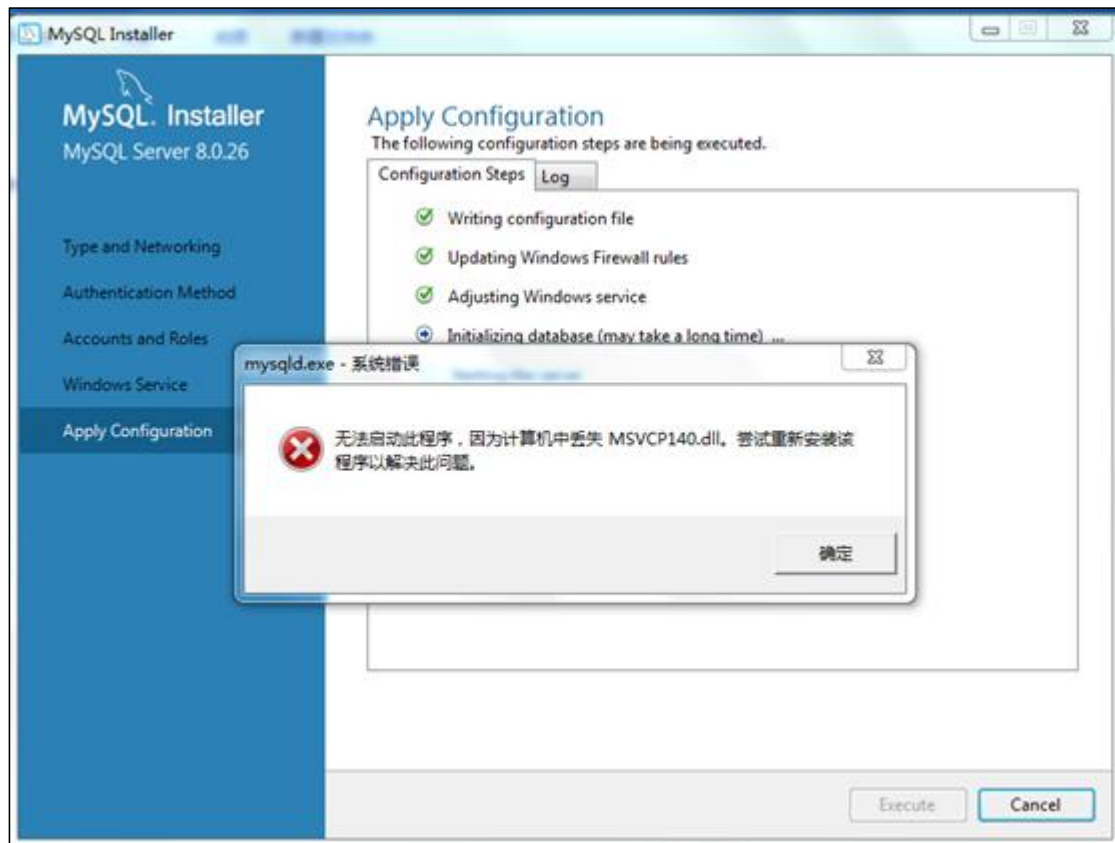
Windows 上安装 MySQL8.0 需要操作系统提前已安装好 Microsoft Visual C++ 2015-2022 Redistributable。



解 决 方 案 :

<https://support.microsoft.com/en-us/topic/the-latest-supported-visual-c-downloads-2647da03-1eea-4433-9aff-95f26a218cc0> 下载相应的环境。

2.7.3 丢失 MSVCP140.dll



解 决 方 案 :

<https://support.microsoft.com/en-us/topic/the-latest-supported-visual-c-downloads-2647da03-1eea-4433-9aff-95f26a218cc0> 下载相应的环境。

如果电脑提示需要更新操作系统，请更新后再安装。

2.8 MySQL 实例初始化失败问题

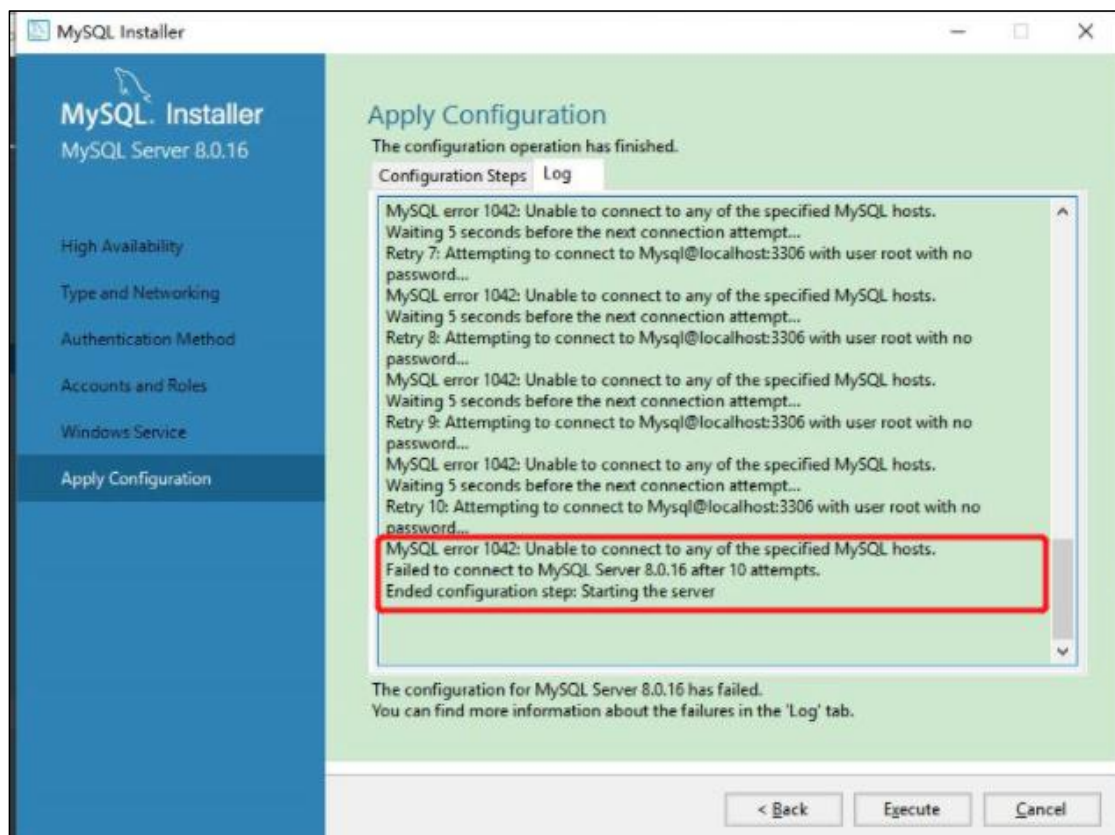
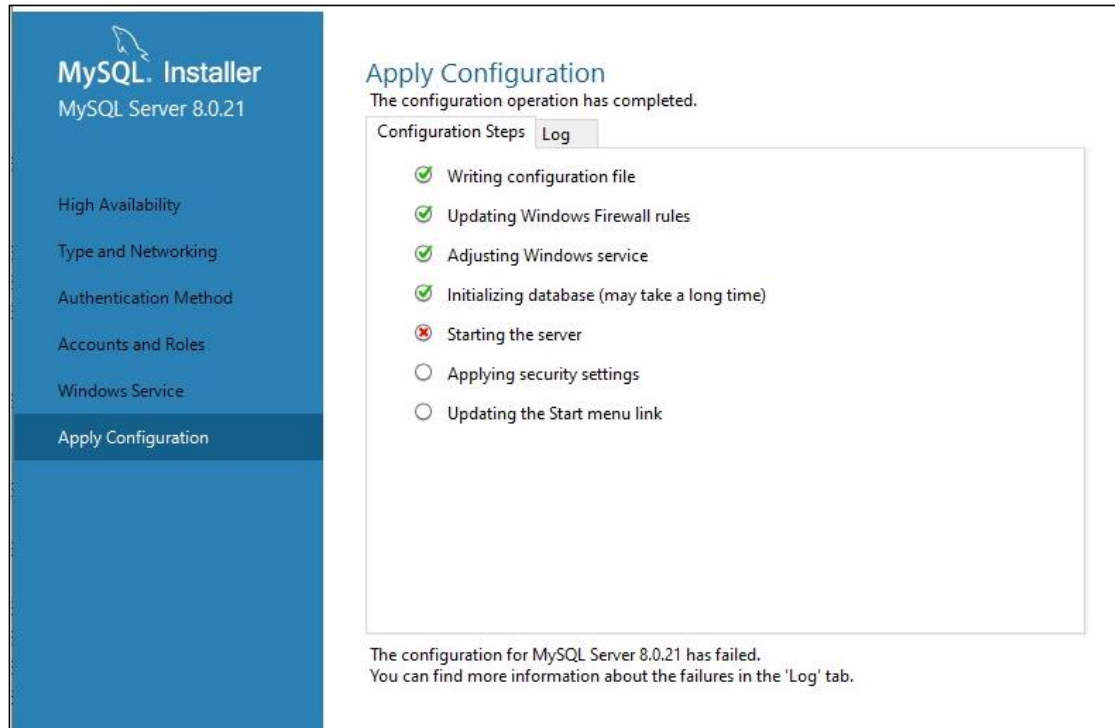
2.8.1 初始化系统库失败之中文乱码问题

例如：mysql: File '!.给ㄟ构褰?bin.index' not found (OS errno 2 - No such file or directory)

解决方案：设置→系统→系统信息→重命名这台电脑→重命名为非中文

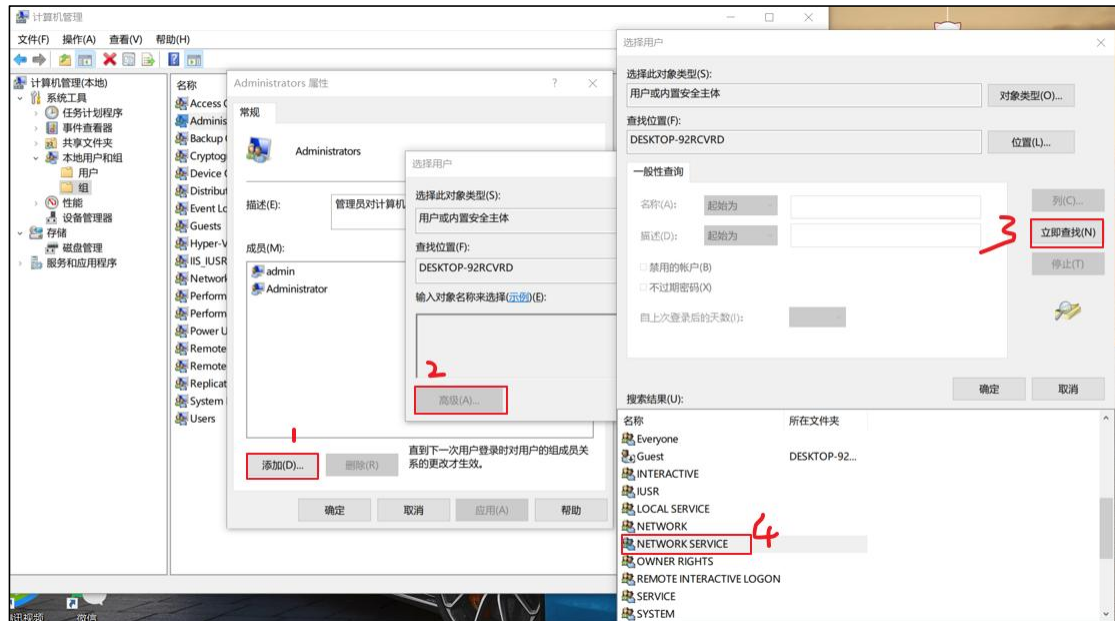
2.8.2 MySQL 服务启动失败

MySQL error 1042: Unable to connect to any of the specified MySQL hosts

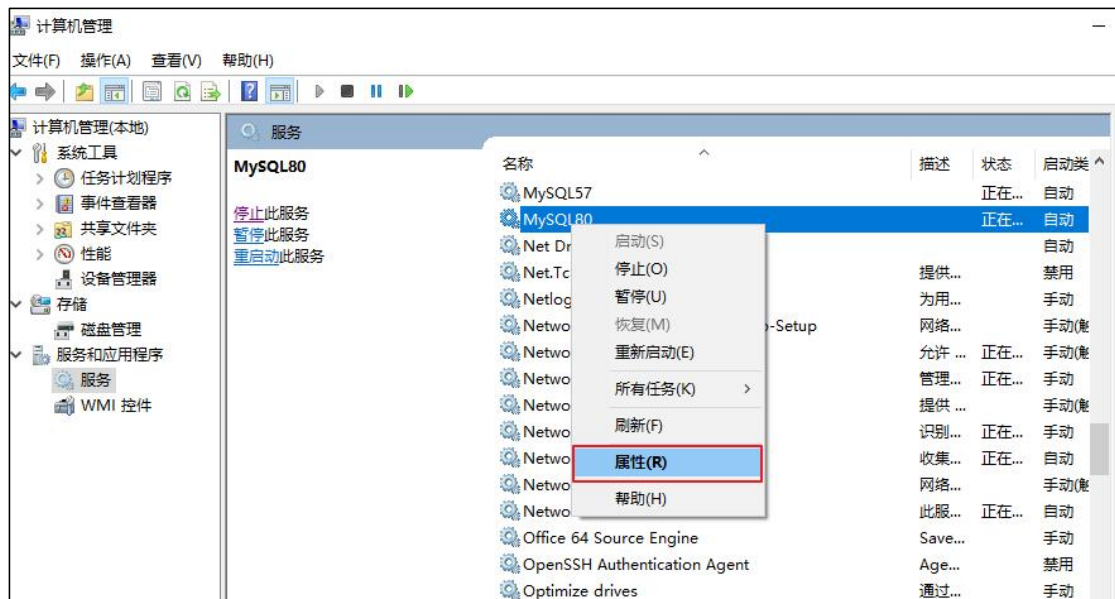


解决方案：

➤ 专业版操作系统：电脑→管理→本地用户和组→组→双击 Administrators→添加→高级→把 NETWORK SERVICE 添加到 Administrators 组



- 家庭版操作系统：Win + R，输入 services.msc → 找到 MySQL80（点击鼠标右键）
- 属性 → 登录选项卡下将选择的此账户改为选择本地系统账户 → 之后重新执行 `excute`





2.9 可视化工具连接 MySQL8 问题

有些图形界面工具，特别是旧版本的图形界面工具，在连接 MySQL8 时出现“Authentication plugin 'caching_sha2_password' cannot be loaded”错误。



出现这个原因是 MySQL8 之前的版本中加密规则是 `mysql_native_password`，而在 MySQL8 之后，加密规则是 `caching_sha2_password`。

解决方案：

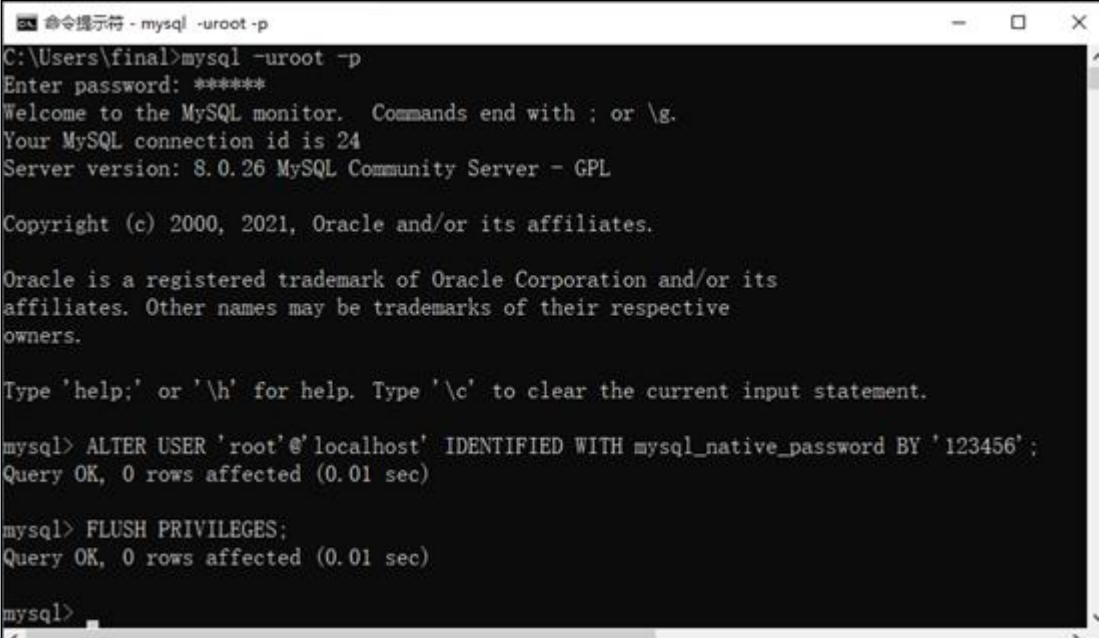
更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

- 第一种是升级图形界面工具版本
- 第二种是把 MySQL8 用户登录密码加密规则还原成 mysql_native_password

第二种解决方案如下：用命令行登录 MySQL 数据库之后，执行如下命令修改用户密码加密规则并更新用户密码，这里修改用户名为“root@localhost”的用户密码规则为“mysql_native_password”，密码值为“123456”。

错误 2058 和 2059 都是这样的方式去解决。

```
-- 修改'root'@'localhost'用户的密码规则和密码
ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password BY
'123456';
-- 刷新权限
FLUSH PRIVILEGES;
```



```
命令提示符 - mysql -uroot -p
C:\Users\final>mysql -uroot -p
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 24
Server version: 8.0.26 MySQL Community Server - GPL

Copyright (c) 2000, 2021, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password BY '123456';
Query OK, 0 rows affected (0.01 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.01 sec)

mysql>
```

2.10 MySQL8 忘记 root 用户密码

忘记 root 用户密码时，如果此时有其他用户拥有系统库 mysql 的 user 表的 UPDATE 权限，可以由其他用户通过 SET 语句修改 root 用户密码。但是如果没有其他用户，或者其他用户没有系统库 mysql 的 user 表的 UPDATE 权限，也没有 GRANT（给用户授权）的权限，那么怎么处理呢？操作步骤如下：

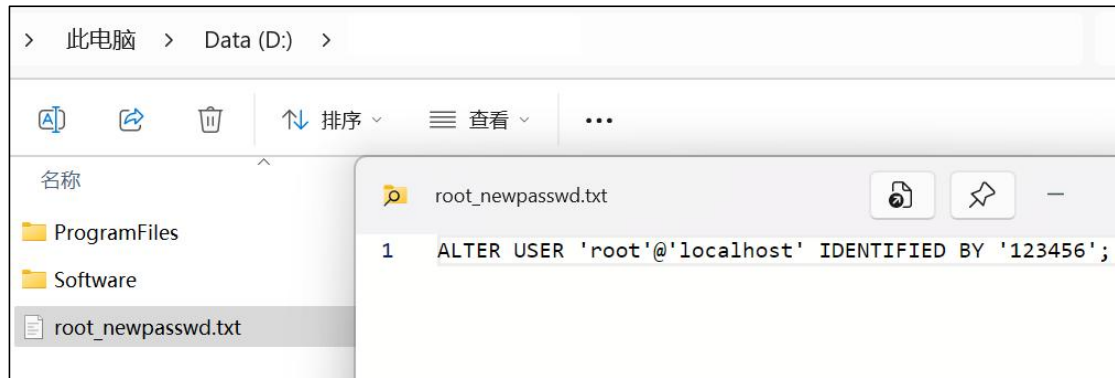
- （1）先停止 MySQL 服务

```
net stop mysql80
```

- （2）新建一个文本文件，文本文件中写一条修改密码的语句

```
ALTER USER 'root'@'localhost' IDENTIFIED BY '123456';
```

比如在 D 盘根目录下新建一个 root_newpasswd.txt 文件。



(3) 使用管理员权限运行 cmd 命令行，运行命令

```
mysqld --defaults-file="MySQL 安装路径的数据目录\my.ini" --init-file="d:\root_newpasswd.txt"
```

例如：

```
mysqld --defaults-file="D:\ProgramFiles\MySQL\MySQL80\my.ini" --init-file="d:\root_newpasswd.txt"
```

命令意思是初始化启动一次数据库，并运行这个修改密码的文件。效果演示如下：

```
C:\Windows\System32>mysqld --defaults-file="D:\ProgramFiles\MySQL\MySQL80\my.ini" --init-file="d:\root_newpasswd.txt"
C
C:\Windows\System32>
```

命令执行后，就像卡住了一样。按 **ctrl + C** 结束上面的运行命令。

(4) 重新启动 MySQL 服务，用新密码登录即可

2.11 修改用户密码

2.11.1 记得原密码

不登陆 MySQL，在命令行使用 **mysqladmin** 命令修改用户密码：

```
mysqladmin -u 用户名 -h 主机名 -p password 新密码
Enter password: 旧密码
```

```
> mysqladmin -uroot -p password 123321
Enter password: ****
```

2.11.2 不记得原密码

```
SET PASSWORD FOR '用户名'@'主机名' = '新密码';
```

例如：

root 用户登录后，创建一个新用户 **atguigu@localhost**，密码为 **atguigu**。

修改用户名为 **atguigu**，主机名为 **localhost** 的用户的密码为 **123456**。

```
CREATE USER 'atguigu'@'localhost' IDENTIFIED BY 'atguigu'; -- 创建 atguigu 用户
```



```
SET PASSWORD FOR 'atguigu'@'localhost' = '123456'; -- 修改 atguigu 用户密码
DROP USER 'atguigu'@'localhost'; -- 删除 atguigu 用户
```

```
mysql> CREATE USER 'atguigu'@'localhost' IDENTIFIED BY 'atguigu';
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> SET PASSWORD FOR 'atguigu'@'localhost' = '123456';
Query OK, 0 rows affected (0.01 sec)
```

```
mysql>
```

```
> mysql -uatguigu -p123456
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 21
Server version: 8.0.26 MySQL Community Server - GPL

Copyright (c) 2000, 2021, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
mysql> |
```

第 3 章 客户端使用演示

3.1 命令行客户端

3.1.1 数据库

```
-- 查看所有的数据库
show databases;

-- 创建 atguigu 数据库
create database atguigu;

-- 使用 atguigudb 数据库
use atguigu;
-- 如果没有使用 use 语句，后面针对数据库的操作也没有加 库名 的限定
-- 会报错 ERROR 1046 (3D000): No database selected (没有选择数据库)
-- 使用完 use 语句之后，如果接下来的 SQL 都是针对一个数据库操作的，那就不用重复 use 了
-- 如果要针对另一个数据库操作，那么要重新 use

-- 删除 atguigudb 数据库
drop database atguigu;
```

3.1.2 数据表

```
create database if not exists atguigu;
-- 查看 atguigu 库的所有表格，要求前面有 use 语句
```

```
use atguigu;
show tables;
-- 或
show tables from atguigu;

-- 创建学生表，用逗号分割每个字段
create table student(
    id int,
    name varchar(20)
);

-- 查看表结构
desc student;

-- 添加两条记录到学生表中
insert into student values(1,'张三');
insert into student values(2,'李四');

-- 查看学生表数据
select * from student;

-- 删除学生表
drop table student;
```

3.1.3 导入数据

命令行登录 MySQL，使用 source 命令导入。

注意：在使用命令行导入 SQL 脚本之前，请先确认 SQL 脚本中是否有 USE 语句，如果没有，那么在命令行中需要先使用 USE 语句指定具体的数据库，否则会报 “No database selected” 的错误。

```
mysql> source d:/练习数据.sql
Query OK, 0 rows affected, 1 warning (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

3.1.4 导出数据

命令行不登录 MySQL，使用 mysqldump 命令。

```
mysqldump -u 用户名 -p 密码 数据库名 > 脚本名.sql
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)


```
mysqldump -u 用户名 -p 密码 数据库名 表名 > 脚本名.sql
```

```
> mysqldump -uroot -p123321 atguigu > d:/atguigu.sql  
mysqldump: [Warning] Using a password on the command line interface can be insecure.
```

3.2 可视化客户端

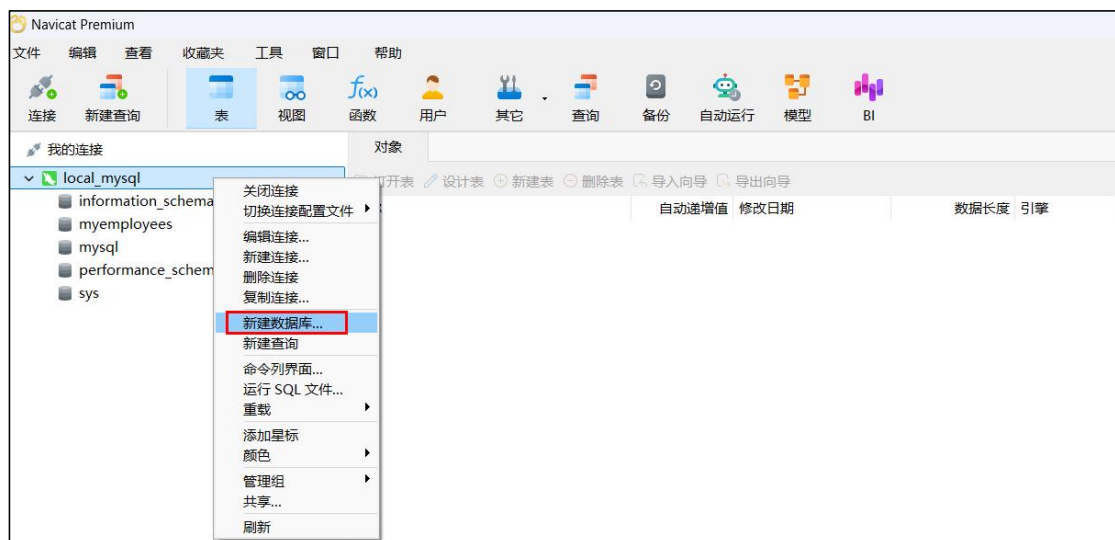
前文介绍了通过命令行来创建数据库和数据表，除此之外，还可以借助 MySQL 图形化工具，而且这种方式更加简单、方便。下面以 SQLyog 图形化工具为代表展开介绍。

3.2.1 数据库

以查看所有数据库的方式登录连接成功后，进入主界面，接下来正式创建数据库。

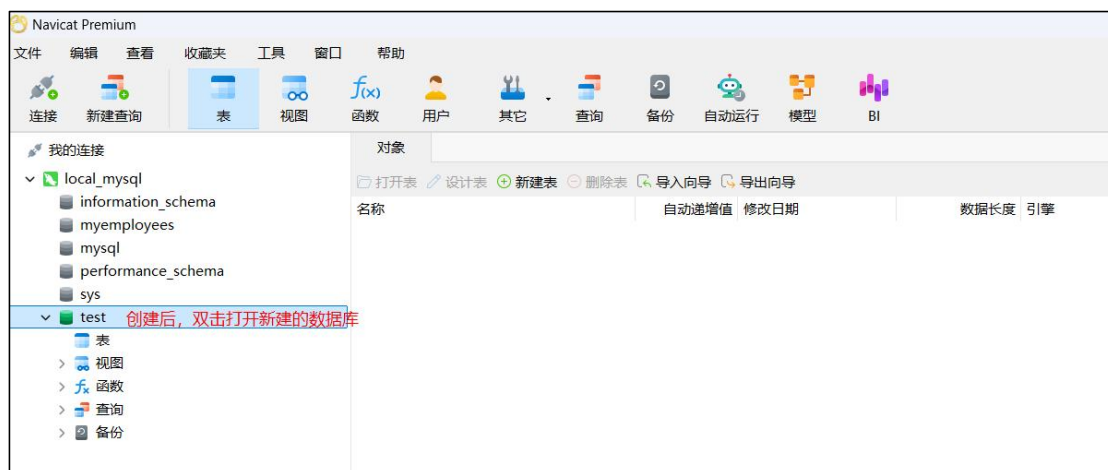
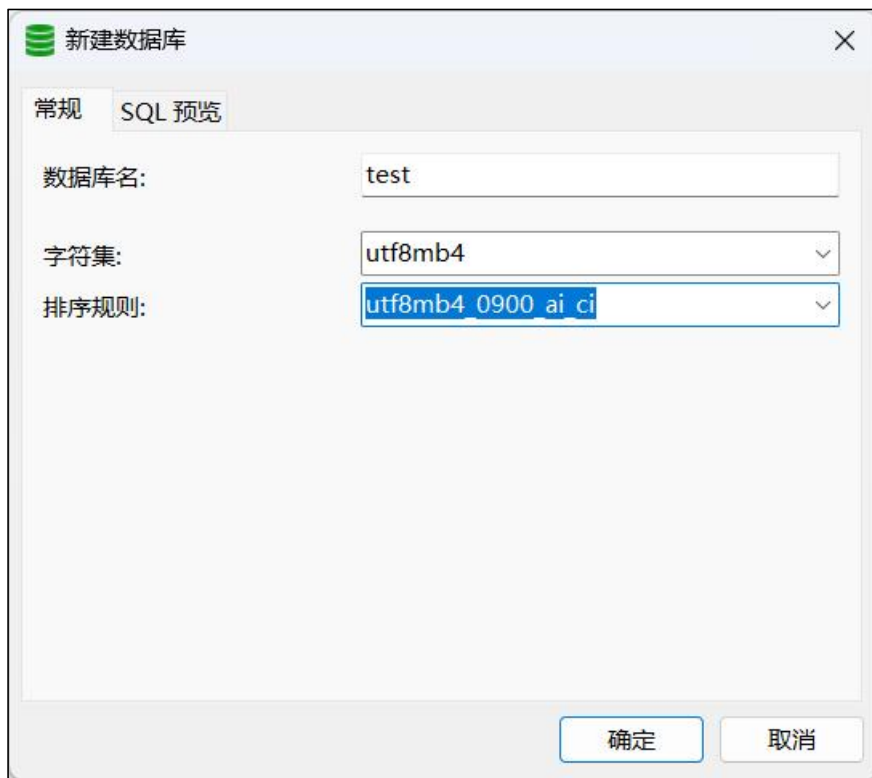
(1) 步骤 1

在主界面左边“导航窗口”选中数据库连接后，选择“新建数据库”菜单项。



(2) 步骤 2

填写新数据库的基本信息。一般只需填写数据库名称，例如“test”，字符集和排序规则有默认选项。如果有特殊要求，也可以选择自己需要的字符集和校对规则，然后点击“确定”按钮。



(3) 步骤 3

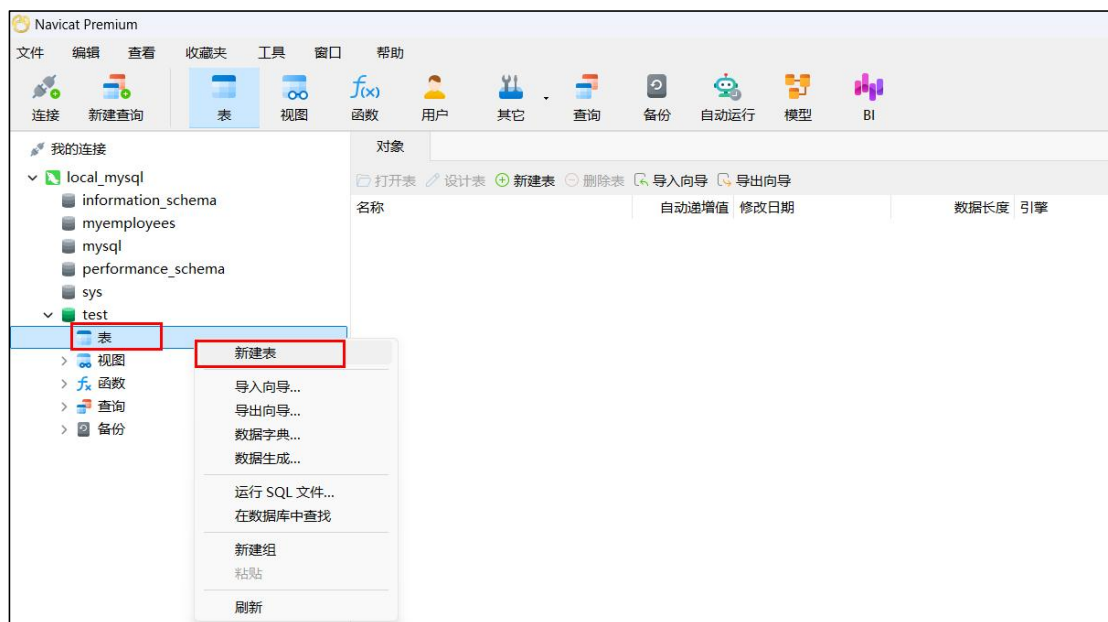
数据库创建成功之后，可以查看或修改数据库属性。选择“test”数据库，右键单击鼠标弹出快捷菜单，选择“编辑数据库”菜单选项，就可以查看或修改数据库属性。

3.2.2 数据表

数据库创建成功之后，就可以在这个数据库下面创建表了。

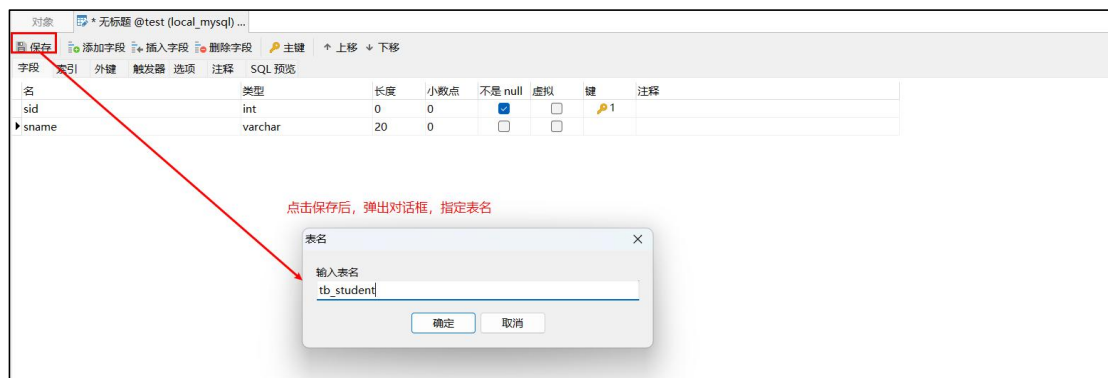
(1) 步骤 1

选择“test”数据库下的表对象，右键单击鼠标弹出快捷菜单，选择“新建表”菜单项。



(2) 步骤 2

点击“新建表”后，在新表的创建页面填写表名称和表的字段等属性信息。例如表名称为“tb_student”，并添加两个字段，一个是 int 类型的 sid，一个是 varchar(20)类型的 sname，分别表示学生编号和学生姓名。表的存储引擎、字符集和校对规则如果不选择，则默认和当前数据库一致。字段信息需要填写字段名和数据类型等基本信息，关于字段其他信息的含义请看后续章节讲解。



(3) 步骤 3

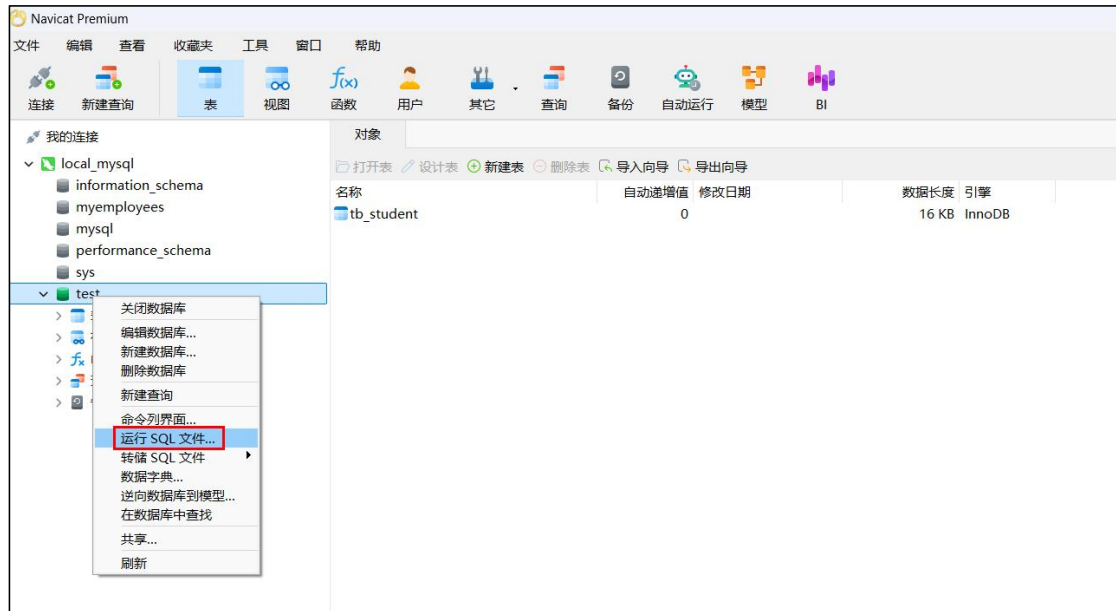
表创建成功之后，也可以查看数据表的定义信息或修改数据表的定义。在表名称（例如“tb_student”）上右键单击弹出的快捷菜单中选择“设计表”，就可以查看表结构的详细信息，也可以修改表的定义。

3.2.3 导入数据&执行 SQL 脚本

各种图形界面工具也都提供导入 SQL 脚本的功能。下面使用 Navicat 图形界面工具演示 SQL 脚本导入情况。

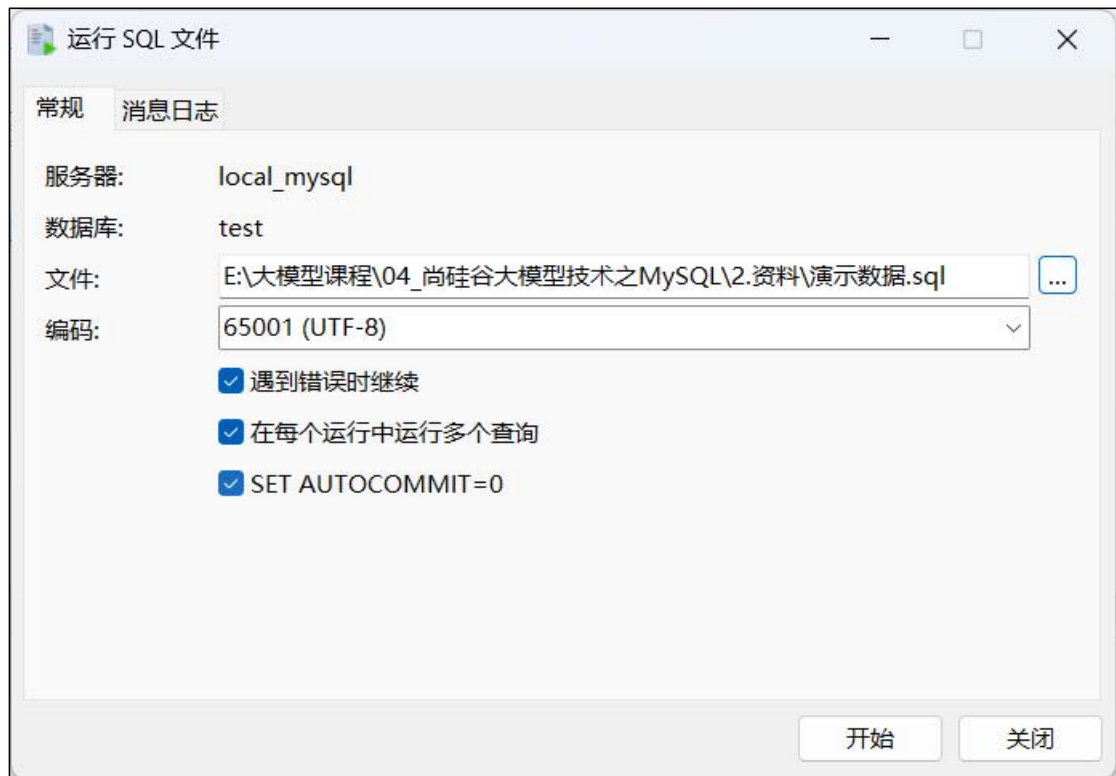
（1）步骤 1

选中数据库，然后单击鼠标右键，在弹出的菜单菜单中选择“运行 SQL 文件”选项。

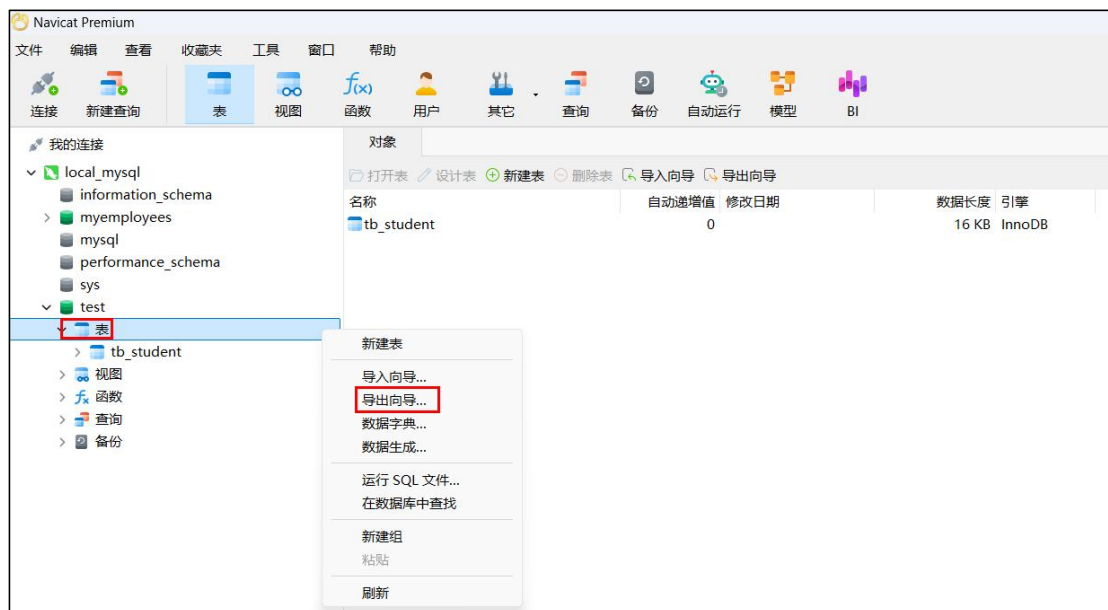


（2）步骤 2

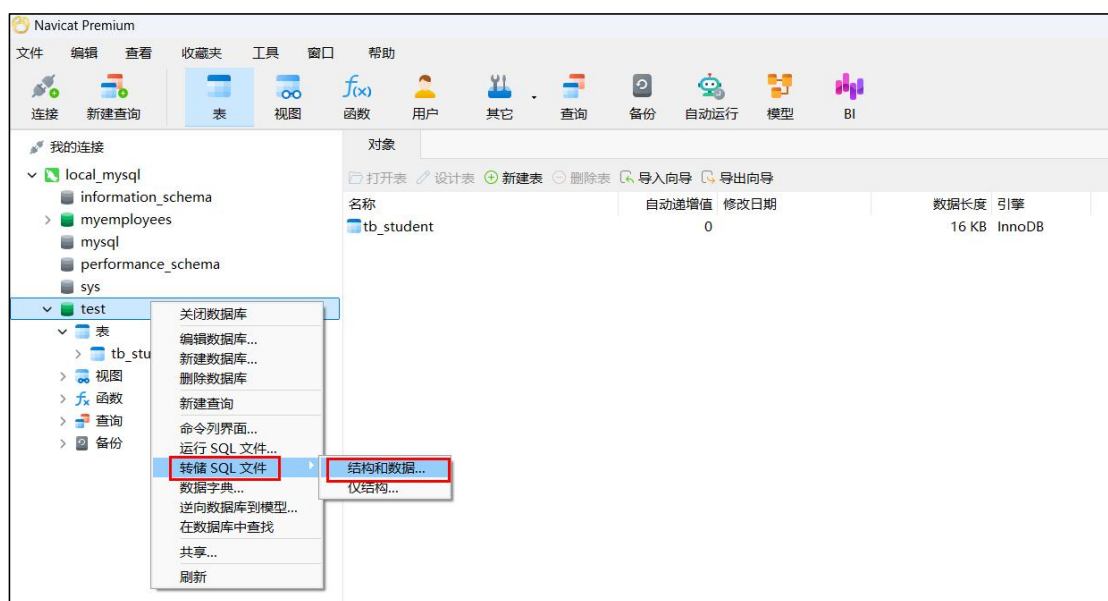
找到本地的 SQL 脚本文件，然后选中“开始”按钮。



3.2.4 导出表数据



3.2.5 导出整个数据库



第 4 章 SQL 语句简介

4.1 SQL 语句的分类

SQL 语句分为三种类型：

- DDL：数据定义语句（Data Define Language）如创建（create），修改（alter），删除（drop）等。

- DML: 数据操作语句, 如增 (insert), 删 (delete), 改 (update), 查 (select)。

因为查询语句使用的非常频繁, 因此很多人把查询语句单独划分为 DQL (数据查询语句)。

- DCL: 数据控制语句, 如 grant, commit, rollback 等。
- 其他语句: use 语句, show 语句, set 语句等。这类的官方文档中一般称为命令。

4.2 SQL 语法规范

- MySQL 的 SQL 语法不区分大小写。
- 命名时尽量使用 26 个英文字母大小写, 数字 0-9, 下划线, 不要使用其他符号。
- 建议不要使用 MySQL 的关键字等来作为表名、字段名、数据库名等, 如果不小心使用, 请在 SQL 语句中使用 ` (飘号) 引起来。
- 数据库和表名、字段名等对象名中间不要包含空格。
- 同一个 MySQL 软件中, 数据库不能同名, 同一个库中, 表不能重名, 同一个表中, 字段不能重名。

4.3 SQL 脚本中的注释

单行注释: # (MySQL 特有的) 或 -- 注释内容

```
show databases; #显示库
show databases; -- 显示库
```

多行注释: /* 注释内容 */

```
/*
显示库
*/
show databases;
```

第 5 章 DDL

5.1 库相关

5.1.1 创建库

```
create databases 数据库名;
```

5.1.2 查看库

1) 查看所有库

```
show databases;
```

2) 查看某个数据库的详细定义语句

```
show create database 数据库名;
```

5.1.3 修改库编码

修改数据库编码只会影响之后新创建的表的默认编码，之前创建的表不会受影响。

```
alter database 数据库名 character set 字符集名称 collate 字符集对应校对规则;
```

例如：

```
alter database db01 character set utf8 collate utf8_general_ci;
```

5.1.4 使用库

```
use 数据库名;
```

5.1.5 删除库

```
drop database 数据库名;
```

5.1.6 示例

```
-- 创建数据库
create database if not exists atguigu;
-- 查看所有数据库
show databases;
-- 查看数据库的创建语句
show create database atguigu;
-- 切换数据库
use atguigu;
-- 修改数据库的字符集为 Latin1, Latin1 字符集不支持中文
alter database atguigu character set Latin1 collate Latin1_general_ci;
-- 删除数据库
drop database if exists atguigu;
```

5.2 表相关

5.2.1 创建表

```
create table [if not exists] 表名(
    字段名 数据类型,
    字段名 数据类型
);
```

例如：

```
create table teacher(
    tid int,
    tname varchar(5),
    salary double,
    weight double(5,2),
    birthday date,
    gender enum('男','女'),
    blood enum('A','B','AB','O'),
```



```
tel char(11)
);
```

5.2.2 查看表

1) 查看某个数据库的所有表

```
use 数据库名;
show tables;
```

或

```
show tables from 数据库名;
```

2) 查看表的详细定义

```
show create table 表名;
```

3) 查看表结构

```
desc 表名;
```

5.2.3 修改表编码

```
alter table teacher character set 字符集 collate 校对规则;
```

例如:

```
alter table teacher character set utf8mb4 collate utf8mb4_general_ci;
```

5.2.4 修改表结构

1) 添加字段

```
alter table 表名 add 字段名 数据类型; -- 在表末尾添加字段
alter table 表名 add 字段名 数据类型 first; -- 在表开头添加字段
alter table 表名 add 字段名 数据类型 after 另一个字段; -- 在另一个字段后添加
字段
```

2) 删除字段

```
alter table 表名 drop 字段名;
```

3) 修改字段数据类型

```
alter table 表名 modify 字段名 新的数据类型;
```

4) 修改字段名

```
alter table 表名 change 旧字段名 新字段名 新的数据类型;
```

5) 修改字段位置

```
alter table 表名 modify 字段名 数据类型 first; -- 将字段移动到表的开头
alter table 表名 modify 字段名 数据类型 after 另一个字段; -- 将字段移动到表的
另一个字段之后
```

5.2.5 修改表名

```
alter table 旧表名 rename 新表名;
```

或

```
rename table 旧表名 to 新表名;
```

5.2.6 删除表

```
drop table [if exists] 表名;
```

5.2.7 示例

```
create database if not exists atguigu;
use atguigu;
-- 创建表
create table if not exists teacher(
    id int,
    name varchar(20),
    gender enum('男','女'),
    birthday date,
    salary double,
    tel varchar(11)
);
-- 查看数据库中所有表
show tables;
-- 查看表结构
desc teacher;
-- 查看建表语句
show create table teacher;
-- 添加字段到表末尾
alter table teacher add address varchar(200);
-- 添加字段到表开头
alter table teacher add age int first;
-- 添加字段到指定位置
alter table teacher add heart_rate float after age;
-- 删除字段
alter table teacher drop address;
-- 修改字段数据类型
alter table teacher modify heart_rate double;
-- 修改字段名
alter table teacher change heart_rate bpm double;
-- 移动字段位置到表开头
alter table teacher modify bpm double first;
-- 移动字段位置到另一个字段之后
alter table teacher modify bpm double after age;
-- 修改表名
alter table teacher rename teacher_info;
-- 删除表
drop table teacher_info;
```

第 6 章 DML

6.1 添加

6.1.1 只写表名

值列表需与表结构对应。

```
insert into 表名 values(值列表),(值列表),(值列表); -- 值列表中的值的顺序、类型、个数必须与表结构一一对应
```

6.1.2 写表名和字段列表

值列表需与字段列表对应。

```
insert into 表名(字段列表) values(值列表),(值列表),(值列表); -- 值列表中的值的顺序、类型、个数必须与字段列表一一对应
```

6.1.3 示例

```
create database if not exists atguigu;
use atguigu;
drop table if exists teacher;
create table teacher(
    id int,
    name varchar(20),
    gender enum('男','女'),
    birthday date,
    salary double,
    tel varchar(11)
);
-- 插入一条数据
insert into teacher values(1,'张三','男',
    '1990-01-01',10000,'12345678901');
insert into teacher(id,name,gender) values(2,'李四','男');
select * from teacher;
```

id int	name varchar	gender string	birthday date	salary double	tel varchar
1	张三	男	1990-01-01	10000	12345678901
2	李四	男	(NULL)	(NULL)	(NULL)

```
-- 插入多条数据
insert into teacher values
(3,'王五','男','1990-01-01',10000,'12345678901'),
(4,'赵六','男','1990-01-01',10000,'12345678901'),
(5,'孙七','男','1990-01-01',10000,'12345678901');
insert into teacher(id,name) values(6,'周八'),(7,'吴九'),(8,'郑十');
```

```
select * from teacher;
```

id int	name varchar	gender string	birthday date	salary double	tel varchar
1	张三	男	1990-01-01	10000	12345678901
2	李四	男	(NULL)	(NULL)	(NULL)
3	王五	男	1990-01-01	10000	12345678901
4	赵六	男	1990-01-01	10000	12345678901
5	孙七	男	1990-01-01	10000	12345678901
6	周八	(NULL)	(NULL)	(NULL)	(NULL)
7	吴九	(NULL)	(NULL)	(NULL)	(NULL)
8	郑十	(NULL)	(NULL)	(NULL)	(NULL)

6.2 删除

6.2.1 删除多条数据

```
delete from 表名 where 条件;
```

6.2.2 删除表中所有数据

表结构会保留下来。

```
delete from 表名;
```

6.2.3 截断表

表结构会保留下来。

```
truncate 表名;
```

truncate 表和 delete 表的区别：

- delete 是一条一条删除记录的。如果在事务中，事务提交之前支持回滚
- truncate 是把整个表 drop，新建一张，效率更高。就算在事务中，也无法回滚

6.2.4 示例

```
-- 删除姓名为郑十的记录
delete from teacher where name="郑十";
select * from teacher;
-- 删除所有记录
delete from teacher;
-- 截断表
truncate teacher;
```

6.3 修改

6.3.1 修改部分行数据

```
update 表名 set 字段名 = 值, 字段名 = 值 where 条件; -- 修改满足条件的行
```

6.3.2 修改所有行数据

```
update 表名 set 字段名 = 值, 字段名 = 值; -- 修改所有行
```

6.3.3 示例

```
-- 修改周八的信息
update teacher set gender="男", birthday="1960-01-01", salary=100000, tel="12332132131" where name="周八";
-- 修改所有人的薪资
update teacher set salary=salary+10000;
```

6.4 查询

6.4.1 select 语句

select 语句是用于查看计算结果、或者查看从数据表中筛选出的数据的。

select 语句的基本语法：

```
select 常量;
select 表达式;
select 函数;
```

例如：

```
select 1;
select 9/2;
select now();
```

如果要从数据表中筛选数据，需要加 from 子句。from 指定数据来源。字段列表筛选列。

```
select 字段列表 from 表名;
```

如果要从数据表中根据条件筛选数据，需要加 from 和 where 子句。where 筛选行。

```
select 字段列表 from 表名 where 条件;
```

完整的 select 语句后面可以跟 7 个子句，后面会逐一讲解。

6.4.2 使用别名

在当前 select 语句中给某个字段或表达式计算结果，或表等取个临时名称，便于当前 select 语句的编写和理解。这个临时名称称为别名。

```
select 字段名 1 as "别名 1", 字段名 2 as "别名 2" from 表名 as 表别名;
```

- 列的别名有空格时需要加双引号。列的别名中没有空格时双引号可以不加
- 表的别名不能加双引号，表的别名中间不能包含空格

➤ as 大小写都可以，as 也完全可以省略

例如：

```
select id as 编号,name "姓 名" from teacher;
```

6.4.3 结果去重

mysql 可以在查询结果中使用 distinct 关键字去重。

```
select distinct 字段列表 from 表名;
```

第 7 章 运算符

7.1 算数运算符

运算符	说明
+	求和，MySQL 中 + 没有字符串拼接功能
-	减
*	乘
/	除
div	整除，只保留整数部分
%	模，取余

示例：

```
select 2+3;
select 9/2,9 div 2;
select 10.1/2.1,10.1 div 2.1;
select 9%2,9 mod 2;
select "hello"+"world";
-- NULL 与其他数据类型的运算结果为 NULL
select 2+null;
select "hello"+null;
select date("2020-01-01")+null;
```

7.2 比较运算符

运算符	说明
>	大于
<	小于
>=	大于等于
<=	小于等于

=	等于，不能用于 NULL 判断
!= 或 <>	不等于，不能用于 NULL 判断

示例：

```
use atguigu;
-- 查询薪资高于 15000 的员工姓名和薪资
select ename,salary from t_employee where salary>15000;
-- 查询薪资为 9000 的员工的姓名和薪资
select ename,salary from t_employee where salary=9000;
-- 查询地址不在北苑的
select * from t_employee where address!="北苑";
select * from t_employee where address<>"北苑";
```

7.3 区间或集合范围比较运算符

运算符	说明
between x and y	处于区间内
not between x and y	不处于区间内
in(x,y,z)	处于集合内
not in (x,y,z)	不处于集合内

示例：

```
use atguigu;
-- 查询薪资在[10000,15000]
select * from t_employee where salary>=10000 && salary<=15000;
select * from t_employee where salary between 10000 and 15000;
-- 查询薪资不在[10000,15000]
select * from t_employee where salary not between 10000 and 15000;
-- 查询地址在这几个地方的
select * from t_employee where address in ('北苑', '望京', '龙泽');
-- 查询地址不在这几个地方的
select * from t_employee where address not in ('北苑', '望京', '龙泽');
```

7.4 模糊匹配比较运算符

运算符	说明
%	代表任意个字符
_	代表 1 个字符，如果 2 个下划线就代表 2 个字符

示例：

```
use atguigu;
-- 查询名字有 冰 字的
```



```
select * from t_employee where ename like '%冰%';
-- 查询名字以 雨 结尾的
select * from t_employee where ename like '%雨';
-- 查询名字以 李 开头的
select * from t_employee where ename like '李%';
-- 查询名字有 红 这个字，但是 红 的前面只能有 1 个字
select * from t_employee where ename like '_红%';
-- 查询当前 MySQL 数据库的字符集情况
show variables like '%character%';
```

7.5 逻辑运算符

运算符	说明
&& 或 and	逻辑与
或 or	逻辑或
! 或 not	逻辑非
xor	逻辑异或

示例：

```
use atguigu;
-- 查询薪资高于 15000，并且性别是男的员工
select * from t_employee where salary>15000 and gender='男';
-- 查询薪资高于 15000，或者 did 为 1 的员工
select * from t_employee where salary>15000 || did = 1;
-- 查询薪资不在[15000,20000]范围的
select * from t_employee where !(salary between 15000 and 20000);
-- 查询薪资高于 15000，或者 did 为 1 的员工，两者只能满足其一
select * from t_employee where salary>15000 xor did = 1;
select * from t_employee where (salary>15000) ^ (did = 1);
```

7.6 关于 NULL 值的问题

运算符	说明
is null 或 <=>null	是 NULL
is not null	不是 NULL
ifnull(x,y)	当 x 是 NULL 时，用 y 代替

示例：

```
use atguigu;
-- 查询奖金比例为 null 的员工
select * from t_employee where commission_pct is null;
select * from t_employee where commission_pct <=> null;
```

```
-- 查询奖金比例不为 null 的员工
select * from t_employee where commission_pct is not null;
-- 查询员工的实发工资，实发工资 = 薪资 + 薪资 * 奖金比例
select ename ,salary , commission_pct, salary + salary *
ifnull(commission_pct,0) "实发工资" from t_employee;
```

7.7 位运算符

运算符	说明
<<	左移
>>	右移
&	按位与
	按位或
^	按位异或
~	按位取反

第 8 章 MySQL 数据类型

8.1 数值类型

8.1.1 整数类型

数据类型	字节	范围 (有符号)	范围 (无符号)	说明
TINYINT	1	-128 ~ 127	0 ~ 255	小范围整数
SMALLINT	2	-32,768 ~ 32,767	0 ~ 65,535	中小范围整数
MEDIUMINT	3	-8,388,608 ~ 8,388,607	0 ~ 16,777,215	中范围整数
INT/INTEGER	4	-2,147,483,648 ~ 2,147,483,647	0 ~ 4,294,967,295	标准整数类型
BIGINT	8	-2 ⁶³ ~ 2 ⁶³ -1	0 ~ 2 ⁶⁴ -1	大范围整数

对于整数类型，MySQL 还支持在类型名称后面加小括号(M)，而小括号中的 M 表示显示宽度，M 的取值范围是(0, 255)。int(M)这个 M 在字段的属性中指定了 unsigned（无符号）和 zerofill（零填充）的情况下才有意义。表示当整数值不够 M 位时，用 0 填充。如果整数值超过 M 位但是没有超过当前数据类型的范围时，就按照实际位数存储。当 M 宽度超过当前数据类型可存储数值范围的最大宽度时，也是以实际存储范围为准。

MySQL8 之前，int 没有指定(M)，默认显示(11)。最多能存储和显示 11 位整数。从 MySQL 8.0.17 开始，整数数据类型不推荐使用显示宽度属性，默认显示 int。

```
-- 创建表
```

```
create table t_int(
    i1 int,
    i2 int(2) -- 没有 unsigned zerofill, (2)没有意义
);
-- 查看表结构
desc t_int;
-- 创建表
create table t_int2(
    i1 int,
    i2 int(2) unsigned zerofill
);
-- 查看表结构
desc t_int2;
-- 添加数据
insert into t_int2 values(1234,1234);
-- 查询数据
select * from t_int2;
-- 添加数据
insert into t_int2 values(1,1);
-- 查询数据
select * from t_int2;
```

8.1.2 浮点数类型

数据类型	字节	范围	说明
FLOAT	4	$\pm 3.402823466E+38$ (~7位精度)	单精度浮点数
DOUBLE	8	$\pm 1.7976931348623157E+308$ (~16位精度)	双精度浮点数
DECIMAL/ NUMERIC	可变	用户定义精度和范围	精确小数，常用于财务计算

MySQL 中使用浮点数和定点数来表示小数。浮点数有两种类型：单精度浮点数(FLOAT)和双精度浮点数(DOUBLE)，定点数只有 DECIMAL。

浮点数和定点数都可以用 (M, D) 来表示：

➤ M 是精度，表示该值总共显示 M 位，包括整数位和小数位，对于 FLOAT 和 DOUBLE 类型来说，M 取值范围为 0~255，而对于 DECIMAL 来说，M 取值范围为 0~65。

➤ D 是标度，表示小数的位数，取值范围为 0~30，同时必须 $\leq M$ 。

FLOAT 和 DOUBLE 类型在不指定 (M, D) 时，默认会按照实际的精度来显示。DECIMAL 类型在不指定 (M, D) 时，默认为 (10, 0)，即只保留整数部分。例如，定义 DECIMAL (5,2) 的类型，表示该列取值范围是 -999.99~999.99。如果用户插入数据的小数部分位数超

过 D 位，MySQL 会四舍五入处理，但是如果用户插入数据的整数部分位数超过 M-D 位，则会报 Out of range 的错误。

DECIMAL 实际是以字符串形式存放的，在对精度要求比较高的时候（如货币、科学数据等）使用 DECIMAL 类型会比较好。浮点数相对于定点数的优点是在长度一定的情况下，浮点数能够表示更大的数据范围，它的缺点是会引起精度问题。

```
-- 创建表
create table t_double(
    d1 double,
    d2 double(5,2) -- -999.99~999.99
);
-- 查看表结构
desc t_double;
-- 添加数据
insert into t_double values(2.5,2.5);
-- 查询数据
select * from t_double;
-- 添加数据
insert into t_double values(2.5526,2.5526);
insert into t_double values(2.5586,2.5586);
-- 查询数据
select * from t_double;
-- 添加数据
insert into t_double values(12852.5526,12852.5526);
-- d2 字段整数部分超过(5-2=3)位，添加失败
-- 创建表
create table t_decimal(
    d1 decimal, -- 没有指定(M,D)默认是(10,0)
    d2 decimal(5,2)
);
-- 查看表结构
desc t_decimal;
-- 添加数据
insert into t_decimal values(2.5,2.5);
-- 查询数据
select * from t_decimal;
-- 把小数赋值给整数类型的字段时，会截断小数部分，考虑四舍五入
insert into t_int2 values(1.5,1.5);
select * from t_int2;
```

8.1.3 BIT 类型

BIT 是一种存储位数据的类型，每个字段的长度可以指定为 1 到 64 位。数据以位流的形式存储，并以字节（byte）对齐。即使只存储 1 位，最小存储单元也是 1 字节。

可以使用十六进制、二进制或整数形式插入数据。插入值的位数不能超过定义的位数，否则会报错。

查询 BIT 字段时，MySQL 会以二进制格式显示数据。如果需要以整数形式显示，可以使用 BIN() 或 CONV()。

可以对 BIT 数据进行逻辑运算，例如 AND，OR，XOR 等。

```
-- 创建表
create table t_bit(
    b1 bit, -- 没有指定(M)，默认是 1 位二进制
    b2 bit(4) -- 能够存储 4 位二进制 0000~1111
);
-- 查看表结构
desc t_bit;
-- 添加数据
insert into t_bit values(1,1);
-- 查询数据
select * from t_bit;
-- 显示二进制值，需要使用 bin 函数
select bin(b1),bin(b2) from t_bit;
-- 添加数据
insert into t_bit values(2,2);
-- values()中是十进制值，需要转为二进制存储，2 对应 10，超过 1 位，b1 存不下
-- 添加数据
insert into t_bit values(1,8);
-- 查看数据
select * from t_bit;
-- 显示二进制值，需要使用 bin 函数
select conv(b1,2,10),conv(b2,2,10) from t_bit;
-- 添加数据
insert into t_bit values(1,16); -- 16 的二进制 10000
```

8.2 字符串类型

8.2.1 定长与变长字符串

数据类型	最大长度	说明
CHAR(M)	0 ~ 255 字符	定长字符串，长度不足会补空格
VARCHAR(M)	0 ~ 65535 字节	变长字符串，存储长度需加1或2字节

CHAR(M)为固定长度的字符串，M 表示最多能存储的字符数，取值范围是 0~255 个字符，如果未指定(M)表示只能存储 1 个字符。例如 CHAR(4)定义了一个固定长度的字符串列，其包含的字符个数最大为 4，如果存储的值少于 4 个字符，右侧将用空格填充以达到指定的长度，当查询显示 CHAR 值时，尾部的空格将被删掉。

VARCHAR(M)为可变长度的字符串，M 表示最多能存储的字符数，M 的范围由最长的行的大小（通常是 65535）和使用的字符集确定。例如 utf8mb4 字符编码单个字符所需最长字节值为 4 个字节，所以 M 的范围是[0, 16383]。而 VARCHAR 类型的字段实际占用的空间为字符串的实际长度加 1 或 2 个字节，这 1 或 2 个字节用于描述字符串值的实际字节数，即字符串值在[0,255]个字节范围内，那么额外增加 1 个字节，否则需要额外增加 2 个字节。在 VARCHAR 后面的 M 必须指定。

例如，身份证号、手机号码、QQ 号、用户名 username、密码 password、银行卡号等固定长度的文本字符串适合使用 CHAR 类型，而评论、朋友圈、微博不定长度的文本字符串更适合使用 VARCHAR 类型。

另外，存储引擎对于选择 CHAR 和 VARCHAR 是有影响的：

- 对于 MyISAM 存储引擎，最好使用固定长度的数据列代替可变长度的数据列。这样可以使整个表静态化，从而使数据检索更快，用空间换时间。

- 对于 InnoDB 存储引擎，使用可变长度的数据列，因为 InnoDB 数据表的存储格式不分固定长度和可变长度，因此使用 CHAR 不一定比使用 VARCHAR 更好，但由于 VARCHAR 是按照实际的长度存储的，比较节省空间，所以对磁盘 I/O 和数据存储总量比较好。

```
drop table if exists t_char;
create table t_char (
    c1 char,
    c2 char(3)
);
insert into t_char values('男','女'); -- 成功
insert into t_char values('尚硅谷','尚硅谷'); -- 失败
insert into t_char values('男','尚硅谷'); -- 成功
select * from t_char;

drop table if exists t_char;
create table t_char (
    c1 varchar -- 错误
);
```

```
create table t_char (
    c1 varchar(3) -- 最多不超过 3 个字符
);
insert into t_char values('尚硅谷');
insert into t_char values('尚硅谷真好'); -- 失败
drop table if exists t_char;
create table t_char (
    name varchar(65535)
);
-- 错误，字符串过长
```

8.2.2 枚举与集合

特性	ENUM	SET
存储值个数	单选（一个值）	多选（一个或多个值）
存储方式	数字索引	位图
选项个数限制	最多 65,535 个	最多 64 个
插入非法值	转为空字符串或报错	忽略非法值
适用场景	需要单选的状态字段（如性别）	多选标签（如用户角色）

有时候我们指定在固定的几个值范围内选择一个或多个，那么就需要使用 ENUM 枚举类型和 SET 集合类型了。比如性别只有“男”或“女”；上下班交通方式可以有“地铁”、“公交”、“出租车”、“自行车”、“步行”等。

枚举和集合类型字段声明的语法格式如下：

- 字段名 ENUM(值 1,值 2,值 3)
- 字段名 SET(值 1,值 2,值 3)

ENUM 类型的字段在赋值时，只能在指定的枚举列表中取值，而且一次只能取一个。枚举列表最多可以有 65535 个成员。ENUM 值在内部用整数表示，每个枚举值均有一个索引值，MySQL 存储的就是这个索引编号。

SET 类型的字段在赋值时，可从定义的值列表中选择 1 个或多个值的组合。SET 列最多可以有 64 个成员。SET 值在内部也用整数表示，分别是 1, 2, 4, 8……，都是 2 的 n 次方值，因为这些整数值对应的二进制都是只有 1 位是 1，其余是 0。

```
drop table if exists t_enum;
create table t_enum (
    gender enum('男','女'),
    hobby set('睡觉','打游戏','运动','写代码')
);
```



```
desc t_enum;
insert into t_enum values('男','睡觉,打游戏'); -- 成功
insert into t_enum values('男,女','睡觉,打游戏'); -- 失败
insert into t_enum values('妖','睡觉,打游戏'); -- 失败
insert into t_enum values('男','睡觉,打游戏,吃饭'); -- 失败
select * from t_enum;
insert into t_enum values(2, 2);
select * from t_enum;
insert into t_enum values(1, 5);
-- 5(0101)是 1(0001)、4(0100)的组合
select * from t_enum;
insert into t_enum values(1, 7);
-- 7(0111)是 1(0001)、2(0010)、4(0100)的组合
select * from t_enum;
insert into t_enum values(2, 15);
select * from t_enum;
```

8.2.3 文本类型

数据类型	最大长度	说明
TINYTEXT	255 字节	小文本
TEXT	65,535 字节 (64 KB)	常规文本
MEDIUMTEXT	16,777,215 字节	中等大小文本
LONGTEXT	4,294,967,295 字节	超大文本

8.2.4 二进制类型

数据类型	最大长度	说明
BINARY(M)	0 ~ 255 字节	定长二进制数据
VARBINARY(M)	0 ~ 65535 字节	变长二进制数据
BLOB	0 ~ 4 GB	用于存储二进制大对象

BINARY 和 VARBINARY 类似于 CHAR 和 VARCHAR，只是它们存储的是二进制字符串。

BINARY (M)为固定长度的二进制字符串，M 表示最多能存储的字节数，取值范围是 0~255 个字节，如果未指定(M)表示只能存储 1 个字节。例如 BINARY (8)，表示最多能存储 8 个字节，如果字段值不足(M)个字节，将在右边填充'\0'以补齐指定长度。

VARBINARY (M)为可变长度的二进制字符串，M 表示最多能存储的字节数，总字节数不能超过行的字节长度限制 65535，另外还要考虑额外字节开销，VARBINARY 类型的数据

除了存储数据本身外，还需要 1 或 2 个字节来存储数据的字节数。VARBINARY 类型和 VARCHAR 类型一样必须指定(M)，否则报错。

8.3 日期时间类型

数据类型	字节	范围	格式	说明
DATE	3	1000-01-01 ~ 9999-12-31	YYYY-MM-DD	日期
DATETIME	8	1000-01-01 00:00:00 ~ 9999-12-31 23:59:59	YYYY-MM-DD HH:MM:SS	日期时间，精确到秒
TIMESTAMP	4	1970-01-01 00:00:01 UTC ~ 2038-01-19 03:14:07 UTC	YYYY-MM-DD HH:MM:SS	时间戳，受时区影响
TIME	3	-838:59:59 ~ 838:59:59	HH:MM:SS	时间
YEAR	1	1901 ~ 2155	YYYY	年份

如果仅仅是表示年份信息，可以只使用 YEAR 类型，这样更节省空间，格式为“YYYY”，例如“2022”。YEAR 允许的值范围是 1901~2155。YEAR 还有格式为“YY”2 位数字的形式，值是 00~69，表示 2000~2069 年，值是 70~99，表示 1970~1999 年，从 MySQL5.5.27 开始，2 位格式的 YEAR 已经不推荐使用。YEAR 默认格式就是“YYYY”，没必要写成 YEAR(4)，从 MySQL 8.0.19 开始，不推荐使用指定显示宽度的 YEAR(4)数据类型。这个 0 年，如果是整数的 0 添加的话，那么是 0000 年，如果是日期/字符串的'0'添加的话，是 2000 年。

如果要表示年月日，可以使用 DATE 类型，格式为“YYYY-MM-DD”，例如“2022-02-04”。

如果要表示时分秒，可以使用 TIME 类型，格式为“HH:MM:SS”，例如“10:08:08”。

如果要表示年月日时分秒的完整日期时间，可以使用 DATETIME 类型，格式为“YYYY-MM-DD HH:MM:SS”，例如“2022-02-04 10:08:08”。

如果需要经常插入或更新日期时间为系统日期时间，则通常使用 TIMESTAMP 类型，格式为“YYYY-MM-DD HH:MM:SS”，例如“2022-02-04 10:08:08”。TIMESTAMP 与 DATETIME 的区别在于 TIMESTAMP 的取值范围小，只支持 1970-01-01 00:00:01 UTC 至 2038-01-19 03:14:07 UTC 范围的日期时间值，其中 UTC 是世界标准时间，并且 TIMESTAMP 类型的日期时间值在存储时会当前时区的日期时间值转换为时间标准时间值，检索时再转换回当前时区的日期时间值，这会更方便。而 DATETIME 则只能反映出插入时当地的时区，其他时区的人查看数据必然会有误差的。另外，TIMESTAMP 的属性受 MySQL 版本和服务器的 SQLMode 的影响很大。

```
drop table if exists t_date;
create table t_date (
    d1 datetime,
    d2 timestamp
);
insert into t_date values('2021-9-2 14:45:52','2021-9-2 14:45:52');
select * from t_date;
-- 修改当前的时区
set time_zone = '+2:00';
insert into t_date values('202192144552','202192144552'); -- 格式错误
insert into t_date values('20210902144552','20210902144552');
insert into t_date values('2021&9&2 14%45%52','2021#9#2 14@45@52');
select * from t_date;
drop table if exists t_date;
create table t_date (
    d year
);
insert into t_date values(2021),(85),(22),(69),(0),('0');
select * from t_date;
```

8.4 其他类型

8.4.1 JSON 类型

在 MySQL5.7 之前,如果需要在数据库中存储 JSON 数据只能使用 VARCHAR 或 TEXT 字符串类型。从 5.7.8 版本之后开始支持 JSON 数据类型。

8.4.2 空间类型

MySQL 空间类型扩展支持地理特征的生成、存储和分析。这里的地理特征表示世界上具有位置的任何东西,可以是一个实体,例如一座山;可以是空间,例如一座办公楼;也可以是一个可定义的位置,例如一个十字路口等等。现在的应用程序开发中空间数据的存储越来越多了,例如,钉钉的打卡位置是否在办公区域范围内,滴滴打车的位置、路线等。MySQL 提供了非常丰富的空间函数以支持各种空间数据的查询和处理。

MySQL 中使用 Geometry (几何)来表示所有地理特征。Geometry 指一个点或点的集合,代表世界上任何具有位置的事物。MySQL 的空间数据类型 (Spatial Data Type) 对应于 OpenGIS 类,包括 GEOMETRY、POINT、LINESTRING、POLYGON 等单值类型以及 MULTIPOINT、MULTILINESTRING、MULTIPOLYGON、GEOMETRYCOLLECTION 存放不同几何值的集合类型。

第 9 章 系统预定义函数

函数：代表一个独立的可复用的功能。

MySQL 中的函数必须有返回值，参数可以有可以没有。

MySQL 中函数分为：

（1）系统预定义函数：MySQL 数据库管理软件提供的函数，直接用就可以，任何数据库都可以用公共的函数。

➤ 单行函数：表示会对表中的每一行记录分别计算，有 n 行得到还是 n 行结果。如数学函数、字符串函数、日期时间函数、条件判断函数、窗口函数等。

➤ 分组函数：或者又称为聚合函数，多行函数，表示会对表中的多行记录一起做一个“运算”，得到一个结果。如求平均值的 `avg`，求最大值的 `max`，求最小值的 `min`，求总和 `sum`，求个数的 `count` 等。

（2）用户自定义函数：由开发人员自己定义的，通过 `CREATE FUNCTION` 语句定义，是属于某个数据库的对象。

9.1 单行函数

9.1.1 常用数学函数

函数	说明
<code>abs(x)</code>	绝对值
<code>ceil(x)</code>	向上取整
<code>floor(x)</code>	向下取整
<code>mod(x,y)</code>	x 模 y
<code>rand()</code>	返回 0~1 的随机值
<code>round(x,y)</code>	返回参数 x 的四舍五入的有 y 位的小数的值
<code>truncate(x,y)</code>	返回数字 x 截断为 y 位小数的结果
<code>format(x,y)</code>	强制保留小数点后 y 位，整数部分超过三位的时候以逗号分割，并且返回的结果是文本类型
<code>sqrt(x)</code>	x 的平方根
<code>pow(x,y)</code>	x 的 y 次方

示例：

```

use atguigu;
-- 在 t_employee 表中查询员工无故旷工一天扣多少钱
-- 分别使用 ceil,floor,round,truncate 函数
-- 假设本月工作日总天数是 22 天
-- 旷工一天扣的钱=salary/22
select
    ename,
    salary/22,
    ceil(salary/22),
    floor(salary/22),
    round(salary/22,2),
    truncate(salary/22,2)
from t_employee;
-- 查询公司平均薪资，并对平均薪资分别
-- 分别使用 ceil,floor,round,truncate 函数
select
    avg(salary),
    ceil(avg(salary)),
    floor(avg(salary)),
    round(avg(salary),2),
    truncate(avg(salary),2)
from t_employee;

```

9.1.2 常用字符串函数

函数	说明
concat(s1,s2,...)	拼接字符串
concat(a,s1,s2,...)	在字符串间加上 a 拼接字符串
char_length(s)	s 的字符数
length(s)	s 的字节数，与字符集有关
locate(s,str) 或 instr(str,s)	返回 s 在 str 中的开始位置
upper(s) 或 ucase(s)	所有字母转大写
lower(s) 或 lcase(s)	所有字母转小写
left(s,n)	返回最左边的 n 个字符
right(s,n)	返回最右边的 n 个字符
lpad(str,len,pad)	用 pad 从左边填充 str 直到长度达到 len
rpadd(str,len,pad)	用 pad 从右边填充 str 直到长度达到 len
ltrim(s)	去掉 s 左侧空格

rtrim(s)	去掉 s 右侧空格
trim(s)	去掉 s 两侧空格
trim([both] s from str)	去掉 str 两侧的 s
trim([leading] s from str)	去掉 str 左侧的 s
trim([trailing] s from str)	去掉 str 右侧的 s
insert(str,index,len,instr)	str 从 index 位置开始的 len 个字符替换为 instr
replace(str,a,b)	str 中的 a 全部替换为 b
repeat(s,n)	返回 s 重复 n 次的结果
reverse(s)	反转字符串
strcmp(s1,s2)	比较 s1,s2
substring(str,index,len)	str 从 index 位置截取 len 个字符
substring_index(str,分隔符,count)	如果 count 是正数，那么从左往右数，截取第 n 个分隔符的左边的全部内容。例如， substring_index("www.atguigu.com",".",1)是"www"。 如果 count 是负数，那么从右边开始数，截取第 n 个分隔符右边的所有内容。例如， substring_index("www.atguigu.com",".",-1)是"com"。

示例：

```

use atguigu;
-- 在 t_employee 表中查询员工姓名 ename 和电话 tel
-- 并使用 concat 函数，concat_ws 函数
select concat(ename,tel),concat_ws('-',ename,tel) from t_employee;
-- 在 t_employee 表中查询薪资高于 15000 的男员工姓名
-- 并把姓名处理成 张 xx 的样式
-- left(s,n)函数表示取字符串 s 最左边的 n 个字符
-- 而 rpad(str,len,pad)函数表示在字符串 str 的右边填充 pad 使得字符串长度达到 len
select rpad(left(ename,1),3,'x'),salary
from t_employee
where salary>15000 and gender='男';
-- 在 t_employee 表中查询薪资高于 10000 的男员工姓名，姓名包含的字符数和占用的字节数
select ename,char_length(ename) as 占用字符数,length(ename) as 占用字节数量
from t_employee
where salary>10000 and gender='男';

```

```
-- 在 t_employee 表中查询薪资高于 10000 的男员工姓名和邮箱 email
-- 并把邮箱名 @ 字符之前的字符串截取出来
-- MySQL 中 substring 函数截取字符串，位置从 1 开始
select ename,email,substring(email,1,position('@' in email)-1)
from t_employee
where salary>10000 and gender='男';
-- trim()默认是去掉前后空白符
select trim('    hello    world    ');
select concat('[',trim('    hello    world    '),']');
-- 去掉前后的 &
select trim(both '&' from '&&&&hello    world&&&&');
select trim(leading '&' from '&&&&hello    world&&&&');
select trim(trailing '&' from '&&&&hello    world&&&&');
```

9.1.3 日期时间函数

函数	说明
curdate() 或 current_date()	当前系统日期
curtime() 或 current_time()	当前系统时间
now() 或 sysdate() 或 current_timestamp() 或 localtime() 或 localtimestamp()	当前系统日期时间
utc_date() 或 utc_time()	当前 UTC 日期值/时间值
unix_timestamp(date)	UNIX 时间戳
year(date)/month(date)/ day(date)/ hour(time)/ minute(time)/ second(time)	取年/月/日/小时/分钟/秒
extract(type from date)	从日期中提取一部分值
dayofmonth(date)	一月中第几天
dayofyear(date)	一年中第几天
week(date) 或 weekofyear(date)	一年中的第几周
dayofweek(date)	返回周几，周日是 1，周一是 2，…周六是 7
weekday(date)	返回周几，周一是 0，周二是 1，…周日是 6
dayname(date)	返回星期，Monday,Tuesday,….Sunday

monthname(date)	返回月份，January,...
datediff(date1,date2)	date1-date2 的日期间隔
timediff(time1,time2)	time1-time2 的时间间隔
date_add(date,interval expr type) 或 adddate/date_sub/subdate	返回与给定日期相差 interval 时间段的日期
addtime(time,expr)/subtime(time,expr)	返回给定时间加上/减去 expr 的时间值
date_format(datetime,fmt)	按照字符串 fmt 格式化日期 datetime 值
time_format(time,fmt)	按照字符串 fmt 格式化时间 time 值
str_to_date(str,fmt)	按照字符串 fmt 将 str 解析为一个日期
get_format(val_type,format_type)	返回日期时间字符串的显示格式

函数中日期时间类型说明：

参数类型	说明	参数类型	说明
YEAR	年	YEAR_MONTH	年月
MONTH	月	DAY_HOUR	日时
DAY	日	DAY_MINUTE	日时分
HOUR	小时	DAY_SECOND	日时分秒
MINUTE	分钟	HOUR_MINUTE	时分
SECOND	秒	HOUR_SECOND	时分秒
WEEK	星期	MINUTE_SECOND	分秒
QUARTER	一刻		

函数中 format 参数说明：

格式符	说明	格式符	说明
%Y	4 位数字表示年份	%y	2 位数字表示年份
%M	月名表示月份（January,...）	%m	2 位数字表示月份（01,02,03, ...）
%b	缩写的月名（Jan.,Feb.,...）	%c	数字表示月份（1,2,3...）
%D	英文后缀表示月中的天数 （1st,2nd,3rd,...）	%d	2 位数字表示表示月中的天数 （01,02,...）
%e	数字形式表示月中的天数 （1,2,3,...）	%p	AM 或 PM

%H	2 位数字表示小时，24 小时制 (01,02,03,...)	%h 和 %I	2 位数字表示小时，12 小时制 (01,02,03,...)
%k	数字形式的小时，24 小时制 (1,2,3,...)	%l	数字表示小时，12 小时制 (1,2,3,...)
%i	2 位数字表示分钟 (00,01,02,...)	%S 和 %s	2 位数字表示秒 (00,01,02,...)
%T	时间，24 小时制 (hh:mm:ss)	%r	时间，12 小时制 (hh:mm:ss) 后 加 AM 或 PM
%W	一周中的星期名称 (Sunday,...)	%a	一周中的星期缩写 (Sun.,Mon.,Tues.,...)
%w	以数字表示周中的天数 (0=Sunday,1=Monday,...)	%j	以 3 位数字表示年中的天数 (001,002,...)
%U	以数字表示的第几周 (1,2,3,...) 其中 Sunday 为周 中的第 1 天	%u	以数字表示年中的年份 (1,2,3,...) 其中 Monday 为周 中第 1 天
%V	一年中第几周 (01~53)，周日 为每周的第 1 天，和%X 同时使 用	%X	4 位数形式表示该周的年份，周 日为每周第 1 天，和%V 同时使 用
%v	一年中第几周 (01~53)，周一 为每周的第 1 天，和%x 同时使 用	%x	4 位数形式表示该周的年份，周 一为每周第 1 天，和%v 同时使 用
%%	表示%		

GET_FORMAT 函数中 val_type 和 format_type 参数说明:

值类型	格式化类型	显示格式字符串
DATE	EUR	%d.%m.%Y
DATE	INTERVAL	%Y%m%d
DATE	ISO	%Y-%m-%d
DATE	JIS	%Y-%m-%d
DATE	USA	%m.%d.%Y

TIME	EUR	%H.%i.%s
TIME	INTERVAL	%H%i%s
TIME	ISO	%H:%i:%s
TIME	JIS	%H:%i:%s
TIME	USA	%h:%i:%s %p
DATETIME	EUR	%Y-%m-%d %H.%i.%s
DATETIME	INTERVAL	%Y%m%d %H%i%s
DATETIME	ISO	%Y-%m-%d %H:%i:%s
DATETIME	JIS	%Y-%m-%d %H:%i:%s
DATETIME	USA	%Y-%m-%d %H.%i.%s

示例：

```

use atguigu;
-- 获取系统日期。curdate()和 current_date()函数都可以获取当前系统日期
select curdate(),current_date();
-- 将日期值 +0
select curdate()+0;
-- 获取系统时间。curtime()和 current_time()函数都可以获取当前系统时间
select curtime(),current_time();
-- 将时间值 +0
select curtime()+0;
-- 获取系统日期时间值。current_timestamp(),localtime(),sysdate(),now()
select current_timestamp(),localtime(),sysdate(),now();
-- 获取当前 UTC（世界标准时间）日期或时间值
-- 本地时间是根据地球上不同时区所处的位置调整 UTC 得来的
-- 例如，北京时间比 UTC 时间晚 8 个小时
-- utc_date(),curdate(),utc_time(),curtime()
select utc_date(),curdate(),utc_time(),curtime();
-- 获取 UNIX 时间戳
select unix_timestamp(),unix_timestamp("2000-1-1");
-- 获取具体的时间值，比如年、月、日、时、分、秒
-- year(date),month(date),day(date)
-- hour(time),minute(time),second(time)
select year(now()),month(now()),day(now()),
       hour(now()),minute(now()),second(now());
-- 获取日期时间的指定值。extract(type from date/time)函数
select extract(year_month from now());
-- 获取两个日期或时间之间的间隔
-- datediff(date1,date2) 返回两个日期之间间隔的天数

```

```
-- timediff(time1,time2) 返回两个时间之间间隔的时分秒
select datediff(now(),"2000-1-1");
select timediff(now(),"2000-1-1 12:00:00");
-- 查询今天距离员工入职的日期间隔天数
select ename,datediff(curdate(),hiredate) from t_employee;
-- 查询现在距离中午放学还有多少时间
select timediff("12:00:00",curtime());
-- 在 t_employee 表中查询本月生日的员工姓名、生日
select ename,birthday
from t_employee
where month(curdate()) = month(birthday);
-- 查询入职时间超过 5 年的
select ename,hiredate,datediff(curdate(),hiredate)
from t_employee
where datediff(curdate(),hiredate)>365*5;
```

9.1.4 常用加密函数

函数	说明
password(str)	返回字符串 str 的加密版本，41 位长的字符串（MySQL8 不再支持）
md5(str)	返回字符串 str 的 md5 值，也是一种加密方式
sha(str)	返回字符串 str 的 sha 算法加密字符串，40 位十六进制值的密码字符串
sha2(str,hash_length)	返回字符串 str 的 sha 算法加密字符串，密码字符串的长度是 hash_length/4。hash_length 可以是 224、256、384、512、0，其中 0 等同于 256

示例：

```
use atguigu;
-- 当用户需要对数据进行加密时
-- 比如做登录功能时，给用户的密码加密等
select md5('123456'),sha('123456'),sha2('123456',0);
select
    char_length(md5('123456')),
    char_length(sha('123456')),
    char_length(sha2('123456',0));
drop table if exists t_user;
create table t_user(
    id int primary key auto_increment,
    username varchar(20),
```

```
password varchar(100)
);
insert into t_user values(null,"chai",md5("123456"));
select * from t_user where username="chai" and password="123456";
select * from t_user where username="chai" and password=md5("123456");
drop table if exists t_user;
```

9.1.5 常用系统信息函数

函数	说明
database()	当前数据库名
version()	当前数据库版本
user()	当前登录用户名

9.1.6 条件判断函数

函数	说明
if(a,x,y)	如果 a 为真，返回 x，否则返回 y
ifnull(x,y)	如果 x 不为空，返回 x，否则返回 y
case when 条件 1 then result1 when 条件 2 then result2 else resultn end	依次判断条件，哪个条件满足了，就返回对应的 result,所有条件都不满足就返回 else 的 result。如果没有单独的 else 子句，当所有 when 后面的条件都不满足时则返回 NULL
case 表达式 when 常量值 1 then 值 1 when 常量值 2 then 值 2 else 值 n end	判断表达式与哪个常量值匹配，找到匹配的就返回对应值，都不匹配就返回 else 的值。如果没有单独的 else 子句，当所有 when 后面的常量值都不匹配时则返回 NULL

示例：

```
use atguigu;
-- 条件判断函数不是筛选记录的函数
-- 而是根据条件不同显示不同的结果的函数
-- 如果薪资大于 20000，显示高薪，否则显示正常
select ename,salary,if(salary>20000,'高薪','正常')
from t_employee;
-- 计算实发工资。实发工资 = 薪资 + 薪资 * 奖金比例
```

```
select
    ename,
    salary,
    commission_pct,
    salary + salary * commission_pct as 实发工资
from t_employee;
-- 如果 commission_pct 是, 计算完结果是 NULL
select
    ename,
    salary,
    commission_pct,
    salary + salary * ifnull(commission_pct,0) as 实发工资
from t_employee;
-- 查询员工编号, 姓名, 薪资, 等级, 等级根据薪资判断
-- 如果薪资大于 20000, 显示 羡慕级别
-- 如果薪资 15000-20000, 显示 努力级别
-- 如果薪资 10000-15000, 显示 平均级别
-- 如果薪资 10000 以下, 显示 保底级别
select eid,ename,salary,
case
    when salary>20000 then '羡慕级别'
    when salary>15000 then '努力级别'
    when salary>10000 then '平均级别'
    else '保底级别'
end as "等级"
from t_employee;
-- 在 t_employee 表中查询入职 7 年以上的员工姓名、工作地点、轮岗的工作地点数量情况
-- 计算工作地点的数量可以转换为求 work_place 中逗号的数量+1
-- work_place 中逗号的数量 = work_place 的总字符数 - work_place 去掉逗号的字符数
-- 使用 replace 函数去掉 work_place 中逗号
select work_place,
char_length(work_place)-char_length(replace(work_place,',',''))+1 as 工作地点数量
from t_employee;

select ename,work_place,
case char_length(work_place)-char_length(replace(work_place,',',''))+1
    when 1 then '只在一个地方工作'
    when 2 then '在两个地方来回奔波'
    when 3 then '在三个地方流动'
    else '频繁出差'
end as "工作地点数量情况"
```

```
from t_employee
where datediff(curdate(),hiredate)>365*7;
```

9.1.7 其他函数

从 5.7.8 版本之后开始支持 JSON 数据类型，并提供了操作 JSON 类型数据的相关函数。

MySQL 提供了非常丰富的空间函数以支持各种空间数据的查询和处理。

这两类函数暂时不讲，如果项目中有用到查询 API 使用。

9.2 分组函数

分组函数有合并计算过程。调用完分组函数后，结果的行数变少，可能得到一行，可能得到少数几行。

常用的分组函数：

函数	说明
avg(x)	平均值
sum(x)	求和
max(x)	最大值
min(x)	最小值
count(x)	计数

示例：

```
use atguigu;
-- 统计 t_employee 表的员工的数量
select count(*) from t_employee;
select count(1) from t_employee;
select count(eid) from t_employee;
select count(commission_pct) from t_employee;
/*
count(*)或 count(常量值)：都是统计实际的行数
count(字段/表达式)：统计时忽略 NULL 值
*/
-- 找出 t_employee 表中最高的薪资值
select max(salary) from t_employee;
-- 找出 t_employee 表中最低的薪资值
select min(salary) from t_employee;
-- 统计 t_employee 表中平均薪资值
select avg(salary) from t_employee;
-- 统计所有人的薪资总和
select sum(salary) from t_employee;
```



```
select sum(salary+salary*ifnull(commission_pct,0)) from t_employee;
-- 找出年龄最小、最大的员工的出生日期
select min(birthday),max(birthday) from t_employee;
-- 查询最新入职的员工的入职日期
select max(hiredate) from t_employee;
```

分组函数一般和 group by 子句结合在一起使用，例如

```
-- 查询每一个部门的平均薪资
select did,round(avg(salary),2)
from t_employee
group by did;
```

9.3 窗口函数

窗口函数也叫 OLAP 函数（Online Analytical Processing，联机分析处理），可以对数据进行实时分析处理。窗口函数是每条记录都会分析，有几条记录执行完还是几条，因此也属于单行函数。

窗口函数	说明
row_number()	顺序排序，每行按照不同的分组逐行编号，例如：1,2,3,4
rank()	并列排序，每行按照不同的分组进行编号，同一个分组中排序字段值出现重复值时，并列排序并跳过重复序号，例如：1,1,3
dense_rank()	并列稠密排序，每行按照不同的分组进行编号，同一个分组中排序字段值出现重复值时，并列排序不跳过重复序号，例如：1,1,2
lag()/lead()	访问窗口中当前行前/后一定偏移量的值
first_value()/last_value()	访问窗口中第一个或最后一个值
sum()/avg()/count()/max()/min()	求和/平均值/计数/最大值/最小值

窗口函数的语法格式如下：

```
函数名(参数列表) over(
    [partition by column]
    [order by column]
    [rows between <start> and <end>]
)
```

over 关键字用来指定窗口函数的窗口范围。如果 over 后面是空（），则表示 select 语句筛选的所有行是一个窗口。over 后面的（）支持以下语法来设置窗口范围：

➤ window：给窗口指定一个别名

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

➤ **partition by:** 一个窗口范围还可以分为多个区域。按照哪些字段进行分区/分组，窗口函数在不同的分组上分别处理分析

➤ **order by:** 按照哪些字段进行排序，窗口函数将按照排序后结果进行分析处理

➤ **rows/range between <start> and <end>:** 在计算窗口函数时，指定哪些行/值将被包含在计算范围内，<start>和<end>用于定义窗口范围：

➤ **unbounded preceding:** 窗口从分区的第一行开始

➤ **n preceding:** 当前行之前的 n 行

➤ **current row:** 当前行

➤ **n following:** 当前行之后的 n 行

➤ **unbounded following:** 窗口到分区的最后一行

示例：

```
use atguigu;
-- 在 t_employee 表中查询薪资在[8000,10000]之间的员工姓名和薪资并给每一行记录
编号
select row_number() over() as rn,ename,salary
from t_employee
where salary between 8000 and 10000;
-- 计算每一个部门的平均薪资与全公司的平均薪资的差值
select distinct
    did,
    avg(salary) over(partition by did),
    avg(salary) over(),
    round(avg(salary) over(partition by did)-avg(salary) over(),2) as
deviation
from t_employee;
-- 在 t_employee 表中查询女员工姓名，部门编号，薪资
-- 查询结果按照部门编号分组后在按薪资升序排列
-- 并分别使用 row_number()、rank()、dense_rank()三个序号函数给每一行记录编
号
select ename,did,salary,gender,
row_number() over(partition by did order by salary) as "row_num",
rank() over(partition by did order by salary) as "rank_num",
dense_rank() over(partition by did order by salary) as "ds_rank_num"
from t_employee where gender='女';
-- 或使用 window 给窗口指定别名
select ename,did,salary,
row_number() over w as "row_num",
rank() over w as "rank_num" ,
dense_rank() over w as "ds_rank_num"
```

```
from t_employee where gender='女'
window w as(partition by did order by salary);
-- 在 t_employee 表中查询每个部门最低 3 个薪资值的女员工姓名，部门编号，薪资值
select row_number() over() as rn,temp.*
from(select ename,did,salary,
row_number() over w as "row_num",
rank() over w as "rank_num" ,
dense_rank() over w as "ds_rank_num"
from t_employee where gender='女'
window w as(partition by did order by salary))temp
where temp.rank_num<=3;
-- 在 t_employee 表中查询每个部门薪资排名前 3 的员工姓名，部门编号，薪资值
select temp.*
from(select ename,did,salary,
dense_rank() over w as "ds_rank_num"
from t_employee
window w as(partition by did order by salary desc))temp
where temp.ds_rank_num<=3;
-- 在 t_employee 表中查询全公司薪资排名前 3 的员工姓名，部门编号，薪资值
select temp.*
from(select ename,did,salary,
dense_rank() over w as "ds_rank_num"
from t_employee
window w as(order by salary desc))temp
where temp.ds_rank_num<=3;
-- 查找薪资排名的上一位、下一位、首位、末位
select
    ename,
    salary,
    lag(ename,1,'-') over(order by salary) as '上一位姓名',
    lag(salary,1,0) over(order by salary) as '上一位薪资',
    lead(ename) over(order by salary) as '下一位姓名',
    lead(salary) over(order by salary) as '下一位薪资',
    first_value(salary) over(order by salary) as '首位薪资',
    last_value(ename) over(order by salary rows between unbounded preceding
and unbounded following) as '末位姓名'
from t_employee;
```

第 10 章 关联查询

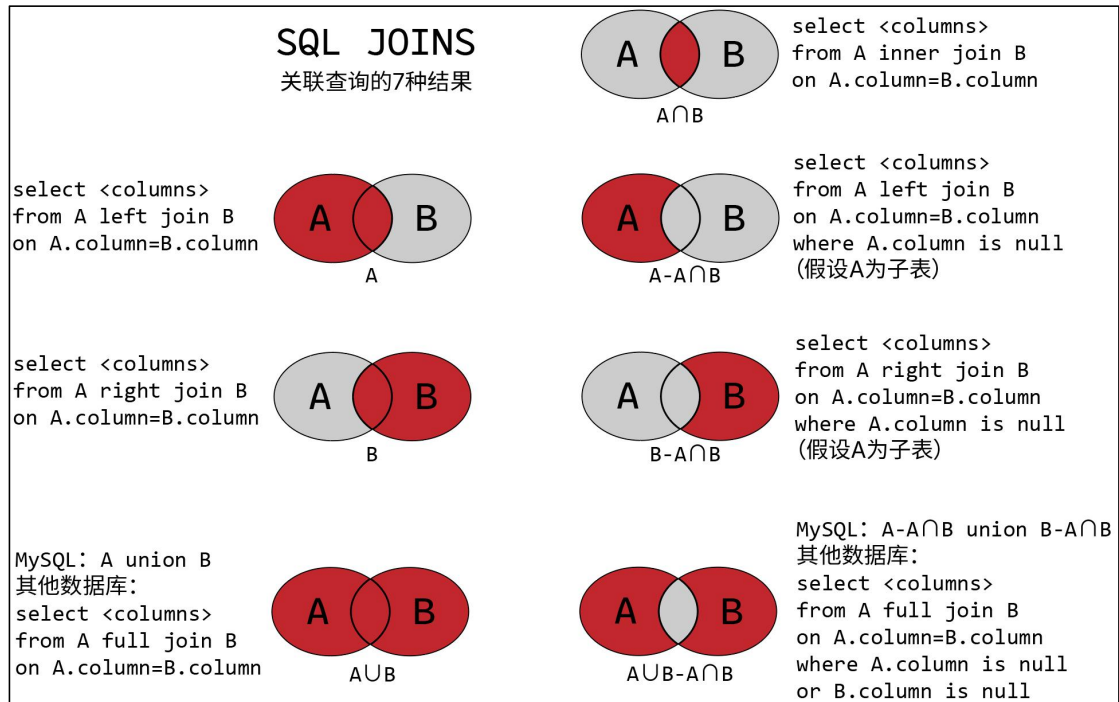
10.1 什么是关联查询

关联查询：两个或更多个表一起查询。

前提条件：这些一起查询的表之间是有关系的（一对一、一对多），它们之间一定是有
关联字段，这个关联字段可能建立了外键，也可能没有建立外键。

比如：员工表和部门表，这两个表依靠“部门编号”进行关联。

10.2 关联查询的几种情况



（1）凡是联合查询的两个表，必须有“关联字段”

关联字段是逻辑意义一样，数据类型一样，名字可以一样也可以不一样的两个字段。比
如：t_employee（A表）中 did 和 t_department（B表）中的 did。

关联字段其实就是“可以”建外键的字段。当然联合查询不要求一定建外键。

（2）关联查询必须写关联条件，关联条件的个数 = $n - 1$ ， n 是联合查询的表的数量

2 个表一起联合查询，关联条件数量是 1，

3 个表一起联合查询，关联条件数量是 2，

4 个表一起联合查询，关联条件数量是 3，

否则就会出现笛卡尔积现象。

（3）关联条件可以用 on 子句编写，也可以写到 where 中

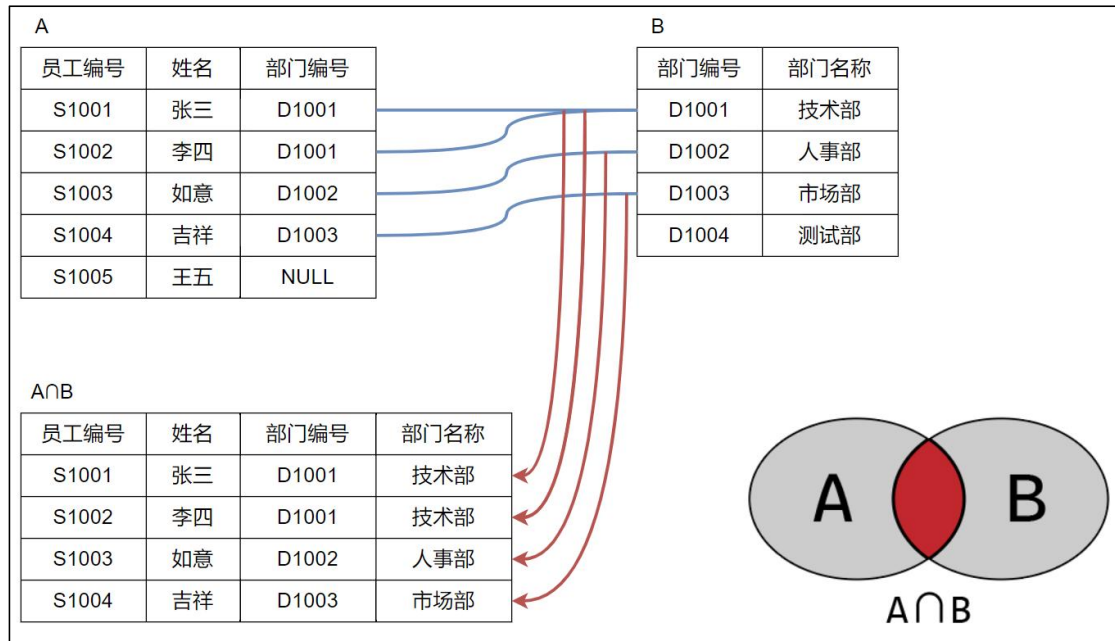
但是建议用 on 单独编写，这样可读性更好。

每一个 join 后面都要加 on 子句。

A inner|left|right join B on 关联条件

A inner|left|right join B on 关联条件 inner|left|right join C on 关联条件

10.3 内连接



```
use atguigu;
-- t_employee 看成 A 表
-- t_department 看成 B 表
-- 此时 t_employee (A 表) 中的李红和周洲的 did 是 NULL, 没有对应部门
-- t_department (B 表) 中的测试部, 没有对应员工

-- 内连接结果:
-- AnB

-- 查询所有员工的姓名, 部门编号, 部门名称
-- 排除没有部门的员工
-- 排除没有员工的部门
-- 员工的姓名在 t_employee (A 表)
-- 部门的编号在 t_employee (A 表) 和 t_department (B 表) 都有
-- 部门名称在 t_department (B 表)
-- 所以需要联合两个表一起查询
select ename,did,dname
from t_employee inner join t_department;
-- 上述 sql 报错
-- did 在两个表中都有, 名字相同, 不知道取哪个表中字段
-- 有可能存在两个表都有 did, 但是 did 的意义不同的情况
-- 为了避免这种情况, 需要在编写 sql 的时候, 明确指出是用哪个表的 did
select ename,t_department.did,dname
from t_employee inner join t_department;
-- 上述 sql 语法对, 结果不对
-- 出现 笛卡尔积 现象, A 表记录*B 表记录
```

```

select ename,t_department.did,dname
from t_employee inner join t_department
on t_employee.did = t_department.did;

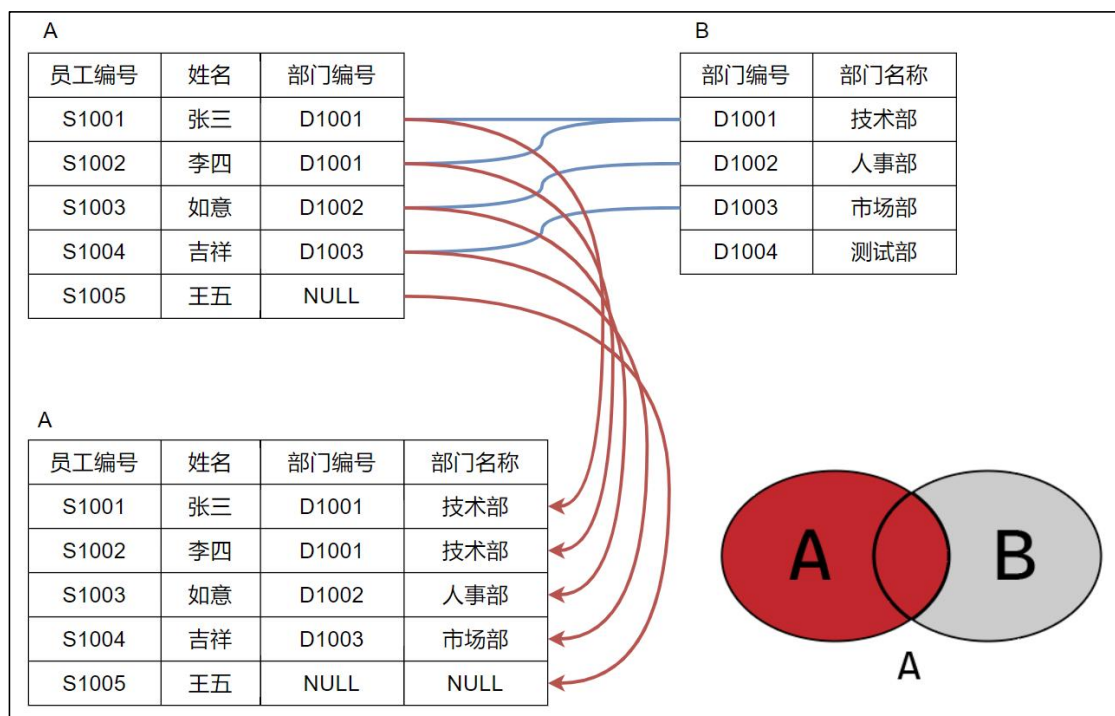
select *
from t_employee inner join t_department
on t_employee.did = t_department.did;

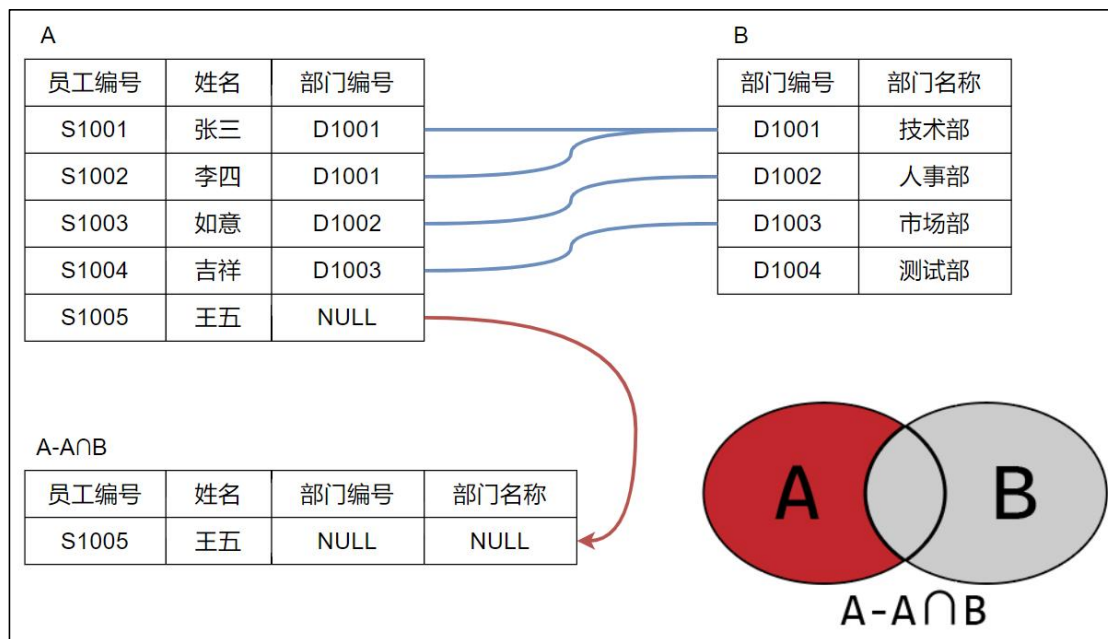
-- 查询部门编号为 1 的女员工的姓名、部门编号、部门名称、薪资等情况
select ename,gender,t_department.did,dname,salary
from t_employee inner join t_department
on t_employee.did = t_department.did
where t_department.did = 1 and gender = '女';

-- 查询部门编号为 1 的员工姓名、部门编号、部门名称、薪资、职位编号、职位名称等情况
select ename,gender,t_department.did,dname,salary,job_id,jname
from t_employee
inner join t_department on t_employee.did = t_department.did
inner join t_job on t_employee.job_id = t_job.jid
where t_department.did = 1;

```

10.4 左连接





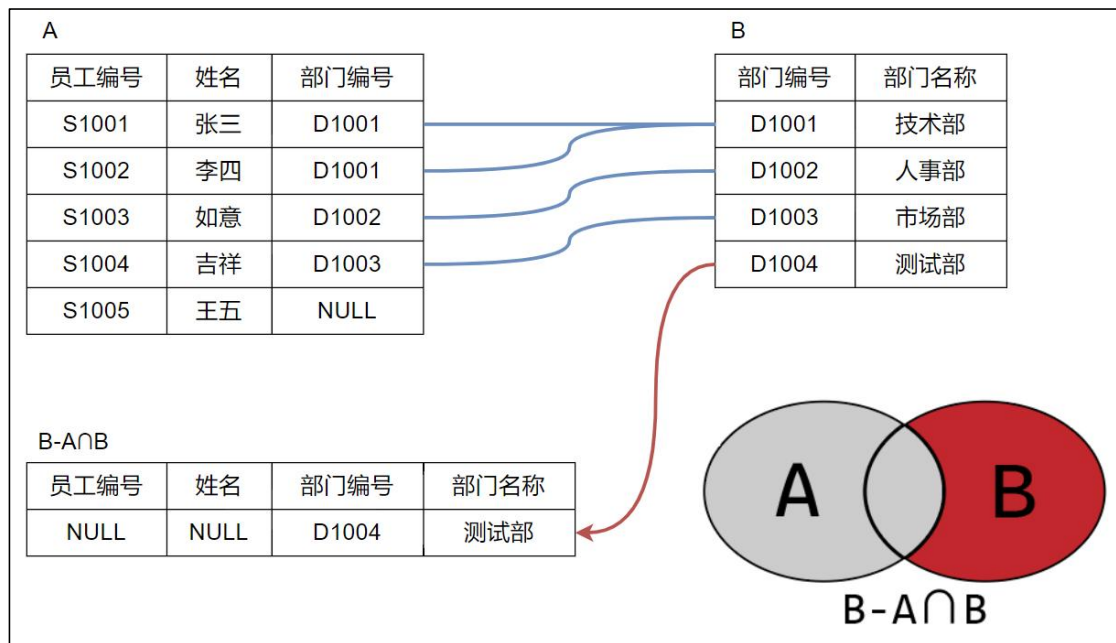
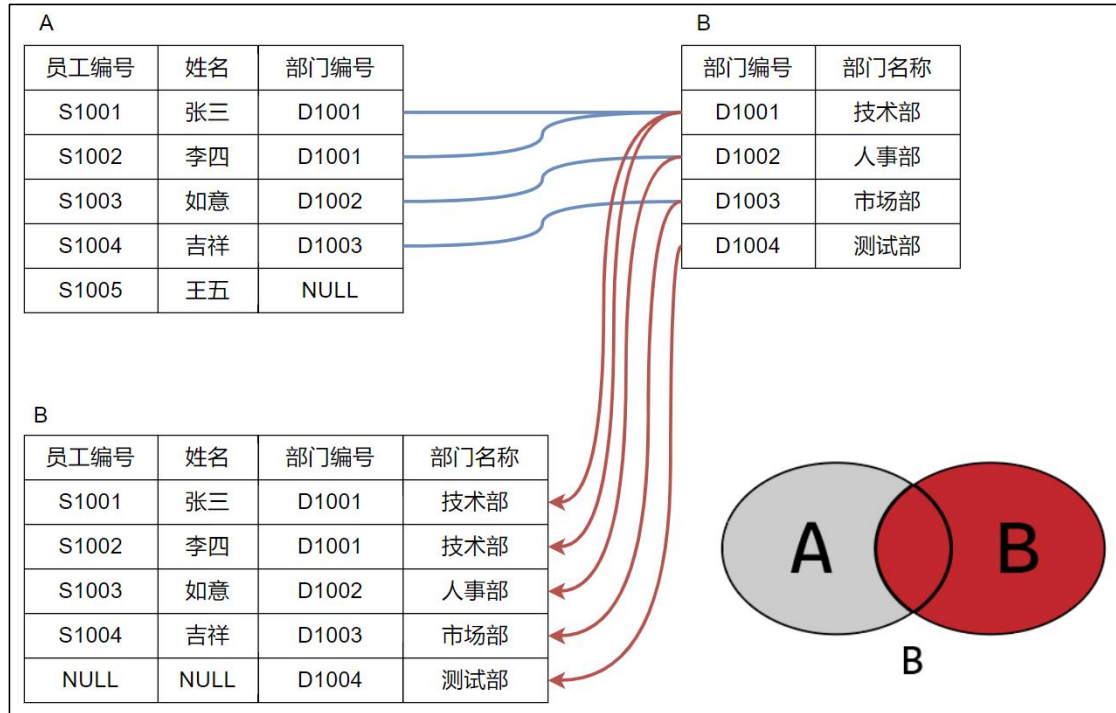
```
use atguigu;
-- t_employee 看成 A 表
-- t_department 看成 B 表
-- 此时 t_employee (A 表) 中的李红和周洲的 did 是 NULL, 没有对应部门
-- t_department (B 表) 中的测试部, 没有对应员工

-- 左连接结果:
-- A
-- A-A∩B

-- 查询所有员工, 包括没有指定部门的员工, 他们的姓名、薪资、部门编号、部门名称
select ename,salary,t_department.did,dname
from t_employee left join t_department
on t_employee.did = t_department.did;
-- 查询的结果是 A

-- 查询没有部门的员工信息
select ename,salary,t_department.did,dname
from t_employee left join t_department
on t_employee.did = t_department.did
where t_employee.did is null;
-- 查询的结果是 A-A∩B
-- 此时的 where 条件, 建议写子表的关联字段 is null, 这样更准确一点
-- 如果要建外键, 它们之间有子表和父表的角色, 写子表的关联字段 is null
-- 因为父表中这个字段一般是主键, 不会为 null
```

10.5 右连接



```
use atguigu;
-- t_employee 看成 A 表
-- t_department 看成 B 表
-- 此时 t_employee (A 表) 中的李红和周洲的 did 是 NULL，没有对应部门
-- t_department (B 表) 中的测试部，没有对应员工

-- 右连接结果：
-- B
-- B-A∩B
```

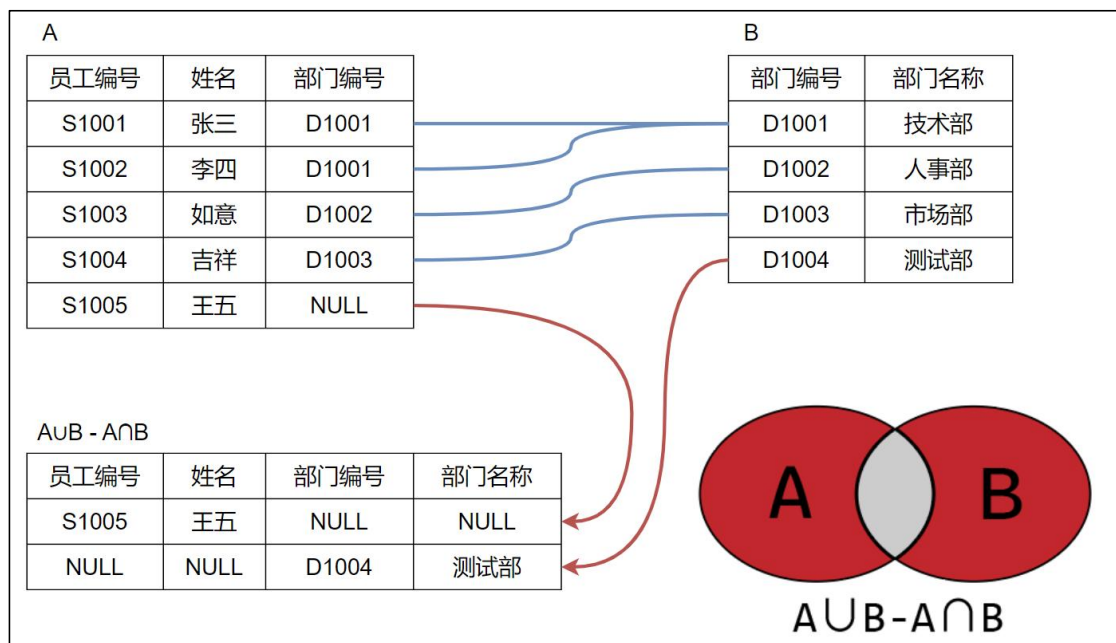
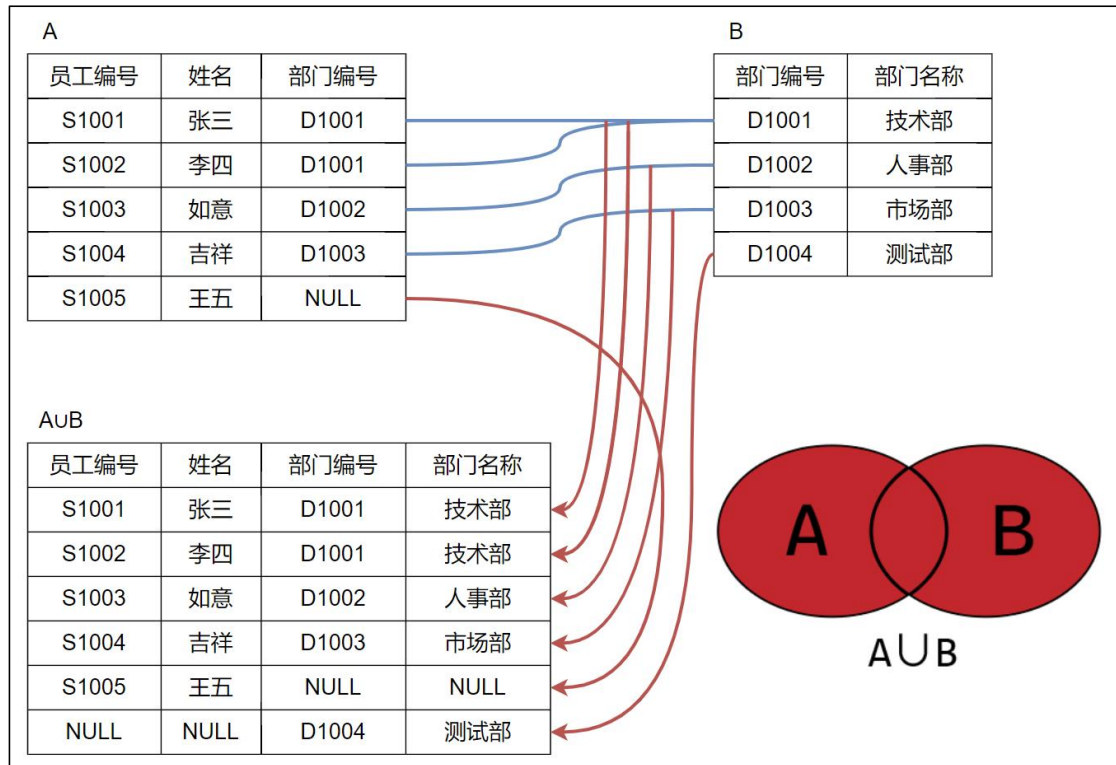
```
-- 查询所有部门，包括没有对应员工的部门，他们的姓名、薪资、部门编号、部门名称
select ename,salary,t_department.did,dname
from t_employee right join t_department
on t_employee.did = t_department.did;
-- 查询的结果是 B

-- 查询没有员工部门的信息
select ename,salary,t_department.did,dname
from t_employee right join t_department
on t_employee.did = t_department.did
where t_employee.did is null;
-- 查询的结果是 B-AnB

-- 查询所有员工，包括没有指定部门的员工，他们的姓名、薪资、部门编号、部门名称
select ename,salary,t_department.did,dname
from t_department right join t_employee
on t_employee.did = t_department.did;
-- 查询的结果是 B

--查询没有部门的员工信息
select ename,salary,t_department.did,dname
from t_department right join t_employee
on t_employee.did = t_department.did
where t_employee.did is null;
-- 查询的结果是 B-AnB
```

10.6 union



```
use atguigu;
-- t_employee 看成 A 表
-- t_department 看成 B 表
-- 此时 t_employee (A 表) 中的李红和周洲的 did 是 NULL，没有对应部门
-- t_department (B 表) 中的测试部，没有对应员工

-- union 结果:
-- A∪B
-- A∪B - A∩B = A - A∩B ∪ B - A∩B
```

```
-- union 合并时要注意：
-- 两个表要查询的结果字段是一样的
-- union all 表示直接合并结果，保留重复的记录
-- union 表示合并结果时，去重
-- 要实现 AUB 的结果，那么必须是合并查询是 A 表结果和查询是 B 表结果的 select 语句
-- 要实现 AUB-AnB 的结果，那么必须是合并查询是 A-AnB 结果和查询是 B-AnB 的 select 语句

-- 查询所有员工和所有部门，包括没有指定部门的员工和没有分配员工的部门
select *
from t_employee left join t_department
on t_employee.did=t_department.did
union
select *
from t_employee right join t_department
on t_employee.did=t_department.did;
-- 以下 union 会报错，两个 select 语句的列数是不同的
select * from t_employee
union
select * from t_department;

-- 查询那些没有分配部门的员工和没有指定员工的部门，即 A 表和 B 表在对方那里找不到
对应记录的数据
select *
from t_employee left join t_department
on t_employee.did = t_department.did
where t_employee.did is null
union
select *
from t_employee RIGHT join t_department
on t_employee.did = t_department.did
where t_employee.did is null;
```

10.7 自连接

```
use atguigu;
-- 物理上是一张表；逻辑上是两张表
-- t_employee 表中 mid 表示员工的领导的编号，即该员工领导的员工编号
-- 例如 eid 为 3 的员工邓超远，他的 mid 是 7，表示他的领导是员工编号为 7 的员工
-- mid 的取值范围受到 eid 字段的限制。mid 的值选择必须是 eid 现有值范围

-- 查询每一个员工的编号、名字、薪资和他领导的编号、姓名、薪资
select
    emp.eid,emp.ename,emp.salary,
    mgr.eid,mgr.ename,mgr.salary
```

```
from t_employee as emp inner join t_employee as mgr
on emp.mid = mgr.eid;
-- 把 t_employee 当成两张表，通过取别名的方式
-- t_employee as emp 把员工表 当成员工表
-- t_employee as mgr 把员工表 当成存储领导信息的领导表
-- emp.mid = mgr.eid; 员工表的领导编号就是领导表的员工编号
```

第 11 章 select 的 7 个字句

11.1 七个子句顺序

- (1) from: 从哪些表中筛选数据
- (2) join on: 多表关联查询
- (3) where: 从表中筛选数据的条件
- (4) group by: 分组依据
- (5) having: 在分组结果中再次筛选
- (6) order by: 排序
- (7) limit: 分页

11.2 from

```
select * from t_employee;
```

11.3 on

```
select temp1.*,temp2.ename,temp2.salary
from t_employee as temp1
join t_employee as temp2
on temp2.salary > temp1.salary*2;
```

11.4 where

```
select * from t_employee
where salary>10000;
```

11.5 group by

```
-- 查询每一个部门的平均薪资
select did,round(avg(salary),2)
from t_employee
group by did;
-- 查询每一个部门的平均薪资，显示部门编号，部门的名称，该部门的平均薪资
select t_department.did,dname,round(avg(salary),2)
from t_department left join t_employee
on t_department.did = t_employee.did
```

```
group by t_department.did;
-- 查询每一个部门的平均薪资，显示部门编号，部门的名称，该部门的平均薪资
-- 要求，如果没有员工的部门，平均薪资不显示 null，显示 0
select t_department.did,dname,ifnull(round(avg(salary),2),0)
from t_department left join t_employee
on t_department.did = t_employee.did
group by t_department.did;
```

11.5.1 with rollup 合计

```
-- 按照部门统计人数
select did,count(*)
from t_employee
group by did;
-- 按照部门统计人数，并合计总数
select did,count(*)
from t_employee
group by did with rollup;

select
    ifnull(did,'合计') as "部门编号",
    count(*) as "人数"
from t_employee
group by did with rollup;
```

11.5.2 多字段分组

```
-- 按照不同的部门，不同的职位，分别统计男和女的员工人数
select did,job_id,gender,count(*)
from t_employee
group by did,job_id,gender;
```

11.5.3 分组统计时，select 后字段列表的问题

```
-- 分组统计时，select 后面只写和分组统计有关的字段
-- 其他无关字段不要出现，否则会引起歧义
select eid,ename,did,count(*)
from t_employee
group by did;
-- eid,ename 此时不应该出现在 select 后面
select did,count(*)
from t_employee
group by did;
```

11.6 having

having 子句后面也写条件。

where 是对原表中的记录的筛选。where 后面不能出现分组函数。

having 是对统计结果（分组函数计算后）的筛选。having 后面能出现分组函数。

```
-- 查询每一个部门的女员工的平均薪资，显示部门编号，部门的名称，该部门的平均薪资
-- 要求，如果没有员工的部门，平均薪资不显示 null，显示 0
-- 最后只显示平均薪资高于 12000 的部门
select t_department.did,dname,ifnull(round(avg(salary),2),0)
from t_department left join t_employee
on t_department.did=t_employee.did
where gender='女'
group by t_department.did
having ifnull(round(avg(salary),2),0)>12000;
-- 查询每一个部门薪资超过 10000 的男女员工的人数，显示部门编号，部门的名称，性别，
人数
-- 只显示人数低于 3 人的
select t_department.did,dname,gender,count(eid)
from t_employee right join t_department
on t_employee.did=t_department.did
where salary>10000
group by t_department.did,gender
having count(eid)<3;
```

11.7 order by

asc 代表升序，desc 代表降序，默认升序。

```
-- 查询员工信息，按照薪资从高到低
select * from t_employee
order by salary desc;
-- 查询每一个部门薪资超过 10000 的男女员工的人数，显示部门编号，部门的名称，性别，
人数
-- 只显示人数低于 3 人的，按照人数升序排列
select t_department.did,dname,gender,count(eid)
from t_employee
right join t_department
on t_employee.did=t_department.did
where salary>10000
group by t_department.did,gender
having count(eid)<3
order by count(eid);
-- 查询员工的薪资，按照薪资从低到高，薪资相同按照员工编号从高到低
select *
from t_employee
order by salary asc,eid desc;
```

11.8 limit

limit 子句是用于分页显示结果。

limit m,n:

- n: 表示最多该页显示几行
- m: 表示从第几行开始取记录, 第一个行的索引是 0
- $m = (page-1)*n$, page 表示第几页

如每页最多显示 5 条, n=5:

第 1 页, page=1, $m = (1-1)*5 = 0$; limit 0,5

第 2 页, page=2, $m = (2-1)*5 = 5$; limit 5,5

第 3 页, page=3, $m = (3-1)*5 = 10$; limit 10,5

```
-- 查询员工表的数据, 分页显示, 每页显示 5 条记录
-- 第 1 页
select * from t_employee limit 0,5;
-- 第 2 页
select * from t_employee limit 5,5;
-- 第 3 页
select * from t_employee limit 10,5;
-- 查询所有的男员工信息, 分页显示, 每页显示 3 条, 第 2 页
select *
from t_employee
where gender='男'
limit 3,3;
-- 查询每一个编号为偶数的部门, 显示部门编号, 名称, 员工数量
-- 只显示员工数量>=2 的结果, 按照员工数量升序排列
-- 每页显示 2 条, 显示第 1 页
select t_department.did,dname,count(eid)
from t_employee
right join t_department
on t_employee.did=t_department.did
where t_department.did%2=0
group by t_department.did
having count(eid)>=2
order by count(eid)
limit 0,2;
```

第 12 章 子查询

12.1 select 中嵌套子查询

子查询: 嵌套在另一个 SQL 语句中的查询。

select 语句可以嵌套在另一个 select、update、delete、insert、create 等语句中。

12.1.1 select 中嵌套子查询

```
-- 在 t_employee 表中查询每个人薪资和公司平均薪资的差值
-- 并显示员工薪资和公司平均薪资相差 5000 元以上的记录
select
    ename as "姓名",
    salary as "薪资",
    round((select avg(salary) from t_employee),2) as "全公司平均薪资",
    round(salary-(select avg(salary) from t_employee),2) as "差值"
from t_employee
where abs(round(salary-(select avg(salary) from t_employee),2))>5000;
-- 在 t_employee 表中查询每个部门平均薪资和公司平均薪资的差值
select
    did,
    avg(salary),
    avg(salary)-(select avg(salary) from t_employee)
from t_employee
group by did;
-- 使用子查询按薪资大小编号
select
    ename,
    salary,
    (select count(*) from t_employee as temp2 where
temp1.salary>temp2.salary) as rk
from t_employee as temp1
order by rk;
```

12.1.2 where 或 having 中嵌套子查询

当子查询结果作为外层另一个 SQL 的过滤条件，通常把子查询嵌入到 where 或 having 中。根据子查询结果的情况，分为如下三种情况：

- 当子查询的结果是单列单个值，那么可以直接使用比较运算符，如 “<”、“<=”、“>”、“>=”、“=”、“!=” 等与子查询结果进行比较
- 当子查询的结果是单列多个值，那么可以使用比较运算符 in 或 not in 进行比较
- 当子查询的结果是单列多个值，还可以使用比较运算符，如 “<”、“<=”、“>”、“>=”、“=”、“!=” 等搭配 any、all 等关键字与查询结果进行比较

```
-- 在 t_employee 表中查询薪资最高的员工姓名和薪资
-- 取表中第一行员工的姓名和全公司最高的薪资值一起显示
select ename,salary
from t_employee
```

```
where salary=(select max(salary) from t_employee);
-- 在 t_employee 表中查询比全公司平均薪资高的男员工姓名和薪资
select ename,salary
from t_employee
where salary>(select avg(salary) from t_employee) and gender='男';
-- 在 t_employee 表中查询和 白露, 谢吉娜 同一部门的员工姓名和电话
select ename,tel,did
from t_employee
where did in (select did from t_employee where ename='白露' or ename='谢吉娜');

select ename,tel,did
from t_employee
where did=any(select did from t_employee where ename='白露' or ename='谢吉娜');
-- 在 t_employee 表中查询薪资比 白露, 李诗雨, 黄冰茹 三个人的薪资都要高的员工姓名和薪资
select ename,salary
from t_employee
where salary>all(select salary from t_employee where ename in('白露','李诗雨','黄冰茹'));

select ename,salary
from t_employee
where salary>(select max(salary) from t_employee where ename in('白露','李诗雨','黄冰茹'));
-- 查询 t_employee 和 t_department 表, 按部门统计平均工资
-- 显示部门平均工资比全公司的总平均工资高的部门编号、部门名称、部门平均薪资
-- 并按照部门平均薪资升序排列
select t_department.did,dname,avg(salary)
from t_employee
right join t_department
on t_employee.did=t_department.did
group by t_department.did
having avg(salary)>(select avg(salary) from t_employee)
order by avg(salary);
```

12.1.3 exists 型子查询

exists 型子查询也是存在外层 select 的 where 子句中, 不过它和上面的 where 型子查询的工作模式不相同, 所以这里单独讨论它。

如果 exists 关键字后面的参数是一个任意的子查询, 系统将对子查询进行运算以判断它是否返回行, 如果至少返回一行, 那么 exists 的结果为 true, 此时外层查询语句将进行查询;

如果子查询没有返回任何行,那么 `exists` 的结果为 `false`,此时外层查询语句不进行查询。`exists` 和 `not exists` 的结果只取决于是否返回行,而不取决于这些行的内容,所以这个子查询输入列表通常是无关紧要的。

如果 `exists` 关键字后面的参数是一个关联子查询,即子查询的 `where` 条件中包含与外层查询表的关联条件,那么此时将对外层查询表做循环,即在筛选外层查询表的每一条记录时,都看这条记录是否满足子查询的条件,如果满足就再用外层查询的其他 `where` 条件对该记录进行筛选,否则就丢弃这行记录。

```
-- exists()中的子查询和外面的查询没有联合的情况下
-- 如果 exists()中的子查询没有返回任何行,那么外面的子查询就不查了

-- exists()中的子查询与外面的查询有联合工作的情况下
-- 循环进行把外面查询表的每一行记录的值,代入()中子查询,如果可以查到结果
-- 就留下外面查询的这条记录,否则就舍去

-- 查询 t_employee 表中是否存在部门编号为 null 的员工
-- 如果存在,查询 t_department 表的部门编号、部门名称
select * from t_department
where exists(select * from t_employee where did is null);
-- 查询 t_department 表是否存在与 t_employee 表相同部门编号的记录
-- 如果存在,查询这些部门的编号和名称
select * from t_department
where exists(select * from t_employee where
t_employee.did=t_department.did);
-- 查询结果等价于下面的 SQL
select distinct t_department.*
from t_department
inner join t_employee
on t_department.did=t_employee.did;
```

12.1.4 from 中嵌套子查询

当子查询结果是多列的结果时,通常将子查询放到 `from` 后面,然后采用给予子查询结果取别名的方式,把子查询结果当成一张“动态生成的临时表”使用。

```
-- 当一个查询要基于另一个查询结果来筛选的时候
-- 另一个查询还是多行多列的结果,那么就可以把这个查询结果当成一张临时表
-- 放在 from 后面进行再次筛选

-- 在 t_employee 表中,查询每个部门的平均薪资
-- 然后与 t_department 表联合查询
-- 所有部门的部门编号、部门名称、部门平均薪资
select t_department.did,dname,pingjun
```

```

from t_department
left join (select did,avg(salary) as pingjun from t_employee group by did)
temp
on t_department.did=temp.did;
-- 在 t_employee 表中查询每个部门中薪资排名前 2 的员工姓名、部门编号和薪资
select *
from (
    select
        ename,
        did,
        salary,
        dense_rank() over (partition by did order by salary desc) as paiming
    from t_employee
) temp
where temp.paiming<=2;

```

12.2 update 中嵌套子查询

```

-- 修改 t_employee 表中部门编号和测试部部门编号相同的员工薪资为原来薪资的 1.5 倍
update t_employee
set salary = salary * 1.5
where did=(select did from t_department where dname='测试部');
-- 修改 t_employee 表中 did 为 null 的员工信息
-- 将他们的 did 值修改为测试部的部门编号
-- 这种子查询必须是单个值，否则无法赋值
update t_employee
set did = (select did from t_department where dname='测试部')
where did is null;
-- 修改 t_employee 表中李冰冰的薪资值等于孙红梅的薪资值
update t_employee
set salary = (select salary from(select salary from t_employee where ename='
孙红梅')temp)
where ename='李冰冰';
-- 当 update 的表和子查询的表是同一个表时，需要将子查询的结果用临时表的方式表示
-- 即再套一层子查询，使得 update 和最外层的子查询不是同一张表

-- 修改 t_employee 表李冰冰的薪资与她所在部门的平均薪资一样
update t_employee
set salary =
(
    select pingjun
    from
        (
            select avg(salary) pingjun
            from t_employee

```

```

        where did=(
            select did
            from t_employee
            where ename='李冰冰'
        )
    ) temp
)
where ename='李冰冰';

```

12.3 delete 中嵌套子查询

```

-- 从 t_employee 表中删除测试部的员工记录
delete from t_employee
where did = (select did from t_department where dname='测试部');
-- 从 t_employee 表中删除和李冰冰同一个部门的员工记录
delete from t_employee
where did=(select did from t_employee where ename='李冰冰');
-- 报错，因为删除和子查询是同一张表
delete from t_employee
where did=(select did from (select did from t_employee where ename='李冰冰')temp);

```

12.4 使用子查询复制表结构和数据

1) 复制表结构

```

-- 仅复制表结构，可以用 create 语句
create table department like t_department;

```

2) 复制一条或多条记录

```

-- 使用 insert 语句+子查询，复制数据，此时 insert 不用写 values
insert into department (select * from t_department where did<=3);

```

3) 同时复制表结构和记录

```

-- 同时复制表结构+数据
create table d_department as (select * from t_department);
-- 如果 select 后面是部分字段，复制的新表就只有这一部分字段

```

12.5 通用表达式

通用表达式简称为 CTE (Common Table Expressions)。CTE 是命名的临时结果集，作用范围是当前语句。CTE 可以理解为一个可以复用的子查询，但是和子查询又有区别，一个 CTE 可以引用其他 CTE，CTE 还可以是自引用(递归 CTE)，也可以在同一查询中多次引用，但子查询不可以。

```

with [recursive]
cte_name [(字段名 1, 字段名 2)] as (子查询),
cte_name [(字段名 1, 字段名 2)] as (子查询)

```

通用表达式以 with 开头，如果 with 后面加 recursive 就表示接下来在通用表达式中需要递归引用自己，否则就不递归引用。每一个通用表达式都需要有一个名字，它相当于是子查询结果集的名字。

```
-- 在 t_employee 表中查询每个人薪资和公司平均薪资的的差值
with
temp as (select round(avg(salary),2) as pingjun from t_employee)
select
    ename as "员工姓名",
    salary as "薪资",
    pingjun "公司平均薪资",
    round(salary - pingjun,2) "差值"
from t_employee,temp
having abs(差值)>5000;
-- 查询薪资低于 9000 的员工编号，员工姓名，员工薪资，领导编号，领导姓名，领导薪资
with
emp as (select eid,ename,salary,`mid` from t_employee where salary<9000),
mgr(meid,mename,msalary) as (select eid,ename,salary from t_employee)
select
    eid as "员工编号",
    ename as "员工姓名",
    salary as "员工薪资",
    meid as "领导编号",
    mename as "领导姓名",
    msalary as "领导薪资"
from emp join mgr on emp.mid=mgr.meid;
-- 查询 eid 为 21 的员工，和他所有领导，直到最高领导
-- 建表，设置多层领导
create table emp as (select eid,ename,salary,tel,`mid` from t_employee where salary<10000);
update emp set mid=19 where eid=21;
update emp set mid=17 where eid=19;
update emp set mid=16 where eid=17;
update emp set mid=15 where eid=16;
update emp set mid=4 where eid=15;
update emp set mid=null where eid=4;
select * from emp;
with recursive
cte as(
    select eid,ename,`mid`
    from emp
    where eid=21
    union all
```



```
select emp.eid,emp.ename,emp.mid
from emp join cte
on emp.eid=cte.mid
where emp.eid is not null
)
select * from cte;
```

第 13 章 约束

13.1 约束的作用

约束是为了保证数据的完整性,数据完整性(Data Integrity)是指数据的精确性(Accuracy)和可靠性(Reliability)。它是应防止数据库中存在不符合语义规定的数据和防止因错误信息的输入输出造成无效操作或错误信息而提出的。

数据的完整性要从以下四个方面考虑:

- **实体完整性(Entity Integrity)**: 例如, 同一个表中, 不能存在两条完全相同无法区分的记录
- **域完整性(Domain Integrity)**: 例如, 年龄范围 0-120, 性别范围“男/女”
- **引用完整性(Referential Integrity)**: 例如, 员工所在部门, 在部门表中要能找到这个部门
- **用户自定义完整性(User-defined Integrity)**: 例如, 用户名唯一、密码不能为空等, 本部门经理的工资不得高于本部门职工的平均工资的 5 倍

可以在创建表时规定约束(通过 create table 语句), 或者在表创建之后也可以添加约束(通过 alter table 语句)。

13.2 约束的类型

- 键约束: 主键约束、外键约束、唯一键约束
- Not NULL 约束: 非空约束
- Check 约束: 检查约束
- Default 约束: 默认值约束
- 自增是键约束字段的一个额外的属性

13.2.1 表级约束和列级约束

表级约束: 不仅要看约束字段当前单元格的数据, 还要看其他单元格的数据。

键约束和检查约束是表级约束。

列级约束：约束字段只看当前单元格的数据即可，和其他单元格无关。

非空约束和默认值约束是列级约束。

所有的表级约束都可以在 “information_schema.table_constraints” 表中查看。

```
select * from information_schema.table_constraints where table_name = '表名';
```

13.2.2 约束和索引

在 MySQL 中键约束会自动创建索引，提高查询效率。

约束和索引不同：

- 约束是一个逻辑概念，不会占用物理空间
- 索引是一个物理概念，会占用物理空间

例如，字典里面有要求，不能有重复的字（字一样，读音也一样），这是约束。字典里面有“目录”，它可以快速的查找某个字，目录需要占用单独的页，这是索引。

13.3 非空约束

not null：限定某个字段/某列的值不允许为空。

- 只能某个列单独限定非空，不能组合非空
- 一个表可以有很多列都分别限定了非空

13.3.1 指定非空约束

1) 建表时

```
create table 表名(  
    字段名 数据类型 not null,  
    字段名 数据类型 not null  
);
```

2) 建表后

```
alter table 表名 modify 字段名 数据类型 not null;  
-- 如果该字段已经有值了，给该字段增加非空约束，要求该字段的值不能有 NULL 值  
-- 否则需要先处理 NULL 值才能加上非空约束
```

13.3.2 删除非空约束

```
alter table 表名 modify 字段名 数据类型;  
-- 如果使用 modify 修改有非空约束的字段时想要保留非空约束，必须带上 not null  
-- 否则会在修改时，把非空约束丢掉
```

13.3.3 示例

```
create table not_null_demo(  
    字段名 数据类型 not null,  
    字段名 数据类型 not null  
);
```

```

    id int not null,
    name varchar(20) not null,
    birthday date
);
insert into not_null_demo values(null,null,null);
insert into not_null_demo(id) values(1);
-- 此时 name 设定了非空约束，但没有指定默认值
-- 如果没赋值只能处理为 NULL，违反了非空约束
-- 在 insert 添加记录时，必须给所有没有指定默认值的非空约束字段赋值
insert into not_null_demo(id,name) values(1,'张三');
insert into not_null_demo(id,name,birthday) values(2,'李四','2000-1-1');
insert into not_null_demo values(3,'王五','2000-2-1');
insert into not_null_demo values(4,'赵六',null);
-- 给 birthday 字段添加非空约束
alter table not_null_demo modify column birthday date not null;
-- 失败的原因是因为 birthday 字段现在有 NULL 值，添加不上非空约束
-- 如果要加，必须先将原来的 NULL 值修改掉
update not_null_demo set birthday = '2000-5-1' where birthday is null;
alter table not_null_demo modify column birthday date not null;
desc not_null_demo;
-- 删除 birthday 字段的非空约束
alter table not_null_demo modify column birthday date;
desc not_null_demo;
-- 修改 name 字段的数据类型为 varchar(30)，本来想要保留非空约束的
alter table not_null_demo modify column name varchar(30);
desc not_null_demo;
-- alter 没有加 not null，会把 name 字段的非空约束给丢掉

```

13.4 唯一键约束

unique key：用来限制某列的值或几个字段的值组合不能重复。

- 一个表可以有很多个唯一键约束
- 每一个唯一键约束字段都会自动创建索引
- 唯一键约束允许为空
- 唯一键约束也可以是复合唯一
- 删除唯一键约束的索引来删除唯一键约束
- 索引名默认是字段名，复合唯一默认是第一个字段名

13.4.1 指定唯一键约束

1) 建表时

```
create table 表名(
```

```
    字段名 数据类型 unique key,  
    字段名 数据类型 unique key  
);  
-- 或  
create table 表名(  
    字段名 1 数据类型,  
    字段名 2 数据类型,  
    unique key(字段名 1),  
    unique key(字段名 2)  
);
```

2) 建表后

```
alter table 表名 add unique (字段名);
```

13.4.2 复合唯一

```
-- 多个字段的组合是唯一  
create table 表名(  
    字段名 1 数据类型,  
    字段名 2 数据类型,  
    字段名 3 数据类型,  
    unique key(字段名 1, 字段名 2, 字段名 3)  
);
```

13.4.3 查看唯一键约束

```
desc 表名;  
show create table 表名;  
show index from 表名; -- 查看表的索引信息  
select * from information_schema.table_constraints where table_name = '  
表名';
```

13.4.4 删除唯一键约束

```
-- 删除唯一键约束需要手动删除对应的索引  
alter table 表名 drop index 索引名;
```

13.4.5 示例

```
-- 建表，限定编号、身份证号码和手机号码唯一  
drop table if exists emp;  
create table emp(  
    id int unique key, -- 表示 id 字段值不能重复  
    name varchar(20),  
    cardid char(18),  
    tel char(11),  
    unique key(cardid), -- 表示 cardid 字段值不能重复  
    unique key(tel) -- 表示 tel 字段值不能重复  
);
```

```
desc emp;
show create table emp;
-- 添加数据
insert into emp values(1,'张三','2567854452222','13589587585');
insert into emp values(2,'张三','2567854452221','13589587596');
select * from emp;
insert into emp values(3,'李四','2567854452223','13589587596');
-- 手机号码重复, 报错
insert into emp values
(3,'李四',null,null),
(4,'王五',null,null);
-- NULL 可以重复
select * from emp;
-- 查看 emp 表的索引
show index from emp;
-- 删除唯一键约束
-- 如果在指定唯一键约束时, 没有手动定义名字, 默认就是字段名
-- 建议在删除时用 show index 语句查看一下索引名
-- 删除 emp 表的 cardid 的唯一键约束
alter table emp drop index cardid;
show index from emp;
desc emp;
-- 建表后给 emp 表的 cardid 增加唯一键约束
alter table emp add unique key(cardid);
desc emp;
show index from emp;
-- 复合唯一, 表示两个或更多个字段值的组合唯一, 单个字段看不唯一
-- 学生表
drop table if exists stu;
create table stu(
    id int,
    name varchar(20)
);
insert into stu values(1,'张三'),(2,'李四');
select * from stu;
-- 课程表
drop table if exists course;
create table course(
    id int,
    title varchar(50)
);
insert into course values(1,'python'),(2,'mysql');
select * from course;
-- 选课表
```

```
drop table if exists course;
create table xuanke(
    xid int unique key, -- xid 不能重复
    sid int,
    cid int,
    score int,
    unique key(sid,cid)
    -- 这么写表示 sid 和 cid 的组合不能重复
    -- 单独看 sid 和 cid 是可以重复的
);
insert into xuanke values
(1,1,1,89),
(2,1,2,96),
(3,2,1,75),
(4,2,2,96);
select * from xuanke;
insert into xuanke values(5,1,1,100); -- sid 为 1 和 cid 为 1 组合重复
desc xuanke;
show index from xuanke;
-- 组合唯一键索引名默认是 unique key(字段 1, 字段 2)的第一个字段名
-- 删除复合唯一键约束
alter table xuanke drop index sid;
desc xuanke;
show index from xuanke;
```

13.5 主键约束

primary key: 用来唯一的确定一条记录

- 唯一并且非空
- 一个表最多只能有一个主键约束
- 如果主键是由多列组成，可以使用复合主键
- 主键列会自动创建索引（能够根据主键查询的，就根据主键查询，效率更高），

mysql 会给每个表的主键列创建索引，会开辟单独的物理空间来存储每一个主键的目录表（Btree 结构）。这样可以根据主键快速查询到某一行的记录

- 如果删除主键约束了，主键约束对应的索引就自动删除了

13.5.1 唯一键约束和主键约束区别

- 唯一键约束一个表可以有多个，但是主键约束只有一个。
- 唯一键约束本身不带非空限制，如果需要非空，需要单独定义。主键约束不用再定义 not null，自身就带非空限制。

13.5.2 指定主键约束

1) 建表时

```
create table 表名(  
    字段名 数据类型 primary key,  
    字段名 数据类型  
);  
-- 或  
create table 表名(  
    字段名 1 数据类型,  
    字段名 2 数据类型,  
    primary key(字段名 1)  
);
```

2) 建表后

```
alter table 表名 add primary key(字段列表);  
-- 字段列表可以是一个字段，也可以是多个字段，如果是多个字段的话，是复合主键
```

13.5.3 复合主键

```
create table 表名(  
    字段名 1 数据类型,  
    字段名 2 数据类型,  
    字段名 3 数据类型,  
    primary key(字段名 1, 字段名 2)  
);
```

13.5.4 删除主键约束

```
alter table 表名 drop primary key;
```

13.5.5 示例

```
drop database if exists atguigu;  
create database atguigu;  
use atguigu;  
-- 创建员工表  
create table emp(  
    id int primary key, -- 主键  
    ename varchar(20) not null, -- 非空  
    cardid char(18) unique key not null, -- 非空且唯一  
    tel char(11) unique key, -- 唯一  
    address varchar(100)  
);  
desc emp;  
insert into emp values  
(1, '张三', '524265198235684255', '18536955456', null),
```



```
(2, '李四', '524265198235685255', null, null),
(3, '李四', '524265198235685895', null, null);
select * from emp;
insert into emp values(3, '王五', '524265198235675895', null, null); -- 主键
重复
insert into emp values(null, '王五', '524265198235675775', null, null); -- 主
键为 null
-- 创建一个表，两个字段设置主键，报错
create table xuanke(
    sid int primary key,
    cid int primary key,
    score int
);
-- 复合主键约束
-- 学生表
create table stu(
    id int,
    name varchar(20)
);
insert into stu values(1, '张三'), (2, '李四');
select * from stu;
-- 课程表
create table course(
    id int,
    title varchar(50)
);
insert into course values(1, 'python'), (2, 'mysql');
select * from course;
-- 选课表
create table xuanke(
    sid int,
    cid int,
    score int,
    primary key(sid, cid) -- 复合主键
);
insert into xuanke values(1, 1, 96), (2, 1, 85), (1, 2, 75), (2, 2, 45);
insert into xuanke values(1, 1, 75);
-- 报错，重复主键
-- 查看 emp 表和 xuanke 表的索引
show index from emp;
show index from xuanke;
-- 删除 xuanke 表的主键约束
alter table xuanke drop primary key;
show index from xuanke;
```

```
desc xuanke;
-- 删除主键约束时，对应的索引、键约束删除了，但是主键约束自带的非空约束没有删除
-- 如果要去掉的话，需要用删除非空约束的方式，单独删除
-- 建表后增加主键约束
alter table xuanke add primary key(sid,cid);
desc xuanke;
-- 删除 emp 的主键约束
alter table emp drop primary key;
desc emp;
-- 自动把 cardid 识别为主键，因为 cardid 定义了唯一键+非空约束，但是不同于真正的主键
show index from emp;
-- 给 emp 表的 id 字段增加主键约束
alter table emp add primary key(id);
show index from emp;
```

13.5.6 自增约束

auto_increment: 给某个字段自动赋值，这个值如无干扰每次+1。

➤ 一个表只能有一个自增字段，因为一个表只有一个 AUTO_INCREMENT 属性记录自增字段值

➤ 自增字段只能是 key 字段，即定义了主键、唯一键等键约束的字段

➤ 自增字段应该是数值类型，一般都是整数类型

➤ 如果自增列指定了 0 和 null，会在当前最大值的基础上自增，如果自增列手动指定了具体值，直接赋值为具体值

➤ 如果手动修改 AUTO_INCREMENT 属性值，必须 > 当前自增字段的最大值

13.5.6.1 指定自增约束

1) 建表时

```
create table 表名(
    字段名 数据类型 primary key auto_increment,
    字段名 数据类型 unique key not null
);
```

2) 建表后

```
alter table 表名 modify 字段名 数据类型 auto_increment;
```

13.5.6.2 删除自增约束

```
alter table 表名 modify 字段名 数据类型;
-- 去掉 auto_increment 相当于删除
```

13.5.6.3 示例

```
drop table if exists emp;
```

```

create table emp(
    eid int auto_increment,
    ename varchar(20)
);
-- 报错，只能有一个自增字段且需为 key 字段
create table emp(
    eid int primary key auto_increment,
    ename varchar(20)
);
insert into emp(ename) values('李四'); -- 不给自增字段指定值，也是自增
insert into emp values(null, '张三'); -- 给自增字段赋值 NULL，也是自增
insert into emp values(0, '王五'); -- 给自增字段赋值 0，也是自增
select * from emp;
insert into emp values(-5, '王五');
insert into emp values(15, '田七');
select * from emp;
-- 当手动给自增字段赋值时
-- 如果这个值大于当前 AUTO_INCREMENT 属性记录的自增值时
-- 会修改 AUTO_INCREMENT 属性值
-- 下次就从这个值基础上自增
-- 建议不要随意修改 AUTO_INCREMENT 的值，让他自动维护
insert into emp values(7, '周八');
select * from emp;
-- 修改 emp 表 AUTO_INCREMENT 值为 8
alter table emp AUTO_INCREMENT = 8;
show create table emp;
-- AUTO_INCREMENT 值都不能修改为<当前自增字段最大的值
-- 删除 eid 字段的自增属性
alter table emp modify column eid int;
desc emp;
-- modify 会影响列级约束，默认值、非空等约束
-- 表级约束（主键、唯一键）不受影响
-- 给 eid 字段的添加自增属性
alter table emp modify column eid int auto_increment;
desc emp;

```

13.6 默认值约束

default: 给某个字段/某列指定默认值，当添加时或修改时，可以使用默认值。

13.6.1 指定默认值约束

1) 建表时

```

create table 表名(
    字段名 数据类型 primary key,

```

```

    字段名 数据类型 unique key not null,
    字段名 数据类型 unique key,
    字段名 数据类型 not null default 默认值
);
-- 默认值约束一般不在唯一键和主键列上加

```

2) 建表后

```
alter table 表名 modify 字段名 数据类型 default 默认值;
```

13.6.2 删除默认值约束

```

alter table 表名 modify 字段名 数据类型;
-- 如果使用 modify 修改有默认值约束的字段时想要保留默认值约束，必须带上 default
默认值
-- 否则会在修改时，把默认值约束丢掉

```

13.6.3 示例

```

drop table if exists emp;
create table emp(
    eid int primary key,
    ename varchar(20) not null,
    gender enum('男','女') default '男' not null, -- 非空并且有默认值
    address varchar(100) default '不详' -- 可以指定为 null
);
insert into emp values(1,'张三'); -- 错误，值的数量和字段的数量不匹配
insert into emp(eid,ename) values(1,'张三');
insert into emp values(2,'李四',default,default);
insert into emp values(3,'王五',default,null);
select * from emp;
-- 删除 emp 表的 address 的默认值约束
alter table emp modify column address varchar(100);
desc emp;
-- 给 emp 表 address 增加 不详 默认值
alter table emp modify column address varchar(100) default '不详';
desc emp;

```

13.7 检查约束

check: 检查约束用于限制字段中的值的范围。如果对单个字段定义检查约束，那么该字段只允许特定范围的值。如果对一个表定义检查约束，那么此约束会基于行中其他字段的值在特定的字段中对值进行限制。

在 MySQL8.0.16 版本之前，create table 语句支持给单个字段定义检查约束的语法，但是不起作用。

在 MySQL8.0.16 版本之后，create table 语句既支持给单个字段定义列级检查约束，也支持定义表级检查约束。

13.7.1 指定检查约束

1) 建表时

```
create table 表名(
    字段名 数据类型 check(条件), -- 在字段后面直接加检查约束
    字段名 数据类型,
    字段名 数据类型,
    check (条件) enforced -- 可以限定两个字段之间的取值条件
);
-- 如果省略或指定为 enforced, 则会强制执行约束, 不满足约束的数据行不能插入成功
-- 如果写 not enforced, 则不满足检查约束也没关系
```

2) 建表后

```
alter table 表名 add check(条件);
```

13.7.2 删除检查约束

```
alter table 表名 drop check 检查约束名;
```

13.7.3 示例

```
drop table if exists emp;
create table emp(
    id int primary key auto_increment,
    name varchar(20) not null,
    age int check(age>=18), -- 年龄>=18 岁
    birthday date not null, -- 出生日期
    hiredate date not null, -- 入职日期
    check(year(hiredate)-year(birthday)>=18) -- 入职时>=18 岁
);
insert into emp values(null,'张三',23,'2000-1-1','2021-11-30');
select * from emp;
insert into emp values(null,'张三',8,'2013-1-1','2021-11-30'); -- 不满足条件
insert into emp values(null,'张三',28,'2013-1-1','2021-11-30'); -- 不满足条件
desc emp;
show create table emp;
-- 查看 emp 表的约束
select * from information_schema.table_constraints where table_name = 'emp';
-- 只能看到主键、唯一键、外键、检查约束
-- 默认值、非空约束这里看不到
```

```
-- 删除 emp 表的 age 字段的检查约束 emp_chk_1
-- 删除 emp 表的 hiredate 和 birthday 字段的检查约束 emp_chk_2
alter table emp drop check emp_chk_1;
alter table emp drop check emp_chk_2;
show create table emp;
-- 使用 modify 给 age 字段添加检查约束 age>=18, 不起作用
alter table emp modify age int check(age>=18); -- 语法没问题, 但是没有真正起作用
insert into emp values(null, '李四', 6, '1998-5-1', '2021-1-2'); -- 添加成功, 说明检查约束没有起作用
-- 使用 add check 给 age 字段添加检查约束 age>=18
alter table emp add check(age>=18);
-- 因为表中有违反(age>=18)的数据, 必须先处理
select * from emp;
update emp set age = 26 where age<18;
-- 使用 add check 给 age 字段添加检查约束 age>=18
alter table emp add check(age>=18);
insert into emp values(null, '李四', 6, '1998-5-1', '2021-1-2');
```

13.8 外键约束

foreign key: 限定某个表的某个字段的引用完整性, 比如: 员工表的员工所在部门的选择, 必须在部门表能找到对应的部分。

外键约束会影响性能, 效率, 所以很多人不愿意加外键约束。

1) 建和不建外键约束有什么区别

➤ 建外键约束, 操作(创建表、删除表、添加、修改、删除)会受到限制, 从语法层面受到限制。例如在员工表中不可能添加一个员工信息, 它的部门的值在部门表中找不到。

➤ 不建外键约束, 操作(创建表、删除表、添加、修改、删除)不受限制, 要保证数据的引用完整性, 只能依靠程序员的自觉, 或者是在程序中进行限定。例如在员工表中, 可以添加一个员工的信息, 它的部门指定为一个完全不存在的部门。

2) 主表和从表/父表和子表

主表(父表): 被引用的表, 被参考的表

从表(子表): 引用别人的表, 参考别人的表

例如员工表与部门表。员工表的员工所在部门这个字段的值要参考部门表, 部门表是主表, 员工表是从表。

例如学生表、课程表、选课表。选课表的学生和课程要分别参考学生表和课程表, 学生表和课程表是主表, 选课表是从表。

3) 外键约束的特点

- 在从表中指定外键约束，并且一个表可以建立多个外键约束
- 创建表时就指定外键约束的话，先创建主表，再创建从表
- 删表时，先删从表（或先删除外键约束），再删除主表。或者先解除关系，再各自删除
- 从表的外键列，必须引用/参考主表的键列（主键或唯一键），因为被依赖/被参考的值必须是唯一的
- 从表的外键列的数据类型，要与主表被参考/被引用的列的数据类型一致，并且逻辑意义一致。例如都是表示部门编号，都是 int 类型
- 外键列也会自动建立索引（根据外键查询效率更快）
- 外键约束删除后索引不会自动删除，如果要删除对应的索引，必须手动删除

13.8.2 指定外键约束

1) 建表时

```
create table 主表(
    字段 1 数据类型 primary key,
    字段 2 数据类型
);

create table 从表(
    字段 1 数据类型 primary key,
    字段 2 数据类型,
    foreign key (从表的某个字段) references 主表(被参考字段)
);
-- (从表的某个字段)的数据类型必须与主表(被参考字段)的数据类型一致，逻辑意义也一样
-- (从表的某个字段)的字段名可以与主表(被参考字段)的字段名不同
```

2) 建表后

```
alter table 从表 add foreign key (从表的字段) references 主表(被引用字段) [on
update xx] [on delete xx];
```

13.8.3 查看外键约束

```
desc 从表; -- 可以看到外键约束，但看不到外键约束名
show create table 从表; -- 可以看到外键约束名
select * from information_schema.table_constraints where table_name='表名';
-- information_schema 数据库名(系统库)
-- table_constraints 表名(专门存储各个表的约束)
```


13.8.4 查看外键字段索引

```
show index from 表名;
```

13.8.5 删除外键约束

```
-- 先查看约束名和删除外键约束
select * from information_schema.table_constraints where table_name='表名';
alter table 从表 drop foreign key 外键约束名;
-- 查看索引名和删除索引
show index from 表名;
alter table 从表 drop index 索引名;
```

13.8.6 示例

```
drop table if exists dept;
create table dept(
    did int primary key auto_increment,
    dname varchar(50) unique key not null
);
drop table if exists emp;
create table emp(
    id int primary key auto_increment,
    name varchar(20) not null,
    departmentid int,
    -- 外键约束只能在字段列表下面单独定义，不能在字段后面直接定义
    foreign key (departmentid) references dept(did)
);
desc dept;
desc emp;
show create table dept;
show create table emp;
-- 查看系统库的约束表
select * from information_schema.table_constraints where table_name='emp';
-- 添加父表数据，没有影响
insert into dept values(null,'财务'),(null,'教学'),(null,'咨询'),(null,'后勤');
select * from dept;
-- 添加子表数据，有影响，受到约束
insert into emp values(null,'张三',1); -- 成功
insert into emp values(null,'李四',1); -- 成功
insert into emp values(null,'王五',2); -- 成功
insert into emp values(null,'赵六',6); -- 失败
-- departmentid=6 在父表 dept 中找不到对应部门
select * from emp;
```

```
-- 修改子表的外键字段的信息，有影响，受到约束
update emp set departmentid = 3 where id=1; -- 成功
update emp set departmentid = 6 where id=3; -- 失败
-- departmentid=6 在父表 dept 中找不到对应部门
select * from emp;
-- 修改父表的被引用字段的值，受约束
update dept set did = 6 where did=1; -- 失败
-- did=1 的部门被子表引用
update dept set did = 6 where did=4; -- 成功
-- did=4 的部门没有被子表引用
select * from dept;
-- 删除父表的记录，受约束
delete from dept where did=6; -- 成功
-- did=6 的部门没有被子表引用
delete from dept where did=1; -- 失败
-- did=1 的部门被子表引用
-- 删除子表的数据，不受约束
delete from emp where name='王五';
select * from emp;
-- 删除父表，受约束
drop table dept; -- 失败
-- 删除子表，不受约束
drop table emp;
-- 建表后添加外键约束
create table emp(
    id int primary key auto_increment,
    name varchar(20) not null,
    departmentid int
);
-- 给 emp 表(子表)增加外键约束
alter table emp add foreign key (departmentid) references dept(did);
-- 查看 emp 的约束信息
select * from information_schema.table_constraints where table_name='emp';
-- 键约束(主键、唯一键、外键)都会自动创建索引
-- 查看 emp 表的索引
show index from emp;
-- 主键字段索引名是 PRIMARY，删除主键时，会自动删除对应索引
-- 唯一键字段索引名是字段名，删除唯一键时，就是通过删除对应的索引方式来删除唯一键约束
-- 外键字段索引名是字段名，删除外键约束时，不会自动删除外键字段的索引，因为它们的命名不一样
-- 删除 emp 表的 departmentid 字段的外键约束
alter table emp drop foreign key emp_ibfk_1;
show index from emp;
```

```
-- 删除 emp 表的 departmentid 字段的索引
alter table emp drop index departmentid;
show index from emp;
```

13.8.7 设置外键约束等级

- Cascade 方式：在父表上 update/delete 记录时，同步 update/delete 子表的匹配记录
- Set null 方式：在父表上 update/delete 记录时，将子表上匹配记录的列设为 null，但是要注意子表的外键列不能为 not null

- No action 方式：如果子表中有匹配的记录,则不允许对父表对应候选键进行 update/delete 操作

- Restrict 方式：同 no action, 都是立即检查外键约束

如果没有指定等级，就相当于 Restrict 方式。

```
-- 创建父表
drop table if exists dept;
create table dept(
    did int primary key auto_increment,
    dname varchar(50) unique key not null
);
insert into dept values(null,'财务'),(null,'教学'),(null,'咨询'),(null,'后勤');
select * from dept;
-- 创建子表
drop table if exists emp;
create table emp(
    id int primary key auto_increment,
    name varchar(20) not null,
    departmentid int,
    foreign key (departmentid) references dept(did) on update cascade on delete set null
    -- on delete set null 要求 departmentid 字段没有 not null 约束
);
-- 添加子表记录
insert into emp values
(null,'张三',1),
(null,'李四',2),
(null,'王五',1);
select * from emp;
-- 修改父表被引用的 did 值
update dept set did = 6 where did=1;
-- 此时 did=1 的记录被子表引用，可以修改
-- 并且会同时修改子表的 departmentid=1 的字段值为 6，级联修改
```

```
select * from dept;
select * from emp;
-- 删除父表被引用的 did 的记录
delete from dept where did=6;
select * from dept;
select * from emp;
```

第 14 章 事务

14.1 事务的特点

事务处理：保证所有事务都作为一个工作单元来执行，即使出现了故障，都不能改变这种执行方式。当在一个事务中执行多个操作时，要么所有的事务都被提交(commit)，那么这些修改就永久地保存下来；要么数据库管理系统将放弃所作的所有修改，整个事务回滚(rollback)到最初状态。

事务的 ACID 属性：

- **原子性 (Atomicity)**：原子性是指事务是一个不可分割的工作单位，事务中的操作要么都发生，要么都不发生
- **一致性 (Consistency)**：事务必须使数据库从一个一致性状态变换到另外一个一致性状态
- **隔离性 (Isolation)**：事务的隔离性是指一个事务的执行不能被其他事务干扰，即一个事务内部的操作及使用的数据对并发的其他事务是隔离的，并发执行的各个事务之间不能互相干扰
- **持久性 (Durability)**：持久性是指一个事务一旦被提交，它对数据库中数据的改变就是永久性的，接下来的其他操作和数据库故障不应该对其有任何影响

原子性例如：

- 张三给李四转账 500
- 张三转账之前余额是 1000
- 李四转账之前余额是 1000
- 成功：张三账号变为 500，李四变为 1500
- 失败：张三账号是 1000，李四还是 1000

一致性例如：

- 张三给李四转账 500

- 张三转账之前余额是 1000
- 李四转账之前余额是 1000
- 要么他俩的余额不变， 还是 1000， 总和就是 2000
- 要么他俩的余额有变化， 张三 500， 李四 1500， 总和仍然是 2000
- 错误：
- 张三 500， 李四 1000， 总和是 1500， 结果不对
- 张三 1000， 李四 1500， 总和是 2500， 结果不对

隔离性例如：

- 张三要给李四转 500
- 王五要给李四转 500
- 张三转账是否成功， 和王五是否转账成功无关

持久性例如：

- 张三要给李四转 500， 一旦成功提交， 就转账成功， 撤不回来了

14.2 事务的开启、提交、回滚

MySQL 默认情况下是自动提交事务。每一条语句都是一个独立的事务，一旦成功就提交了。一条语句失败，单独一条语句不生效，其他语句是生效的。

14.2.1 手动提交模式

```
-- 开启手动提交事务模式
set autocommit=false;
-- 或
set autocommit=0;
-- 上述语句执行之后，它之后的所有 SQL，都需要手动提交才能生效，直到恢复自动提交模式
-- 恢复自动提交模式
set autocommit=true;
-- 或
set autocommit=1;
```

例如：

```
set autocommit=false; -- 设置当前连接为手动提交模式
update t_employee set salary = 15000 where ename='孙红雷';
commit; -- 提交
-- 如果没有提交，直接关了连接，那么修改不会生效
```

14.2.2 自动提交模式下开启事务

```
start transaction;
```

...

```
commit; 或 rollback;
```

...中的语句是属于手动提交模式，其他的仍然是自动提交模式。

```
start transaction; -- 开始事务
update t_employee set salary = 0 where ename='李冰冰';
commit(或 rollback); -- 提交或回滚
```

14.2.3 DDL 语句不支持事务

DDL: create, drop, alter 等创建库、创建表、删除库、删除表、修改库、修改表结构等这些语句不支持事务。

只对 insert, update, delete 语句支持事务。

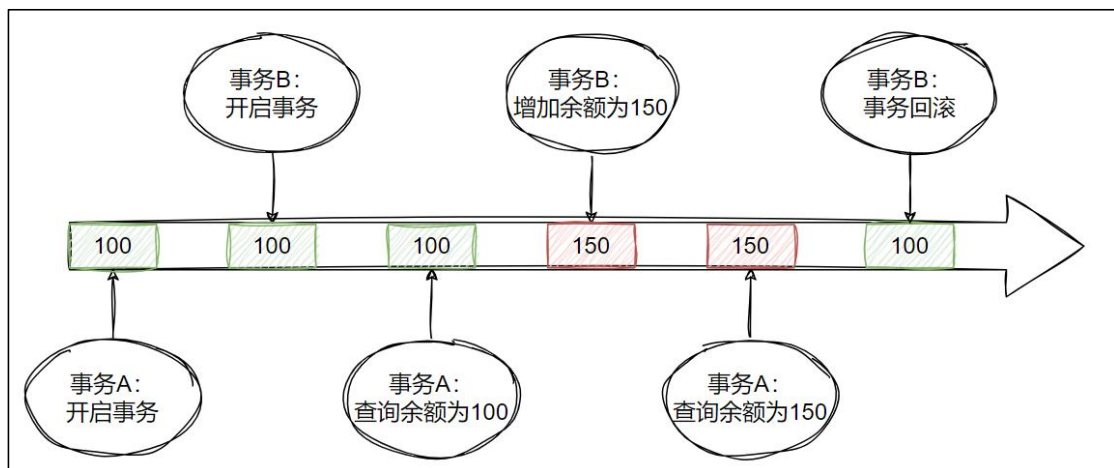
truncate 不支持事务。

14.3 事务的隔离级别

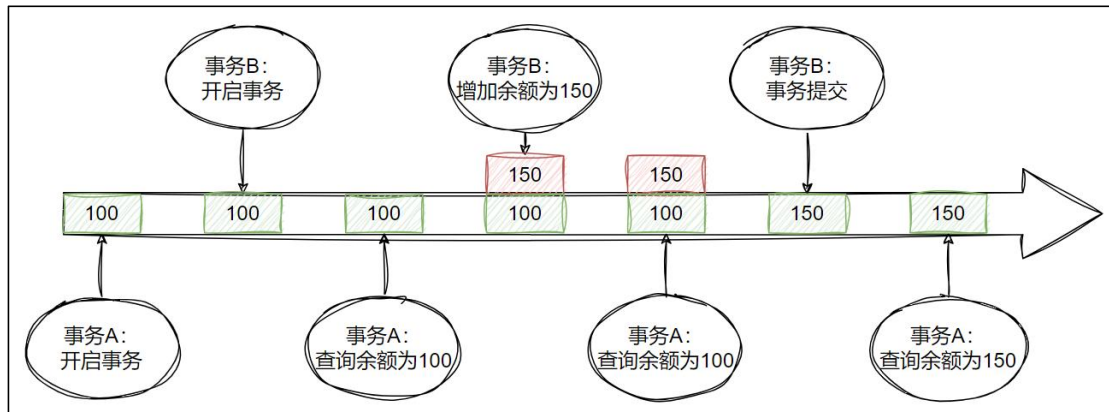
数据库事务的隔离性：数据库系统必须具有隔离并发运行各个事务的能力，使它们不会相互影响，避免各种并发问题。一个事务与其他事务隔离的程度称为隔离级别。数据库规定了多种事务隔离级别，不同隔离级别对应不同的干扰程度，隔离级别越高，数据一致性就越好，但并发性越弱。

14.3.1 并发问题

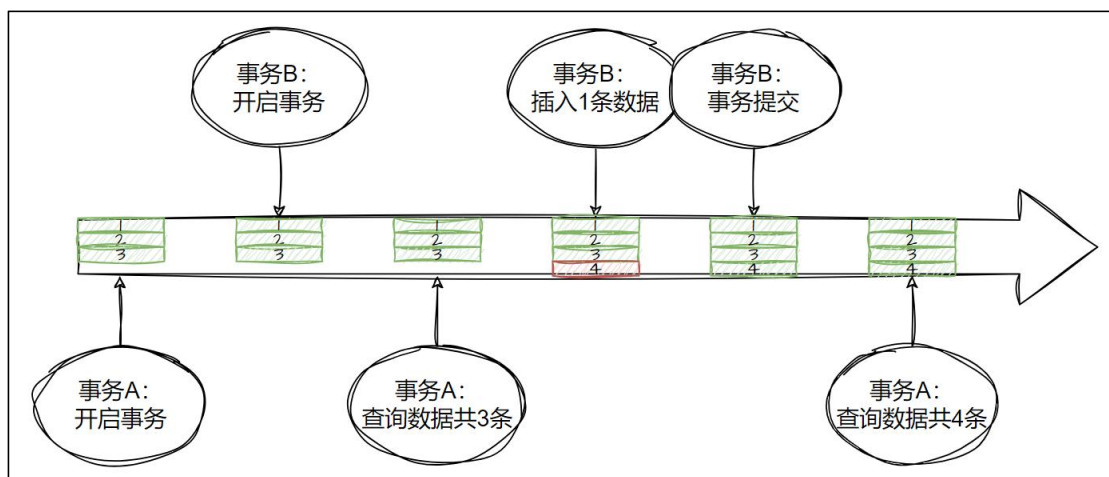
➤ 脏读：一个事务读取了另一个事务未提交数据。对于两个事务 T1、T2。T1 读取了已经被 T2 更新但还没有被提交的字段。之后，若 T2 回滚，T1 读取的内容就是临时且无效的。



➤ 不可重复读：同一个事务中前后两次读取同一条记录不一样。因为被其他事务修改了并且提交了。对于两个事务 T1、T2。T1 读取了一个字段，然后 T2 更新/修改了该字段。之后，T1 再次读取同一个字段，值就不同了。



➤ 幻读：一个事务读取了另一个事务新增、删除的记录情况，记录数不一样，像是出现幻觉。对于两个事务 T1、T2。T1 从一个表中读取了一个字段，然后 T2 在该表中插入/删除了一些新的行。之后，如果 T1 再次读取同一个表，就会多/少几行。



14.3.2 四种事务隔离级别

隔离级别	说明
read-uncommitted	允许 A 事务读取其他事务未提交和已提交的数据。 会出现脏读、不可重复读、幻读问题。
read-committed	只允许 A 事务读取其他事务已提交的数据。 可以避免脏读，但仍然会出现不可重复读、幻读问题。
repeatable-read	确保事务可以多次从一个字段中读取相同的值。在这个事务持续期间，禁止其他事务对这个字段进行更新。

	可以避免脏读和不可重复读。但是幻读问题仍然存在。注意：MySQL 中使用了 MVCC 多版本控制技术，在这个级别也可以避免幻读。
serializable	确保事务可以从一个表中读取相同的行，相同的记录。在这个事务持续期间，禁止其他事务对该表执行插入、更新、删除操作。 所有并发问题都可以避免，但性能十分低下。

14.3.3 修改与查看隔离级别

```
-- 先更改事务隔离级别 再开启事务
-- 修改隔离级别
set transaction_isolation='隔离级别';
-- 查看隔离级别
select @@transaction_isolation;
```

第 15 章 用户管理

15.1 用户管理的目标

15.1.1 登录验证

用户主机 IP 地址 + 用户名 + 密码三重验证。

用户主机 IP 是客户端主机的 IP，而非 MySQL 服务器的 IP。

IP 地址可以是一个明确的 IP(例如: 192.168.1.25)，可以是某个 IP 段(例如: 192.168.1.%)，可以是任意 IP 地址 (%)。例如假设 MySQL 服务器的 IP 地址是 192.168.31.152:

- 192.168.31.% 表示 192.168.31 网段内的客户端都可以连接到 192.168.31.152 机器上的 MySQL 服务。
- % 表示任意一台主机的客户端都可以连接到 192.168.31.152 机器上的 MySQL 服务。
- 192.168.31.23 表示只能从 192.168.31.23 主机的客户端连接到 192.168.31.152 机器上的 MySQL 服务。

15.1.2 权限管理

- 全局权限
- 数据库权限
- 数据表权限
- 字段权限
- 存储过程或函数子程序的权限

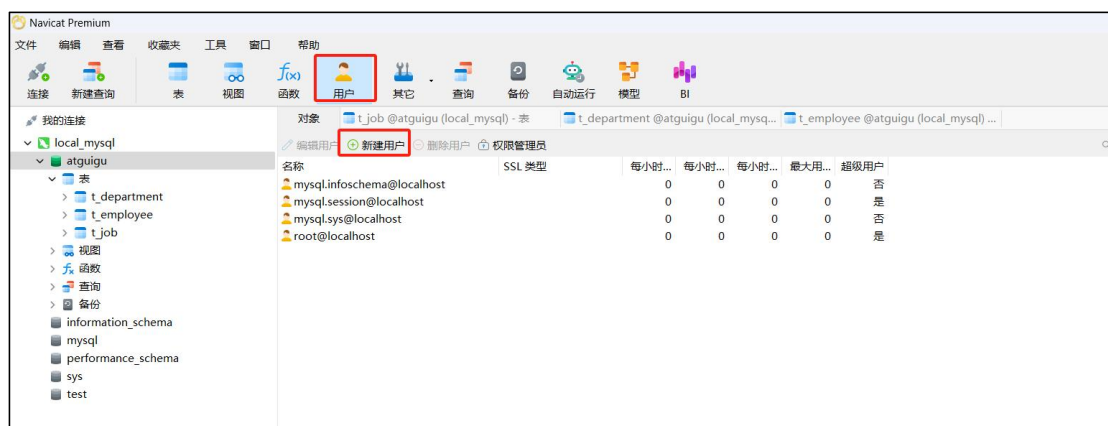
对用户的操作进行逐级权限验证，如果上一级有这个权限，下一级就不再验证。

15.2 用户管理演示

当具有权限管理的用户通过 Navicat 图形界面工具连接 MySQL 服务后，可以按照如下步骤进行用户和权限管理。

(1) 步骤 1

用户→可以选择新建用户，或者对已有用户进行编辑



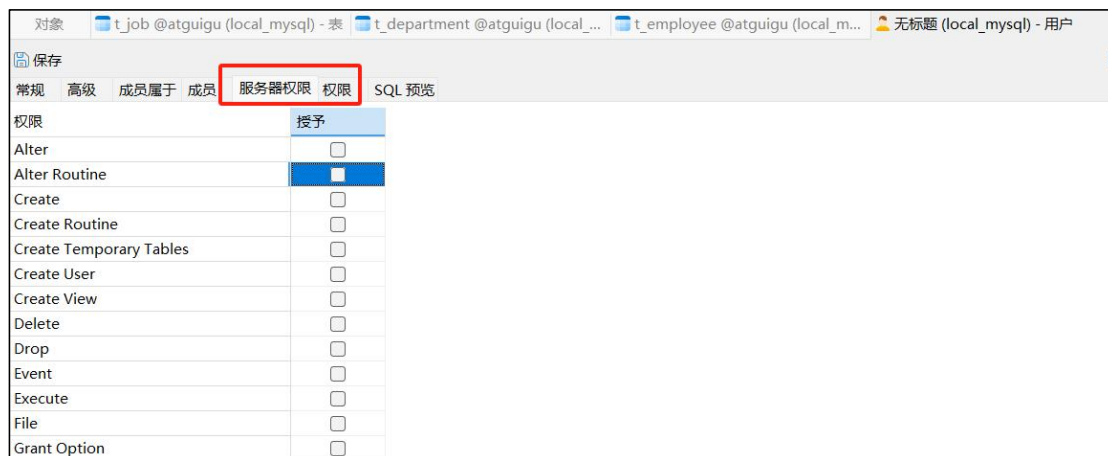
(2) 步骤 2

如果要创建新用户，选择“新建用户”按钮，弹出新用户信息填写窗口。用户名和主机文本框必须填写，其他项可以不填写，按照默认值处理。如果密码和再一次输入密码框为空，表示密码为空。如果要设置密码必须保证密码框和再一次输入密码框输入相同字符，并在 Plugin 选择合适的插件“caching_sha2_password”或“mysql_native_password”，默认是“caching_sha2_password”插件。



(3) 步骤 3

对已有的用户进行授权操作，或撤销已有用户的授权，可以在全限相关的选项卡中，对相应权限打勾或者取消打勾。



第 16 章 MySQL8 部分新特性

16.1 系统表全部为 InnoDB 表

从 MySQL 8.0 开始，mysql 系统表和数据字典表使用 InnoDB 存储引擎，存储在 MySQL 数据目录下的 mysql.libd 表空间文件中。在 MySQL 5.7 之前，这些系统表使用 MyISAM 存储引擎，存储在 MySQL 数据库文件目录下各自的表空间文件中。

查看系统表类型：

```
SELECT DISTINCT(ENGINE) FROM information_schema.tables;
```

ENGINE
NULL
InnoDB
CSV
PERFORMANCE_SCHEMA

16.2 默认字符集为 utf8mb4

在 8.0 版本之前，MySQL 默认的字符集为 Latin1，而 8.0 版本默认字符集为 utf8mb4。

Latin1 是 ISO-8859-1 的别名，有些环境下写作 Latin-1。ISO-8859-1 编码是单字节编码，不支持中文等多字节字符，但向下兼容 ASCII。

MySQL 中 utf8 字符集，它是 utf8mb3 的别称，使用三个字节编码表示一个字符。自 MySQL4.1 版本被引入，能够支持绝大多数语言的字符，但依然有些字符不能正确编码，如 emoji 表情字符等，为此 MySQL5.5 引入了 utf8mb4 字符集。在 MySQL5.7 对 utf8mb4 进行了大幅优化，并丰富了校验字符集。mb4 就是“most byte 4”的意思，专门用来兼容四字节的 Unicode，utf8mb4 编码是 utf8 编码的超集，兼容 utf8，并且能存储 4 字节的表情字符。

如果原来某些库和表的字符集是 utf8，可以直接修改为 utf8mb4，不需要做其他转换。但是从 utf8mb4 转回 utf8 就会有问题。

使用 SHOW 语句查看 MySQL8.0 版本数据库的默认编码：

```
SHOW VARIABLES LIKE 'character_set_database';
```

Variable_name	Value
character_set_database	utf8mb4

字符集校对规则是在字符集内用于字符比较和排序的一套规则，比如有的规则区分大小写，有的则无视。校对规则特征：

- 两个不同的字符集不能有相同的校对规则
- 每个字符集有一个默认校对规则
- 校对规则存在命名约定，以其相关的字符集名开始，中间包括一个语言名，并且以 _ci、_cs 或 _bin 结尾。其中 _ci 表示大小写不敏感、_cs 表示大小写敏感、bin 表示直接比较字符的二进制编码，即区分大小写

使用 SHOW 语句查看 utf8mb4 字符集的部分校对规则如下：

```
SHOW COLLATION LIKE 'utf8mb4_0900%';
```

Collation	Charset	Id	Default	Compiled	Sortlen	Pad_attribute
utf8mb4_0900_ai_ci	utf8mb4	255	Yes	Yes	0	NO PAD
utf8mb4_0900_as_ci	utf8mb4	305		Yes	0	NO PAD
utf8mb4_0900_as_cs	utf8mb4	278		Yes	0	NO PAD
utf8mb4_0900_bin	utf8mb4	309		Yes	1	NO PAD

16.3 用户管理

在 MySQL8 之前默认的身份插件是 “mysql_native_password”，即 MySQL 用户的密码使用 PASSWORD 函数进行加密。在 MySQL 8.x 中，默认的身份认证插件是 “caching_sha2_password”，替代了之前的 “mysql_native_password”，PASSWORD 函数被弃用了。

在 MySQL 版本 5.6.6 版本起，在 mysql.user 表中添加了 “password_expired” 字段，它允许设置密码是否失效。如果 “password_lifetime” 字段值不为 NULL，那么从 MySQL 服务启动时间开始，经过 “password_lifetime” 字段值的时间间隔之后，密码就过期了，即 “password_expired” 字段就为 “Y”。任何密码超期的账号想要连接服务器端进行数据库操作都必须更改密码。MySQL8.0 版本允许数据库管理员手动设置账户密码过期时间。

从 MySQL 8.x 版本开始允许限制重复使用以前的密码。

在 MySQL8 之前，如果要给多个用户授予相同的角色，需要为每个用户单独授权。在 MySQL8 之后，可以为多个用户赋予统一的角色，然后给角色授权即可，角色可以看成是一些权限的集合，这样就无须为每个用户单独授权。如果角色的权限修改，将会使得该角色下的所有用户的权限都跟着修改，这就非常方便。

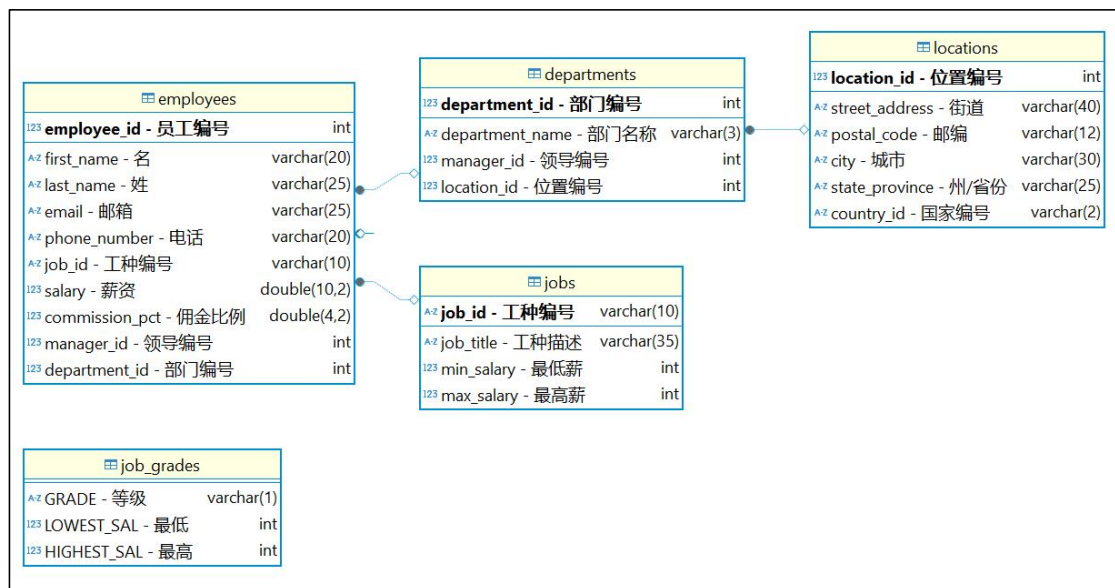
MySQL 的密码字段变化：

- MySQL5.7 之前 mysql 系统库的 user 表，密码字段名是 password
- MySQL5.7 版本 mysql 系统库的 user 表，密码字段名是 authentication_string
- MySQL8.0 版本 mysql 系统库的 user 表，密码字段名是 authentication_string，另外用户管理还有角色概念，mysql 系统库中有 default_roles 表

第 17 章 练习题

17.1 导入数据

导入 SQL 脚本“练习数据.sql”。



17.2 选择与过滤练习

- (1) 查询薪资大于 12000 的员工姓名和薪资
- (2) 查询员工号为 176 的员工的姓名和部门编号
- (3) 查询薪资不在 5000 到 12000 的员工的姓名和薪资
- (4) 查询在 20 或 50 号部门工作的员工姓名和部门编号

- (5) 查询公司中没有管理者的员工姓名及 job_id
- (6) 查询公司中有奖金的员工姓名，薪资和佣金
- (7) 查询员工姓名的第三个字母是 a 的员工姓名
- (8) 查询姓名中有字母 a 和 e 的员工姓名

17.3 多表查询练习

- (1) 查询所有员工的姓名，部门编号和部门名称
- (2) 查询 90 号部门员工的 job_id 和 90 号部门的位置编号
- (3) 查询所有有奖金的员工的姓名，部门名称，位置编号，城市
- (4) 查询 city 在 toronto 工作的员工的姓名,job_id,部门编号,部门名称
- (5) 查询指定员工的姓名，员工号，以及他的管理者的姓名和员工号，结果类似于下面的格式

employees	emp#	manager	mgr#
kochhar	101	king	100

17.4 单行函数练习

- (1) 查询系统时间(注：日期+时间)
- (2) 查询员工编号，姓名，薪资，以及薪资提高百分之 20%后的结果(new salary)
- (3) 将员工的姓名按首字母排序，并写出姓名的长度
- (4) 写一个查询，产生下面的结果

<last_name> earns <salary> monthly but wants <salary*3> dream salary

比如：king earns 24000 monthly but wants 72000

- (5) 使用 case-when

按照下面的条件：

- job grade
- ad_pres a
- st_man b
- it_prog c
- sa_rep d
- st_clerk e

产生下面的结果：

- last_name job_id grade
- king ad_pres a

17.5 分组函数练习

- (1) 查询公司员工薪资的最大值，最小值，平均值，总和
- (2) 查询各 job_id 的员工薪资的最大值，最小值，平均值，总和
- (3) 查询各个 job_id 的员工人数
- (4) 查询员工最高薪资和最低薪资的差距
- (5) 查询各个管理者手下员工的不低于 6000 的最低薪资，不计算没有管理者的员工
- (6) 查询所有部门的名字，位置编号，员工数量和薪资平均值

17.6 子查询练习

- (1) 查询和 zlotkey 相同部门的员工姓名和薪资
- (2) 查询薪资比公司平均薪资高的员工的员工编号，姓名和薪资
- (3) 查询各部门中薪资比本部门平均薪资高的员工的员工编号,姓名和薪资
- (4) 查询与姓名中包含字母 u 的员工在相同部门的员工的员工编号和姓名
- (5) 查询管理者是 king 的员工姓名和薪资
- (6) 查询薪资最低的员工的姓名和薪资

17.7 练习题参考答案

17.7.1 选择与过滤练习

```
-- 1.查询薪资大于 12000 的员工姓名和薪资
select first_name,salary
from employees
where salary > 12000;

-- 2.查询员工号为 176 的员工的姓名和部门编号
select first_name,department_id
from employees
where employee_id = 176;

-- 3.查询薪资不在 5000 到 12000 的员工的姓名和薪资
select first_name,salary
from employees
where salary not between 5000 and 12000;
```



```
-- 4.查询在 20 或 50 号部门工作的员工姓名和部门编号
select first_name,department_id
from employees
where department_id in (20, 50);

-- 5.查询公司中没有管理者的员工姓名及 job_id
select first_name,job_id
from employees
where manager_id is null;

-- 6.查询公司中有奖金的员工姓名，薪资和佣金
select first_name,salary,salary * commission_pct
from employees
where commission_pct is not null;

-- 7.查询员工姓名的第三个字母是 a 的员工姓名
select first_name
from employees
where first_name like '__a%';

-- 8.查询姓名中有字母 a 和 e 的员工姓名
select first_name
from employees
where first_name like '%a%' and first_name like '%e%';
```

17.7.2 多表查询练习

```
-- 1.查询所有员工的姓名，部门编号和部门名称
select last_name, e.department_id, department_name
from employees e
left outer join departments d
on e.department_id = d.department_id;

-- 2.查询 90 号部门员工的 job_id 和 90 号部门的位置编号
select job_id, location_id
from employees e
join departments d
on e.department_id = d.department_id
where e.department_id = 90;

-- 3.查询所有有奖金的员工的姓名，部门名称，位置编号，城市
select last_name,department_name,d.location_id,city
from employees e
left join departments d
on e.department_id = d.department_id
```

```
left join locations l
on d.location_id = l.location_id
where commission_pct is not null;

-- 4.查询 city 在 toronto 工作的员工的姓名,job_id,部门编号,部门名称
select last_name,job_id,e.department_id,department_name
from employees e
join departments d
on e.department_id = d.department_id
join locations l
on l.location_id = d.location_id
where l.city = 'toronto';

-- 5.查询指定员工的姓名,员工号,以及他的管理者的姓名和员工号,结果类似于下面的
格式
-- employees      emp#      manager mgr#
-- kochhar        101 king      100
select
    emp.last_name as employees,
    emp.employee_id as "emp#",
    mgr.last_name as manager,
    mgr.employee_id as "mgr#"
from employees emp
left join employees mgr
on emp.manager_id = mgr.employee_id;
```

17.7.3 单行函数练习

```
-- 1.查询系统时间(注: 日期+时间)
select now();

-- 2.查询员工编号,姓名,薪资,以及薪资提高百分之 20%后的结果(new salary)
select
    employee_id,
    first_name,
    salary,
    salary * 1.2 as new_salary
from employees;

-- 3.将员工的姓名按首字母排序,并写出姓名的长度
select first_name,char_length(first_name)
from employees
order by first_name;

-- 4.写一个查询,产生下面的结果
```

```
-- <last_name> earns <salary> monthly but wants <salary*3> dream salary
-- 比如: king earns 24000 monthly but wants 72000
select concat(
    last_name,
    ' earns ',
    salary,
    ' monthly but wants ',
    salary * 3,
    ' dream salary'
)
from employees;

-- 5.使用 case-when
-- 按照下面的条件:
-- job          grade
-- ad_pres      a
-- st_man       b
-- it_prog      c
-- sa_rep       d
-- st_clerk     e
-- 产生下面的结果:
-- last_name    job_id    grade
-- king        ad_pres    a
select
    last_name,
    job_id,
    case job_id
        when 'ad_pres' then 'a'
        when 'st_man' then 'b'
        when 'it_prog' then 'c'
        when 'sa_rep' then 'd'
        when 'st_clerk' then 'e'
        else 'x'
    end 'grade'
from employees;
```

17.7.4 分组函数练习

```
-- 1.查询公司员工薪资的最大值,最小值,平均值,总和
select max(salary),min(salary),avg(salary),sum(salary)
from employees;

-- 2.查询各 job_id 的员工薪资的最大值,最小值,平均值,总和
select job_id,max(salary),min(salary),avg(salary),sum(salary)
from employees
```

```
group by job_id;

-- 3.查询各个 job_id 的员工人数
select job_id,count(*)
from employees
group by job_id;

-- 4.查询员工最高薪资和最低薪资的差距
select max(salary) - min(salary)
from employees;

-- 5.查询各个管理者手下员工的不低于 6000 的最低薪资，不计算没有管理者的员工
select manager_id,min(salary)
from employees
where manager_id is not null
group by manager_id
having min(salary) > 6000;

-- 6.查询所有部门的名字，位置编号，员工数量和薪资平均值
select
    department_name,
    location_id,
    count(employee_id),
    avg(salary)
from employees e
right join departments d
on e.department_id = d.department_id
group by department_name,location_id;
```

17.7.5 子查询练习

```
-- 1.查询和 zlotkey 相同部门的员工姓名和薪资
select last_name,salary
from employees
where department_id = (
    select department_id
    from employees
    where last_name = 'zlotkey'
);

-- 2.查询薪资比公司平均薪资高的员工的员工编号，姓名和薪资
select employee_id,last_name,salary
from employees
where salary > (
    select avg(salary)
```

```
from employees
);

-- 3. 查询各部门中薪资比本部门平均薪资高的员工的员工编号,姓名和薪资
select employee_id,first_name,salary,avg_sal
from employees e1
join (
    select department_id,avg(salary) avg_sal
    from employees
    group by department_id
) e2
on e1.department_id = e2.department_id
where salary > avg_sal;

-- 4. 查询与姓名中包含字母 u 的员工在相同部门的员工的员工编号和姓名
select employee_id,last_name
from employees
where department_id in (
    select department_id
    from employees
    where last_name like '%u%'
);

-- 5. 查询管理者是 king 的员工姓名和薪资
select e1.first_name,e1.salary,e1.manager_id,e2.last_name
from employees e1
join employees e2
on e1.manager_id = e2.employee_id
where e1.manager_id in (
    select employee_id
    from employees
    where last_name = 'king'
);

-- 6. 查询薪资最低的员工的姓名和薪资
select last_name,salary
from employees
where salary = (
    select min(salary)
    from employees
);
```